COVERS ALL MAJOR SQL DATABASES

SQL PERFORMANCE EXPLAINED

ENGLISH EDITION



EVERYTHING DEVELOPERS NEED TO KNOW ABOUT SQL PERFORMANCE

MARKUS WINAND

Thỏa thuận cấp phép

Ebook này chỉ được cấp phép để bạn sử dụng cá nhân. Ebook này không được bán lại hoặc tặng cho người khác. Nếu muốn chia sẻ sách với người khác, vui lòng mua thêm một bản cho mỗi người. Nếu bạn đang đọc sách này nhưng không mua, hoặc sách không được mua chỉ để bạn sử dụng, vui lòng truy cập http://SQL-Performance-Explained.com/ và mua bản riêng của mình. Cảm ơn bạn đã tôn trọng công sức của tác giả.

Bản sao này được cấp phép cho:

GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Nhà xuất bản:

Markus Winand

Maderspergerstasse 1-3/9/11

1160 Wien

AUSTRIA

<office@winand.at>

Bản quyền © 2012 Markus Winand

Bảo lưu mọi quyền. Không phần nào của ấn phẩm này được sao chép, lưu trữ hoặc truyền đi dưới bất kỳ hình thức hay phương tiện nào — điện tử, cơ học, sao chụp, ghi âm hoặc cách khác — nếu chưa có sự đồng ý trước của nhà xuất bản.

Nhiều tên được nhà sản xuất và người bán sử dụng để phân biệt sản phẩm của họ là nhãn hiệu đã đăng ký. Khi các tên gọi đó xuất hiện trong sách này và chúng tôi biết có yêu cầu về nhãn hiệu, chúng được in bằng toàn chữ hoa hoặc chữ cái đầu viết hoa.

Dù đã thực hiện mọi biện pháp cẩn trọng khi chuẩn bị cuốn sách này, nhà xuất bản và tác giả không chịu trách nhiệm về lỗi, thiếu sót hoặc thiệt hại phát sinh từ việc sử dụng thông tin trong sách. Cuốn sách chỉ phản ánh

quan điểm của tác giả. Các nhà cung cấp database được đề cập không hỗ trợ tài chính cho công trình này và cũng không xác minh nội dung.

DGS - Druck- u. Graphikservice GmbH — Wien — Austria

Thiết kế bìa:

tomasio.design — Mag. Thomas Weninger — Wien — Austria

Ảnh bìa:

Brian Arnold — Turriff — UK

Biên tập viên:

Nathan Ingvalson — Graz — Austria

2014-08-26

SQL Performance Explained

Everything developers need to know about SQL performance

Markus Winand
Vienna, Austria

Contents

Preface vi

1. Anatomy of an Index 1
    The Index Leaf Nodes 2
    The Search Tree (B-Tree) 4
    Slow Indexes, Part I 6

2. The Where Clause 9
    The Equality Operator 9
        Primary Keys 10
        Concatenated Indexes 12
        Slow Indexes, Part II 18
    Functions 24
        Case-Insensitive Search Using UPPER or LOWER 24
        User-Defined Functions 29
        Over-Indexing 31
    Parameterized Queries 32
    Searching for Ranges 39
        Greater, Less and BETWEEN 39
        Indexing LIKE Filters 45
        Index Merge 49
    Partial Indexes 51
    NULL in the Oracle Database 53
        Indexing NULL 54
        NOT NULL Constraints 56
        Emulating Partial Indexes 60
    Obfuscated Conditions 62
        Date Types 62
        Numeric Strings 68
        Combining Columns 70
        Smart Logic 72
        Math 77

3. Performance and Scalability 79
Performance Impacts of Data Volume 80
Performance Impacts of System Load 85
Response Time and Throughput 87

4. The Join Operation 91
Nested Loops 92
Hash Join 101
Sort Merge 109

5. Clustering Data 111
Index Filter Predicates Used Intentionally 112
Index-Only Scan 116
Index-Organized Tables 122

6. Sorting and Grouping 129
Indexing Order By 130
Indexing `ASC`, `DESC` and `NULLS FIRST`/`LAST` 134
Indexing Group By 139

7. Partial Results 143
Querying Top-N Rows 143
Paging Through Results 147
Using Window Functions for Pagination 156

8. Modifying Data 159
Insert 159
Delete 162
Update 163

A. Execution Plans 165
Oracle Database 166
PostgreSQL 172
SQL Server 180
MySQL 188

Index 193

Lời nói đầu
Developer Cần Biết Cách Đánh Index

Các vấn đề SQL performance đã tồn tại từ khi SQL xuất hiện, thậm chí có người cho rằng SQL vốn dĩ chậm. Điều này có thể đúng vào thời kỳ đầu của SQL, nhưng hiện nay chắc chắn không còn đúng. Tuy vậy, vấn đề SQL performance vẫn rất phổ biến. Tại sao lại như vậy?

Ngôn ngữ SQL có lẽ là ngôn ngữ lập trình thế hệ thứ tư (4GL) thành công nhất. Lợi ích chính của nó là khả năng tách biệt "what" và "how". Một câu lệnh SQL mô tả trực tiếp điều cần có mà không chỉ dẫn cách thực hiện. Hãy xem ví dụ sau:

```
SELECT date_of_birth
  FROM employees
 WHERE last_name = 'WINAND'
```

SQL query này đọc như một câu tiếng Anh giải thích dữ liệu được yêu cầu. Việc viết câu lệnh SQL nói chung không đòi hỏi kiến thức về cách database hoặc storage system vận hành bên trong (như disk, file, v.v.). Không cần chỉ cho database biết phải mở file nào hay tìm các row được yêu cầu ra sao. Nhiều developer có nhiều năm kinh nghiệm SQL nhưng biết rất ít về quá trình xử lý diễn ra trong database.

Việc tách biệt mối quan tâm — cần gì so với cách lấy được nó — hoạt động rất tốt trong SQL, nhưng vẫn chưa hoàn hảo. Sự trừu tượng hóa chạm giới hạn khi xét đến performance: theo định nghĩa, tác giả của một câu lệnh SQL không quan tâm database thực thi câu lệnh như thế nào. Do đó, tác giả không chịu trách nhiệm cho việc thực thi chậm. Tuy nhiên, kinh nghiệm cho thấy điều ngược lại; tức là tác giả phải biết một chút về database để tránh các vấn đề performance.

Hóa ra điều duy nhất developer cần học là cách đánh index. Database indexing thực chất là một công việc development. Lý do là thông tin quan trọng nhất để đánh index đúng không phải cấu hình storage system hay thiết lập hardware. Thông tin quan trọng nhất là application query dữ liệu như thế nào. Kiến thức này — về access path — không dễ tiếp cận đối với database administrator (DBA) hoặc consultant bên ngoài. Cần khá nhiều thời gian để thu thập thông tin đó bằng cách reverse engineering application; ngược lại, bộ phận development vốn đã có thông tin này.

Cuốn sách trình bày mọi điều developer cần biết về index — và không hơn. Chính xác hơn, sách chỉ đề cập loại index quan trọng nhất: B-tree index.

B-tree index hoạt động gần như giống hệt nhau trong nhiều database. Sách chỉ sử dụng thuật ngữ của Oracle® database, nhưng các nguyên lý cũng áp dụng cho database khác. Các ghi chú bên lề cung cấp thông tin liên quan về MySQL, PostgreSQL và SQL Server®.

Cấu trúc sách được thiết kế riêng cho developer; đa số chương tương ứng với một phần cụ thể của câu lệnh SQL.

CHƯƠNG 1 - Giải phẫu một Index

Chương đầu là chương duy nhất không đề cập riêng đến SQL; chương này nói về cấu trúc nền tảng của index. Hiểu cấu trúc index là điều thiết yếu để theo dõi các chương sau — đừng bỏ qua phần này! Dù chương này khá ngắn — chỉ khoảng tám trang — sau khi học xong, bạn sẽ hiểu hiện tượng index chậm.

CHƯƠNG 2 - Where Clause

Đây là nơi chúng ta khai thác toàn bộ vấn đề. Chương này giải thích mọi khía cạnh của where clause, từ lookup một column rất đơn giản đến các clause phức tạp cho range và những trường hợp đặc biệt như LIKE. Chương này là phần chính của cuốn sách. Khi đã học cách dùng các kỹ thuật này, bạn sẽ viết SQL nhanh hơn nhiều.

CHƯƠNG 3 - Performance và Scalability

Chương này là một phần đi sâu ngắn về đo lường performance và database scalability. Hãy xem vì sao thêm hardware không phải giải pháp tốt nhất cho query chậm.

CHƯƠNG 4 - Join Operation

Trở lại với SQL: ở đây bạn sẽ thấy cách dùng index để thực hiện table join nhanh.

CHƯƠNG 5 - Clustering Data

Bạn từng tự hỏi liệu chọn một column hay tất cả column có khác biệt gì không? Đây là câu trả lời — cùng một thủ thuật để có performance tốt hơn nữa.

CHƯƠNG 6 - Sorting và Grouping

Ngay cả order by và group by cũng có thể dùng index.

CHƯƠNG 7 - Partial Results

Chương này giải thích cách hưởng lợi từ execution “pipelined” nếu bạn không cần toàn bộ result set.

CHƯƠNG 8 - Insert, Delete và Update

Index ảnh hưởng write performance thế nào? Index không miễn phí — hãy dùng chúng một cách hợp lý!

PHỤ LỤC A - Execution Plans

Hỏi database cách nó thực thi một câu lệnh.

Chương 1
Giải phẫu một Index

“Index làm query nhanh” là lời giải thích cơ bản nhất về index mà tôi từng thấy. Dù nó mô tả rất tốt khía cạnh quan trọng nhất của index, nhưng đáng tiếc là chưa đủ cho cuốn sách này. Chương này

mô tả cấu trúc index theo cách bớt hời hợt hơn nhưng không đi quá sâu vào chi tiết. Nó cung cấp vừa đủ insight để hiểu các khía cạnh SQL performance được thảo luận xuyên suốt sách.

Index là một cấu trúc riêng biệt trong database, được tạo bằng câu lệnh create index. Nó cần disk space riêng và chứa một bản sao dữ liệu table đã được index. Điều đó có nghĩa index hoàn toàn là sự dư thừa. Việc tạo index không thay đổi dữ liệu table; nó chỉ tạo một data structure mới tham chiếu đến table. Suy cho cùng, database index rất giống index ở cuối sách: nó chiếm không gian riêng, có tính dư thừa cao và tham chiếu đến thông tin thực được lưu ở một nơi khác.

> Clustered Index
>
> SQL Server và MySQL (dùng InnoDB) có cách hiểu rộng hơn về "index". Chúng gọi các table chỉ gồm cấu trúc index là clustered index. Trong Oracle database, các table này được gọi là Index-Organized Tables (IOT). Chương 5, "Clustering Data", mô tả chúng chi tiết hơn và giải thích ưu điểm, nhược điểm của chúng.

Tìm kiếm trong database index giống tìm kiếm trong danh bạ điện thoại in sẵn. Khái niệm cốt lõi là mọi entry được sắp xếp theo một thứ tự xác định. Tìm dữ liệu trong data set có thứ tự rất nhanh và dễ vì thứ tự sắp xếp quyết định vị trí của mỗi entry.

Tuy nhiên, database index phức tạp hơn danh bạ in sẵn vì nó liên tục thay đổi. Không thể cập nhật danh bạ in sẵn cho từng thay đổi vì đơn giản là không có khoảng trống giữa các entry hiện hữu để thêm entry mới. Danh bạ in sẵn tránh vấn đề này bằng cách chỉ xử lý các cập nhật tích lũy ở lần in tiếp theo. SQL database không thể chờ lâu như vậy. Nó phải xử lý ngay các câu lệnh insert, delete và update, đồng thời giữ thứ tự index mà không di chuyển lượng lớn dữ liệu.

Database kết hợp hai data structure để đáp ứng thách thức này: doubly linked list và search tree. Hai cấu trúc này giải thích phần lớn đặc tính performance của database.

Các Leaf Node của Index

Mục đích chính của index là cung cấp một biểu diễn có thứ tự của dữ liệu đã được index. Tuy nhiên, không thể lưu dữ liệu tuần tự vì một câu lệnh insert sẽ phải di chuyển các entry phía sau để nhường chỗ cho entry mới. Di chuyển lượng lớn dữ liệu rất tốn thời gian, nên câu lệnh insert sẽ rất chậm. Giải pháp là thiết lập một thứ tự logic độc lập với thứ tự vật lý trong memory.

Thứ tự logic được thiết lập bằng doubly linked list. Mỗi node có link tới hai entry lân cận, rất giống một sợi xích. Node mới được chèn giữa hai node hiện có bằng cách cập nhật link của chúng để tham chiếu đến node mới. Vị trí vật lý của node mới không quan trọng vì doubly linked list duy trì thứ tự logic.

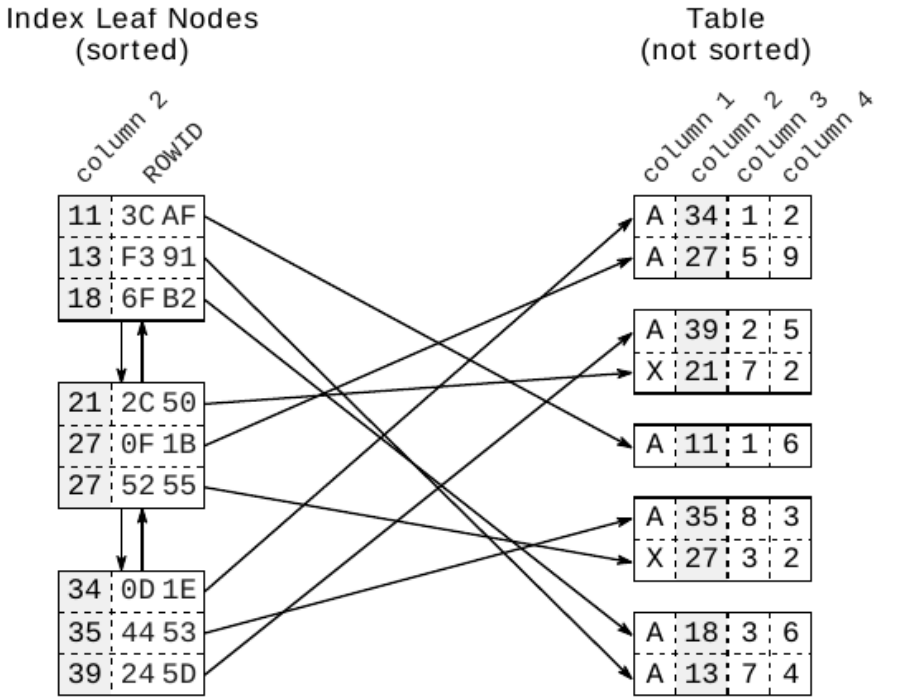
Data structure này được gọi là doubly linked list vì mỗi node tham chiếu đến node đứng trước và node đứng sau. Nó cho phép database đọc index theo chiều xuôi hoặc ngược khi cần. Nhờ vậy có thể chèn entry mới mà không di chuyển lượng lớn dữ liệu — chỉ cần thay đổi một số pointer.

Doubly linked list cũng được dùng cho collection (container) trong nhiều ngôn ngữ lập trình.

Programming Language	Name
Java	java.util.LinkedList
.NET Framework	System.Collections.Generic.LinkedList
C++	std::list

Database dùng doubly linked list để kết nối các index leaf node. Mỗi leaf node được lưu trong một database block hoặc page, tức đơn vị lưu trữ nhỏ nhất của database. Mọi index block có cùng kích thước — thường là vài kilobyte. Database tận dụng không gian trong từng block tối đa có thể và lưu nhiều index entry nhất có thể vào mỗi block. Điều đó có nghĩa thứ tự index được duy trì ở hai cấp: các index entry trong từng leaf node và các leaf node với nhau bằng doubly linked list.

Hình 1.1. Các Index Leaf Node và Dữ liệu Table Tương ứng



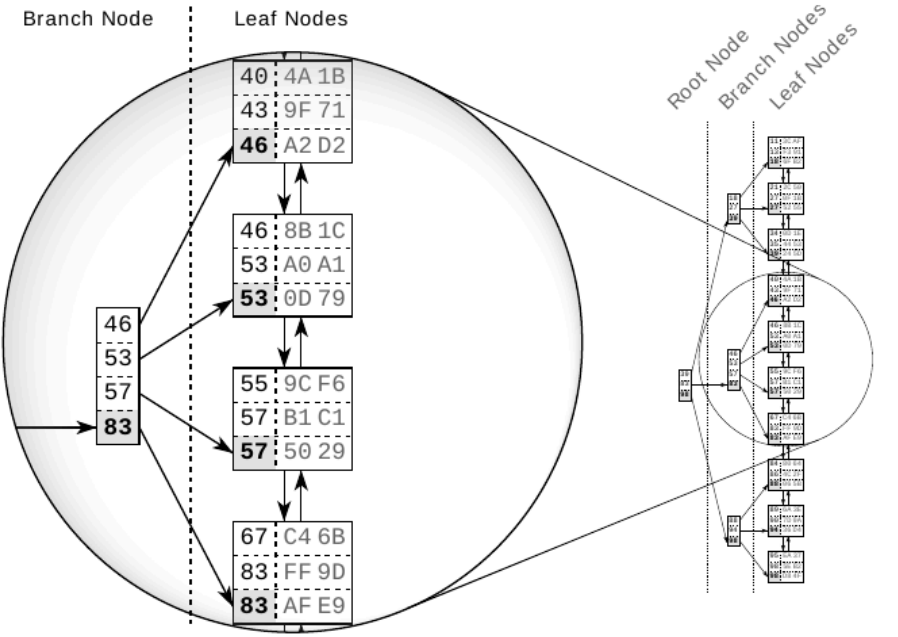
Hình 1.1 minh họa các index leaf node và kết nối của chúng với dữ liệu table. Mỗi index entry gồm các column được index (key, column 2) và tham chiếu đến row tương ứng trong table (qua ROWID hoặc RID). Khác với index, dữ liệu table được lưu trong cấu trúc heap và hoàn toàn không được sắp xếp. Không có quan hệ nào giữa các row được lưu trong cùng một table block, cũng không có kết nối nào giữa các block.

Search Tree (B-Tree)

Các index leaf node được lưu theo thứ tự tùy ý — vị trí trên disk không tương ứng với vị trí logic theo thứ tự index. Nó giống một danh bạ điện thoại có các trang bị xáo trộn. Nếu bạn tìm “Smith” nhưng mở danh bạ ở “Robinson” trước, hoàn toàn không đảm bảo

Smith nằm sau Robinson. Database cần một cấu trúc thứ hai để nhanh chóng tìm entry giữa các trang bị xáo trộn: balanced search tree, gọi ngắn gọn là B-tree.

Hình 1.2. Cấu trúc B-tree



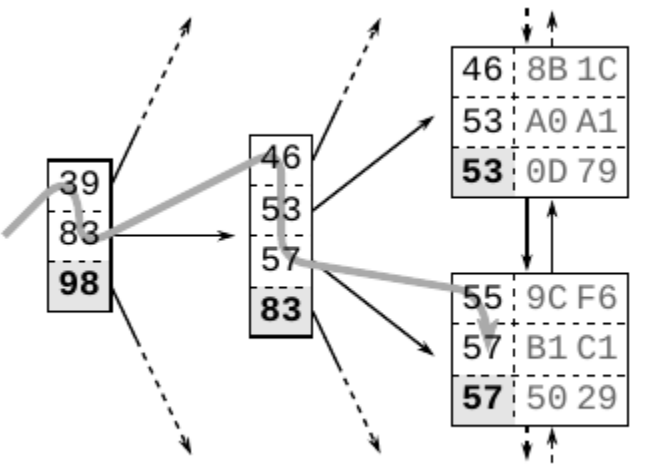
Hình 1.2 minh họa một index mẫu có 30 entry. Doubly linked list thiết lập thứ tự logic giữa các leaf node. Root node và branch node hỗ trợ tìm kiếm nhanh giữa các leaf node.

Hình làm nổi bật một branch node và các leaf node mà nó tham chiếu đến. Mỗi entry của branch node tương ứng với giá trị lớn nhất trong leaf node tương ứng. Tức là 46 trong leaf node đầu tiên, nên entry branch node đầu tiên cũng là 46. Điều này cũng đúng với các leaf node khác, do đó cuối cùng branch node có các giá trị 46, 53, 57 và 83. Theo sơ đồ này, một branch layer được xây dựng cho đến khi tất cả leaf node được một branch node bao phủ. Layer tiếp theo được xây dựng tương tự nhưng nằm trên level branch node đầu tiên. Quy trình lặp lại cho đến khi mọi key vừa trong một node duy

nhất là root node. Cấu trúc này là balanced search tree vì tree depth bằng nhau tại mọi vị trí; khoảng cách giữa root node và leaf node là như nhau ở mọi nơi.

> Lưu ý
>
> B-tree là balanced tree, không phải binary tree.
>
> Sau khi được tạo, database tự động duy trì index. Nó áp dụng mọi thao tác insert, delete và update vào index, đồng thời giữ tree cân bằng, do đó tạo maintenance overhead cho write operation. Chương 8, “Modifying Data”, giải thích chi tiết hơn.

Hình 1.3. Duyệt B-Tree



Hình 1.3 minh họa một phần index để giải thích việc tìm key “57”. Tree traversal bắt đầu từ root node ở phía bên trái. Mỗi entry được xử lý theo thứ tự tăng dần cho đến khi có giá trị lớn hơn hoặc bằng (>=) search term (57). Trong hình, đó là entry 83. Database đi theo tham chiếu đến branch node tương ứng và lặp lại quy trình cho đến khi tree traversal đến leaf node.

> Quan trọng
>
> B-tree cho phép database nhanh chóng tìm một leaf node.

Tree traversal là operation rất hiệu quả — hiệu quả đến mức tôi gọi nó là sức mạnh thứ nhất của indexing. Nó hoạt động gần như tức thì, ngay cả trên data set khổng lồ. Điều đó chủ yếu nhờ tree balance, cho phép truy cập mọi phần tử với cùng số bước, và thứ hai là nhờ tree depth tăng theo logarit. Nghĩa là tree depth tăng rất chậm so với số leaf node. Index trong thực tế với hàng triệu record có tree depth là bốn hoặc năm. Tree depth là sáu hầu như không thấy. Khung "Logarithmic Scalability" mô tả chi tiết hơn.

Index Chậm, Phần I

Dù tree traversal hiệu quả, vẫn có trường hợp index lookup không nhanh như mong đợi. Mâu thuẫn này đã nuôi dưỡng huyền thoại về "degenerated index" trong thời gian dài. Huyền thoại này coi index rebuild là giải pháp thần kỳ. Lý do thực sự khiến các statement đơn giản có thể chậm, ngay cả khi dùng index, có thể được giải thích dựa trên các phần trước.

Thành phần đầu tiên của index lookup chậm là leaf node chain. Hãy xét lại việc tìm "57" ở Hình 1.3. Rõ ràng index có hai entry khớp. Chính xác hơn, có ít nhất hai entry giống nhau: leaf node tiếp theo có thể còn entry khác cho "57". Database phải đọc leaf node tiếp theo để xem còn entry khớp nào không. Nghĩa là index lookup không chỉ cần tree traversal mà còn phải đi theo leaf node chain.

Thành phần thứ hai là truy cập table. Ngay cả một leaf node cũng có thể chứa nhiều hit — thường là hàng trăm. Dữ liệu table tương ứng thường phân tán trên nhiều table block (xem Hình 1.1, "Index Leaf Nodes and Corresponding Table Data"). Điều đó có nghĩa mỗi hit cần thêm một lần table access.

Index lookup cần ba bước: (1) tree traversal; (2) đi theo leaf node chain; (3) fetch dữ liệu table. Tree traversal là bước duy nhất có giới hạn trên về số block được truy cập — index depth. Hai bước còn lại có thể cần truy cập nhiều block — chúng gây ra index lookup chậm.

Khả năng Mở rộng Logarit

Trong toán học, logarit của một số theo một cơ số cho trước là lũy thừa hoặc số mũ mà cơ số phải được nâng lên để tạo ra số đó [Wikipedia 1].

Trong search tree, cơ số tương ứng với số entry trên mỗi branch node và số mũ tương ứng với tree depth. Index ví dụ ở Hình 1.2 chứa tối đa bốn entry mỗi node và có tree depth là ba. Điều đó nghĩa là index có thể chứa tối đa 64 (4^3) entry. Nếu tăng thêm một level, nó đã có thể chứa 256 entry (4^4). Mỗi khi thêm một level, số index entry tối đa tăng gấp bốn. Logarit đảo ngược hàm này. Vì vậy tree depth là log4(number-of-index-entries).

Sự tăng trưởng logarit cho phép index ví dụ tìm kiếm một triệu record với mười tree level, nhưng index thực tế còn hiệu quả hơn. Yếu tố chính ảnh hưởng tree depth, và do đó ảnh hưởng lookup performance, là số entry trong mỗi tree node. Về mặt toán học, số này tương ứng với cơ số của logarit. Cơ số càng lớn, tree càng nông, traversal càng nhanh.

Tree Depth	Index Entries
3	64
4	256
5	1,024
6	4,096
7	16,384
8	65,536
9	262,144
10	1,048,576

Database khai thác khái niệm này tối đa và đưa nhiều entry nhất có thể vào mỗi node — thường là hàng trăm. Điều đó có nghĩa mỗi index level mới hỗ trợ số entry nhiều gấp một trăm lần.

1

http://en.wikipedia.org/wiki/Logarithm

Nguồn gốc của huyền thoại “index chậm” là quan niệm sai rằng index lookup chỉ duyệt tree, từ đó có ý tưởng rằng index chậm hẳn do tree bị “hỏng” hoặc “mất cân bằng”. Sự thật là bạn có thể hỏi hầu

hết database cách chúng dùng một index. Oracle database mô tả khá chi tiết về điều này và có ba operation riêng biệt mô tả một index lookup cơ bản:

INDEX UNIQUE SCAN

INDEX UNIQUE SCAN chỉ thực hiện tree traversal. Oracle database dùng operation này khi unique constraint bảo đảm search criteria khớp không quá một entry.

INDEX RANGE SCAN

INDEX RANGE SCAN thực hiện tree traversal và đi theo leaf node chain để tìm mọi entry khớp. Đây là operation dự phòng khi nhiều entry có thể khớp search criteria.

TABLE ACCESS BY INDEX ROWID

Operation TABLE ACCESS BY INDEX ROWID lấy row từ table. Operation này thường được thực hiện cho mọi record khớp từ operation index scan đứng trước.

Điểm quan trọng là INDEX RANGE SCAN có thể đọc một phần lớn index. Nếu mỗi row cần thêm một table access, query có thể chậm dù đang dùng index.

Chương 2
Where Clause

Chương trước đã mô tả cấu trúc index và giải thích nguyên nhân index performance kém. Bước tiếp theo, chúng ta sẽ học cách nhận diện và tránh các vấn đề này trong câu lệnh SQL. Chúng ta bắt đầu bằng where clause.

Where clause định nghĩa điều kiện tìm kiếm của một câu lệnh SQL, vì vậy nó thuộc phạm vi chức năng cốt lõi của index: tìm dữ liệu nhanh. Dù where clause có ảnh hưởng rất lớn đến performance, nó

thường được viết cẩu thả khiến database phải scan một phần lớn index. Kết quả là where clause viết kém là thành phần đầu tiên của query chậm.

Chương này giải thích các operator khác nhau ảnh hưởng việc sử dụng index thế nào và cách bảo đảm index dùng được cho nhiều query nhất có thể. Phần cuối trình bày các anti-pattern thường gặp và các lựa chọn thay thế có performance tốt hơn.

Equality Operator

Equality operator vừa là SQL operator đơn giản nhất vừa được dùng thường xuyên nhất. Các lỗi indexing ảnh hưởng performance vẫn rất phổ biến, và where clause kết hợp nhiều điều kiện đặc biệt dễ gặp lỗi.

Phần này chỉ ra cách xác minh việc dùng index và giải thích cách concatenated index có thể tối ưu các điều kiện kết hợp. Để dễ hiểu, chúng ta sẽ phân tích một query chậm để thấy tác động trong thực tế của những nguyên nhân đã giải thích ở Chương 1.

Primary Key

Chúng ta bắt đầu với where clause đơn giản nhất nhưng phổ biến nhất: primary key lookup. Trong các ví dụ xuyên suốt chương này, chúng ta dùng table EMPLOYEES được định nghĩa như sau:

```
CREATE TABLE employees (
   employee_id   NUMBER         NOT NULL,
   first_name    VARCHAR2(1000) NOT NULL,
   last_name     VARCHAR2(1000) NOT NULL,
   date_of_birth DATE           NOT NULL,
   phone_number  VARCHAR2(1000) NOT NULL,
   CONSTRAINT employees_pk PRIMARY KEY (employee_id)
);
```

Database tự động tạo index cho primary key. Điều đó có nghĩa có index trên column EMPLOYEE_ID, dù không có câu lệnh create index.

Query sau dùng primary key để lấy tên của một nhân viên:

```
SELECT first_name, last_name
  FROM employees
 WHERE employee_id = 123
```

Where clause không thể khớp nhiều row vì primary key constraint bảo đảm tính duy nhất của các giá trị EMPLOYEE_ID. Database không cần đi theo index leaf node — chỉ cần duyệt index tree. Chúng ta có thể dùng execution plan để xác minh:

```
-------------------------------------------------------------------
|Id |Operation                    | Name         | Rows | Cost |
-------------------------------------------------------------------
| 0 |SELECT STATEMENT             |              |    1 |    2 |
| 1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES    |    1 |    2 |
|*2 |  INDEX UNIQUE SCAN          | EMPLOYEES_PK |    1 |    1 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   2 - access("EMPLOYEE_ID"=123)
```

Primary Key

Execution plan của Oracle cho thấy INDEX UNIQUE SCAN - operation chỉ duyệt index tree. Operation này tận dụng đầy đủ khả năng mở rộng theo logarit của index để tìm entry rất nhanh, gần như không phụ thuộc vào kích thước table.

> Mẹo
>
> Execution plan (đôi khi gọi là explain plan hoặc query plan) cho biết các bước database thực hiện để chạy một SQL statement. Appendix A ở trang 165 giải thích cách lấy và đọc execution plan trong các database khác.

Sau khi truy cập index, database phải thực hiện thêm một bước để lấy dữ liệu được query (FIRST_NAME, LAST_NAME) từ table storage: operation TABLE ACCESS BY INDEX ROWID. Operation này có thể trở thành bottleneck về performance, như đã giải thích trong "Slow Indexes, Part I", nhưng không có rủi ro đó khi dùng INDEX UNIQUE SCAN. Operation này không thể trả về quá một entry nên không thể kích hoạt quá một lần table access. Điều đó nghĩa là các yếu tố tạo nên một query chậm không xuất hiện với INDEX UNIQUE SCAN.

Primary Key Không Có Unique Index

Primary key không nhất thiết phải cần unique index - bạn cũng có thể dùng non-unique index. Trong trường hợp đó, Oracle database không dùng INDEX UNIQUE SCAN mà dùng operation INDEX RANGE SCAN. Tuy vậy, constraint vẫn duy trì tính duy nhất của key, nên index lookup trả về nhiều nhất một entry.

Một lý do dùng non-unique index cho primary key là deferrable constraint. Khác với regular constraint được validate trong khi thực thi statement, database hoãn việc validate deferrable constraint cho

đến khi transaction được commit. Deferred constraint cần thiết để insert dữ liệu vào các table có circular dependency.

Concatenated Index

Dù database tự động tạo index cho primary key, vẫn có chỗ để tinh chỉnh thủ công nếu key gồm nhiều column. Trong trường hợp đó, database tạo index trên tất cả primary key column - gọi là concatenated index (còn gọi là multicolumn, composite hoặc combined index). Lưu ý rằng thứ tự column của concatenated index ảnh hưởng lớn đến khả năng sử dụng, nên phải được chọn cẩn thận.

Để minh họa, giả sử có một vụ sáp nhập công ty. Nhân viên của công ty kia được thêm vào table EMPLOYEES của chúng ta, khiến table lớn gấp mười lần. Chỉ có một vấn đề: EMPLOYEE_ID không còn duy nhất trên cả hai công ty. Chúng ta cần mở rộng primary key bằng một identifier bổ sung, chẳng hạn subsidiary ID. Vì vậy primary key mới có hai column: EMPLOYEE_ID như trước và SUBSIDIARY_ID để khôi phục tính duy nhất.

Vì vậy, index cho primary key mới được định nghĩa như sau:

```
CREATE UNIQUE INDEX employee_pk
    ON employees (employee_id, subsidiary_id);
```

Query cho một nhân viên cụ thể phải xét đến toàn bộ primary key, tức là cũng phải dùng column SUBSIDIARY_ID:

```
SELECT first_name, last_name
  FROM employees
 WHERE employee_id   = 123
   AND subsidiary_id = 30;
```

```
-------------------------------------------------------------------
|Id |Operation                    | Name         | Rows | Cost |
-------------------------------------------------------------------
| 0 |SELECT STATEMENT             |              |    1 |    2 |
| 1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES    |    1 |    2 |
|*2 |  INDEX UNIQUE SCAN          | EMPLOYEES_PK |    1 |    1 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   2 - access("EMPLOYEE_ID"=123 AND "SUBSIDIARY_ID"=30)
```

Bất cứ khi nào query dùng toàn bộ primary key, database có thể dùng INDEX UNIQUE SCAN, bất kể index có bao nhiêu column. Nhưng điều gì xảy ra khi chỉ dùng một key column, chẳng hạn khi tìm tất cả nhân viên của một subsidiary?

```
SELECT first_name, last_name
  FROM employees
 WHERE subsidiary_id = 20;
```

```
----------------------------------------------------
| Id | Operation          | Name      | Rows | Cost |
----------------------------------------------------
|  0 | SELECT STATEMENT   |           |  106 |  478 |
|* 1 |  TABLE ACCESS FULL| EMPLOYEES |  106 |  478 |
----------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - filter("SUBSIDIARY_ID"=20)
```

Execution plan cho thấy database không dùng index. Thay vào đó, nó thực hiện FULL TABLE SCAN. Do đó, database đọc toàn bộ table và đánh giá từng row theo where clause. Execution time tăng theo kích thước table: nếu table tăng gấp mười lần, FULL TABLE SCAN mất thời gian gấp mười lần. Nguy hiểm của operation này là nó thường đủ nhanh trong development environment nhỏ, nhưng lại gây vấn đề performance nghiêm trọng trong production.

Full Table Scan

Operation TABLE ACCESS FULL, còn gọi là full table scan, vẫn có thể là operation hiệu quả nhất trong một số trường hợp, đặc biệt khi lấy một phần lớn table.

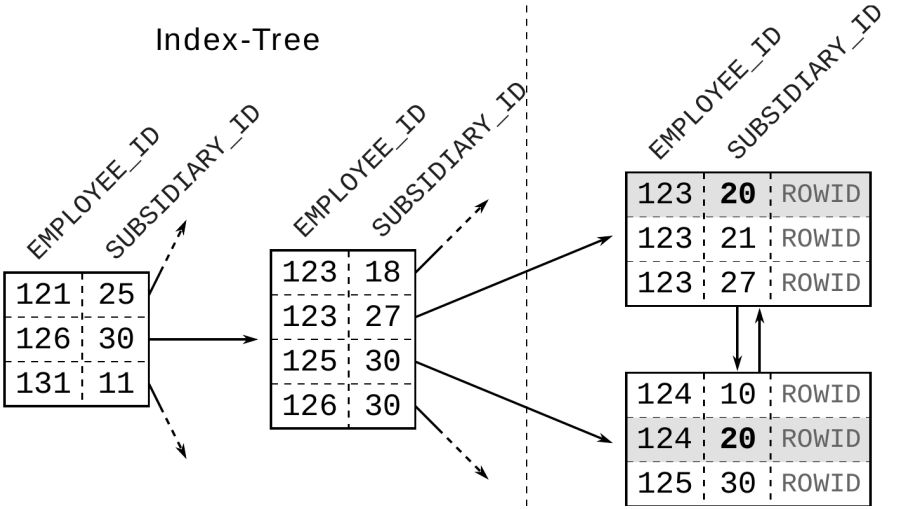
Điều này một phần do overhead của chính index lookup, vốn không có với operation TABLE ACCESS FULL. Chủ yếu là vì index lookup đọc từng block một, do database không biết block nào cần đọc tiếp theo cho đến khi block hiện tại đã được xử lý. FULL TABLE SCAN dù sao cũng phải lấy toàn bộ table, nên database có thể đọc các chunk lớn hơn mỗi lần (multi block read). Dù database đọc nhiều dữ liệu hơn, nó có thể cần thực hiện ít read operation hơn.

Database không dùng index vì không thể tùy ý dùng từng column riêng lẻ từ một concatenated index. Quan sát kỹ cấu trúc index sẽ làm rõ điều này.

Concatenated index cũng chỉ là B-tree index như mọi index khác, lưu dữ liệu đã index trong một danh sách được sắp xếp. Database xét từng column theo vị trí của nó trong định nghĩa index để sắp xếp các index entry. Column đầu tiên là tiêu chí sắp xếp chính, còn column thứ hai chỉ quyết định thứ tự nếu hai entry có cùng giá trị ở column đầu tiên, và cứ như vậy.

> Quan trọng
>
> Concatenated index là một index trải trên nhiều column.

Vì thế, thứ tự của index hai column giống thứ tự của danh bạ điện thoại: trước hết sắp theo họ, rồi theo tên. Điều đó nghĩa là index hai column không hỗ trợ tìm chỉ theo column thứ hai; việc đó giống như tìm danh bạ điện thoại theo tên.



Đoạn trích index trong Hình 2.1 cho thấy các entry của subsidiary 20 không được lưu cạnh nhau. Cũng thấy rõ không có entry nào có SUBSIDIARY_ID = 20 trong tree, dù chúng tồn tại ở leaf node. Do đó tree không hữu ích cho query này.

> Mẹo
>
> Hình dung index giúp hiểu index hỗ trợ những query nào. Bạn có thể query database để lấy các entry theo thứ tự index (cú pháp SQL:2008, xem trang 144 về các giải pháp proprietary dùng LIMIT, TOP hoặc ROWNUM):

```
SELECT <INDEX COLUMN LIST>
  FROM <TABLE>
 ORDER BY <INDEX COLUMN LIST>
 FETCH FIRST 100 ROWS ONLY;
```

> Nếu đưa định nghĩa index và tên table vào query, bạn sẽ nhận được một mẫu từ index. Hãy tự hỏi các row được yêu cầu có được nhóm ở một vị trí tập trung không. Nếu không, index tree không thể giúp tìm vị trí đó.

Dĩ nhiên, chúng ta có thể thêm một index khác trên SUBSIDIARY_ID để cải thiện tốc độ query. Tuy nhiên có một giải pháp tốt hơn, ít nhất nếu giả sử việc tìm chỉ bằng EMPLOYEE_ID không có ý nghĩa.

Chúng ta có thể tận dụng thực tế rằng column index đầu tiên luôn dùng được để tìm kiếm. Một lần nữa, nó giống danh bạ điện thoại: bạn không cần biết tên để tìm theo họ. Mẹo là đảo thứ tự column index để SUBSIDIARY_ID ở vị trí đầu tiên:

```
CREATE UNIQUE INDEX EMPLOYEES_PK
    ON EMPLOYEES (SUBSIDIARY_ID, EMPLOYEE_ID);
```

Hai column kết hợp vẫn là duy nhất, nên query với toàn bộ primary key vẫn có thể dùng INDEX UNIQUE SCAN, nhưng chuỗi index entry đã hoàn toàn khác. SUBSIDIARY_ID trở thành tiêu chí sắp xếp chính. Điều đó nghĩa là mọi entry của một subsidiary nằm liên tiếp trong index, nên database có thể dùng B-tree để tìm vị trí của chúng.

> Quan trọng
>
> Điểm quan trọng nhất khi định nghĩa concatenated index là chọn thứ tự column sao cho index có thể được dùng thường xuyên nhất.
>
> Execution plan xác nhận database dùng index “đảo ngược”. Chỉ SUBSIDIARY_ID không còn duy nhất, nên database phải đi theo các leaf node để tìm mọi entry khớp: vì vậy nó dùng operation INDEX RANGE SCAN.

```
-------------------------------------------------------------------
|Id |Operation                    | Name         | Rows | Cost |
-------------------------------------------------------------------
| 0 |SELECT STATEMENT             |              |  106 |   75 |
| 1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES    |  106 |   75 |
|*2 |  INDEX RANGE SCAN           | EMPLOYEE_PK |  106 |    2 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   2 - access("SUBSIDIARY_ID"=20)
```

Nói chung, database có thể dùng concatenated index khi tìm bằng các column dẫn đầu (bên trái nhất). Index có ba column có thể dùng khi tìm column đầu tiên, khi tìm hai column đầu tiên cùng nhau, và khi tìm bằng tất cả column.

Dù giải pháp hai index cũng cho select performance rất tốt, giải pháp một index vẫn được ưu tiên. Nó không chỉ tiết kiệm storage space mà còn giảm maintenance overhead cho index thứ hai. Table càng ít index thì performance của insert, delete và update càng tốt.

Để định nghĩa index tối ưu, bạn phải hiểu không chỉ cách index hoạt động mà còn phải biết application query dữ liệu thế nào. Điều đó nghĩa là phải biết các tổ hợp column xuất hiện trong where clause.

Vì vậy, định nghĩa index tối ưu rất khó với external consultant vì họ không có cái nhìn tổng thể về access path của application. Consultant thường chỉ có thể xét một query. Họ không khai thác lợi ích bổ sung mà index có thể mang lại cho các query khác. Database administrator cũng ở vị trí tương tự vì họ có thể biết database schema nhưng không có insight sâu về access path.

Nơi duy nhất kiến thức kỹ thuật về database gặp kiến thức chức năng về business domain là bộ phận development. Developer hiểu dữ liệu và biết access path. Họ có thể đánh index đúng để đem lại lợi ích tốt nhất cho toàn bộ application mà không cần nhiều công sức.

Index Chậm, Phần II

Phần trước đã giải thích cách thu được lợi ích bổ sung từ index hiện có bằng cách thay đổi thứ tự column, nhưng ví dụ chỉ xét hai SQL statement. Tuy nhiên, thay đổi index có thể ảnh hưởng đến mọi query trên table được index. Phần này giải thích cách database chọn index và minh họa các side effect có thể xảy ra khi thay đổi index hiện có.

Index EMPLOYEE_PK đã dùng cải thiện performance của mọi query chỉ tìm theo subsidiary. Tuy vậy, nó có thể dùng cho mọi query tìm theo SUBSIDIARY_ID, bất kể có thêm search criteria nào không. Điều đó nghĩa là index trở nên dùng được cho các query vốn dùng index khác với một phần khác của where clause. Trong trường hợp đó, nếu có nhiều access path khả dụng, optimizer có nhiệm vụ chọn access path tốt nhất.

> Query Optimizer
>
> Query optimizer, hay query planner, là component của database biến SQL statement thành execution plan. Quá trình này cũng gọi là compiling hoặc parsing. Có hai loại optimizer riêng biệt.
>
> Cost-based optimizer (CBO) tạo nhiều biến thể execution plan và tính một cost value cho từng plan. Việc tính cost dựa trên các operation đang dùng và số row được estimate. Cuối cùng, cost value làm chuẩn để chọn execution plan “tốt nhất”.
>
> Rule-based optimizer (RBO) tạo execution plan bằng một tập rule hard-coded. Rule-based optimizer ít linh hoạt hơn và ngày nay hiếm khi được dùng.

Thay đổi index cũng có thể có side effect khó chịu. Trong ví dụ của chúng ta, application danh bạ điện thoại nội bộ đã chậm đi rất nhiều từ sau vụ sáp nhập. Phân tích đầu tiên xác định query sau là nguyên nhân làm chậm:

```
SELECT first_name, last_name, subsidiary_id, phone_number
  FROM employees
 WHERE last_name  = 'WINAND'
   AND subsidiary_id = 30;
```

Execution plan là: Ví dụ 2.1. Execution Plan với Primary Key Index Đã Sửa Đổi

```
-----------------------------------------------------------------------
|Id |Operation                     | Name         | Rows | Cost |
-----------------------------------------------------------------------
| 0 |SELECT STATEMENT              |              |    1 |   30 |
|*1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES    |    1 |   30 |
|*2 |  INDEX RANGE SCAN            | EMPLOYEES_PK |   40 |    2 |
-----------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
  1 - filter("LAST_NAME"='WINAND')
  2 - access("SUBSIDIARY_ID"=30)
```

Execution plan dùng một index và có tổng cost value là 30. Đến đây mọi thứ đều ổn. Tuy nhiên, đáng nghi là nó dùng index chúng ta vừa thay đổi; đó là lý do đủ để nghi ngờ thay đổi index gây ra vấn đề performance, nhất là khi nhớ định nghĩa index cũ bắt đầu bằng column EMPLOYEE_ID, vốn hoàn toàn không nằm trong where clause. Trước đây query không thể dùng index đó.

Để phân tích thêm, sẽ hữu ích nếu so sánh execution plan trước và sau thay đổi. Để lấy execution plan gốc, chúng ta có thể triển khai lại định nghĩa index cũ, nhưng đa số database có cách đơn giản hơn để ngăn dùng một index cho query cụ thể. Ví dụ sau dùng Oracle optimizer hint cho mục đích đó.

```
SELECT /*+ NO_INDEX(EMPLOYEES EMPLOYEE_PK) */
       first_name, last_name, subsidiary_id, phone_number
  FROM employees
 WHERE last_name  = 'WINAND'
   AND subsidiary_id = 30;
```

Execution plan được cho là đã dùng trước thay đổi index hoàn toàn không dùng index:

```
-----------------------------------------------------
| Id | Operation         | Name      | Rows | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT  |           |    1 |  477 |
|* 1 |  TABLE ACCESS FULL| EMPLOYEES |    1 |  477 |
-----------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - filter("LAST_NAME"='WINAND' AND "SUBSIDIARY_ID"=30)
```

Dù TABLE ACCESS FULL phải đọc và xử lý toàn bộ table, trong trường hợp này nó có vẻ nhanh hơn dùng index. Điều đó đặc biệt lạ vì query chỉ khớp một row. Dùng index để tìm một row lẽ ra phải nhanh hơn nhiều so với full table scan, nhưng trong trường hợp này thì không. Index có vẻ chậm.

Trong những trường hợp như vậy, tốt nhất là lần lượt xem từng bước của execution plan có vấn đề. Bước đầu là INDEX RANGE SCAN trên index EMPLOYEES_PK. Index đó không cover column LAST_NAME; INDEX RANGE SCAN chỉ có thể xét filter SUBSIDIARY_ID. Oracle database cho thấy điều này trong khu vực “Predicate Information”, entry “2” của execution plan. Tại đó có thể thấy các điều kiện được áp dụng cho từng operation.

> Mẹo
>
> Appendix A, “Execution Plans”, giải thích cách tìm “Predicate Information” cho các database khác.

INDEX RANGE SCAN có operation ID 2 (Ví dụ 2.1 ở trang 19) chỉ áp dụng filter SUBSIDIARY_ID=30. Điều đó nghĩa là nó duyệt index tree để tìm entry đầu tiên có SUBSIDIARY_ID 30. Sau đó nó đi theo leaf node chain để tìm mọi entry khác của subsidiary đó. Kết quả của INDEX RANGE SCAN là danh sách ROWID thỏa điều kiện SUBSIDIARY_ID: tùy quy mô subsidiary, có thể chỉ có vài ROWID hoặc có thể có hàng trăm.

Bước tiếp theo là operation TABLE ACCESS BY INDEX ROWID. Nó dùng ROWID từ bước trước để lấy các row, tức mọi column, từ table. Khi column LAST_NAME đã có, database mới có thể đánh giá phần còn lại của where clause. Điều đó nghĩa là database phải lấy mọi row có SUBSIDIARY_ID=30 trước khi có thể áp dụng filter LAST_NAME.

Response time của statement không phụ thuộc vào kích thước result set mà phụ thuộc vào số nhân viên trong subsidiary cụ thể. Nếu subsidiary chỉ có vài thành viên, INDEX RANGE SCAN có performance tốt hơn. Tuy vậy, TABLE ACCESS FULL có thể nhanh hơn với subsidiary rất lớn vì nó có thể đọc phần lớn table trong một lần (xem “Full Table Scan” ở trang 13).

Query chậm vì index lookup trả về nhiều ROWID, một cho mỗi nhân viên của công ty ban đầu, và database phải lấy chúng riêng lẻ. Đây là sự kết hợp hoàn hảo của hai yếu tố làm index chậm: database đọc một index range rộng và phải lấy nhiều row riêng lẻ.

Việc chọn execution plan tốt nhất cũng phụ thuộc vào data distribution của table, nên optimizer dùng statistics về nội dung database. Trong ví dụ này, một histogram chứa distribution nhân viên theo subsidiary được dùng. Điều này cho phép optimizer estimate số row trả về từ index lookup; kết quả được dùng để tính cost.

Statistics

Cost-based optimizer dùng statistics về table, column và index. Phần lớn statistics được thu thập ở cấp column: số lượng giá trị riêng biệt, giá trị nhỏ nhất và lớn nhất (data range), số lần xuất hiện NULL và column histogram (data distribution). Giá trị statistics quan trọng nhất của table là kích thước của nó (theo row và block).

Statistics index quan trọng nhất là tree depth, số leaf node, số distinct key và clustering factor (xem Chương 5, “Clustering Data”).

Optimizer dùng các giá trị này để estimate selectivity của các predicate trong where clause.

Nếu không có statistics, chẳng hạn vì chúng đã bị xóa, optimizer dùng các giá trị mặc định. Statistics mặc định của Oracle database cho thấy index nhỏ với selectivity trung bình. Chúng dẫn đến estimate rằng INDEX RANGE SCAN sẽ trả về 40 row. Execution plan hiển thị estimate này trong column Rows (một lần nữa, xem Ví dụ 2.1 ở trang 19). Rõ ràng đây là estimate thấp hơn rất nhiều, vì có 1000 nhân viên làm việc cho subsidiary này.

Nếu cung cấp statistics chính xác, optimizer làm việc tốt hơn. Execution plan sau cho thấy estimate mới: 1000 row cho INDEX RANGE SCAN. Vì thế optimizer tính cost value cao hơn cho table access tiếp theo.

```
-------------------------------------------------------------------
|Id |Operation                    | Name         | Rows | Cost |
-------------------------------------------------------------------
| 0 |SELECT STATEMENT             |              |    1 |  680 |
|*1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES    |    1 |  680 |
|*2 |  INDEX RANGE SCAN           | EMPLOYEES_PK | 1000 |    4 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
  1 - filter("LAST_NAME"='WINAND')
  2 - access("SUBSIDIARY_ID"=30)
```

Cost value 680 thậm chí cao hơn cost value của execution plan dùng FULL TABLE SCAN (477, xem trang 20). Vì vậy optimizer sẽ tự động ưu tiên FULL TABLE SCAN.

Ví dụ về index chậm này không nên che khuất thực tế rằng indexing đúng là giải pháp tốt nhất. Dĩ nhiên, tìm theo last name được hỗ trợ tốt nhất bằng index trên LAST_NAME:

```
CREATE INDEX emp_name ON employees (last_name);
```

Với index mới, optimizer tính cost value là 3:

Ví dụ 2.2. Execution Plan với Index Chuyên Dụng

```
-----------------------------------------------------------------
| Id | Operation                    | Name      | Rows | Cost |
-----------------------------------------------------------------
|  0 | SELECT STATEMENT             |           |    1 |    3 |
|* 1 |  TABLE ACCESS BY INDEX ROWID| EMPLOYEES |    1 |    3 |
|* 2 |   INDEX RANGE SCAN           | EMP_NAME  |    1 |    1 |
-----------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - filter("SUBSIDIARY_ID"=30)
   2 - access("LAST_NAME"='WINAND')
```

Theo estimate của optimizer, index access chỉ trả về một row. Vì thế database chỉ phải lấy row đó từ table: điều này chắc chắn nhanh hơn FULL TABLE SCAN. Index được định nghĩa đúng vẫn tốt hơn full table scan ban đầu.

Hai execution plan trong Ví dụ 2.1 (trang 19) và Ví dụ 2.2 gần như giống hệt nhau. Database thực hiện cùng operation và optimizer tính các cost value tương tự, nhưng plan thứ hai hoạt động tốt hơn nhiều. Hiệu quả của INDEX RANGE SCAN có thể biến thiên trong một phạm vi rộng, đặc biệt khi sau đó là table access. Dùng index không tự động nghĩa là statement được thực thi theo cách tốt nhất có thể.

Functions

Index trên LAST_NAME đã cải thiện performance đáng kể, nhưng nó yêu cầu bạn tìm kiếm bằng đúng chữ hoa/chữ thường như được lưu trong database. Phần này giải thích cách bỏ hạn chế này mà không làm giảm performance.

Lưu ý

MySQL 5.6 không hỗ trợ function-based indexing như mô tả bên dưới. Thay vào đó, virtual column từng được lên kế hoạch cho MySQL 6.0 nhưng chỉ được giới thiệu trong MariaDB 5.2.

Tìm Kiếm Không Phân Biệt Hoa Thường Bằng UPPER hoặc LOWER

Bỏ qua chữ hoa/chữ thường trong where clause rất đơn giản. Chẳng hạn, bạn có thể chuyển cả hai vế so sánh thành ký hiệu toàn chữ hoa:

```
SELECT first_name, last_name, phone_number
  FROM employees
 WHERE UPPER(last_name) = UPPER('winand');
```

Dù search term hoặc column LAST_NAME dùng kiểu viết hoa nào, function UPPER vẫn làm chúng khớp như mong muốn.

Lưu ý

Một cách khác để khớp không phân biệt hoa thường là dùng “collation” khác. Collation mặc định của SQL Server và MySQL không phân biệt chữ hoa và chữ thường, chúng mặc định case-insensitive.

```
-----------------------------------------------------
| Id | Operation         | Name      | Rows | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT  |           |   10 |  477 |
|* 1 |  TABLE ACCESS FULL| EMPLOYEES |   10 |  477 |
-----------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - filter(UPPER("LAST_NAME")='WINAND')
```

Logic của query này hoàn toàn hợp lý nhưng execution plan thì không.

Chúng ta lại gặp người bạn cũ full table scan. Dù có index trên LAST_NAME, index đó không dùng được vì việc tìm kiếm không diễn ra trên LAST_NAME mà trên UPPER(LAST_NAME). Theo góc nhìn của database, đây là hai thứ hoàn toàn khác nhau.

Đây là cái bẫy mà tất cả chúng ta có thể gặp phải. Chúng ta nhận ra ngay quan hệ giữa LAST_NAME và UPPER(LAST_NAME) và mong database cũng "thấy" như vậy. Trên thực tế, góc nhìn của optimizer giống thế này hơn:

```
SELECT first_name, last_name, phone_number
  FROM employees
 WHERE BLACKBOX(...) = 'WINAND';
```

Function UPPER chỉ là một black box. Các parameter của function không liên quan vì không có quan hệ tổng quát nào giữa parameter của function và kết quả.

> Mẹo
>
> Hãy thay tên function bằng BLACKBOX để hiểu góc nhìn của optimizer.

> Đánh Giá Khi Compile
>
> Optimizer có thể đánh giá biểu thức ở vế phải trong “compile time” vì nó có tất cả input parameter. Vì vậy execution plan của Oracle (phần “Predicate Information”) chỉ hiển thị ký hiệu chữ hoa của search term. Hành vi này rất giống compiler đánh giá constant expression trong compile time.

Để hỗ trợ query đó, chúng ta cần một index cover search term thực tế. Điều đó nghĩa là không cần index trên LAST_NAME mà trên UPPER(LAST_NAME):

```
CREATE INDEX emp_up_name
    ON employees (UPPER(last_name));
```

Index có định nghĩa chứa function hoặc expression được gọi là function-based index (FBI). Thay vì sao chép trực tiếp dữ liệu column vào index, function-based index áp dụng function trước rồi đưa kết quả vào index. Do đó index lưu tên ở dạng toàn chữ hoa.

Database có thể dùng function-based index nếu expression chính xác của định nghĩa index xuất hiện trong SQL statement, như ví dụ trên. Execution plan xác nhận điều này:

```
-----------------------------------------------------------------
|Id |Operation                    | Name         | Rows | Cost |
-----------------------------------------------------------------
| 0 |SELECT STATEMENT             |              |  100 |   41 |
| 1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES    |  100 |   41 |
|*2 |  INDEX RANGE SCAN           | EMP_UP_NAME |   40 |    1 |
-----------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
  2 - access(UPPER("LAST_NAME")='WINAND')
```

Đây là INDEX RANGE SCAN thông thường như đã mô tả ở Chương 1. Database duyệt B-tree và đi theo leaf node chain. Không có operation hoặc keyword riêng cho function-based index.

Cảnh báo

Đôi khi ORM tool dùng UPPER và LOWER mà developer không biết. Ví dụ, Hibernate chèn LOWER ngầm cho các tìm kiếm không phân biệt hoa thường.

Execution plan vẫn chưa giống execution plan ở phần trước không dùng UPPER; estimate số row quá cao. Đặc biệt lạ là optimizer dự kiến lấy nhiều row từ table hơn số row mà INDEX RANGE SCAN trả về ngay từ đầu. Làm sao nó có thể lấy 100 row từ table nếu index scan trước đó chỉ trả về 40 row? Câu trả lời là không thể. Những estimate mâu thuẫn như vậy thường chỉ ra vấn đề với statistics. Trong trường hợp này, nguyên nhân là Oracle database không cập nhật table statistics khi tạo index mới (xem thêm “Oracle Statistics for Function-Based Indexes” ở trang 28).

Sau khi cập nhật statistics, optimizer tính các estimate chính xác hơn:

```
-------------------------------------------------------------------
|Id |Operation                    | Name         | Rows | Cost |
-------------------------------------------------------------------
| 0 |SELECT STATEMENT             |              |    1 |    3 |
| 1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES    |    1 |    3 |
|*2 |  INDEX RANGE SCAN           | EMP_UP_NAME |    1 |    1 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
  2 - access(UPPER("LAST_NAME")='WINAND')
```

> Lưu ý
>
> “Extended statistics” trên expression và column group được giới thiệu trong Oracle release 11g.
>
> Dù statistics đã cập nhật không cải thiện execution performance trong trường hợp này, vì index vẫn đã được dùng đúng, việc kiểm tra estimate của optimizer luôn là ý hay. Số row được xử lý cho mỗi operation (cardinality estimate) là con số đặc biệt quan trọng, cũng được hiển thị trong execution plan của SQL Server và PostgreSQL.

> Mẹo
>
> Appendix A, “Execution Plans”, mô tả row count estimate trong execution plan của SQL Server và PostgreSQL.
>
> SQL Server không hỗ trợ function-based index như mô tả, nhưng cung cấp computed column có thể dùng thay thế. Để sử dụng, trước tiên phải thêm computed column vào table rồi mới có thể index nó:

```
ALTER TABLE employees ADD last_name_up AS UPPER(last_name);
CREATE INDEX emp_up_name ON employees (last_name_up);
```

SQL Server có thể dùng index này bất cứ khi nào expression đã index xuất hiện trong statement. Bạn không cần viết lại query để dùng computed column.

> Oracle Statistics cho Function-Based Index
>
> Oracle database duy trì thông tin về số distinct column value như một phần của table statistics. Các con số này được dùng lại nếu column là một phần của nhiều index.
>
> Statistics cho function-based index (FBI) cũng được lưu ở cấp table dưới dạng virtual column. Dù Oracle database tự động thu thập index statistics cho index mới (từ release 10g), nó không cập nhật table statistics. Vì vậy, tài liệu Oracle khuyến nghị cập nhật table statistics sau khi tạo function-based index:

Sau khi tạo function-based index, hãy thu thập statistics

trên cả index và base table bằng package DBMS_STATS.

Statistics như vậy sẽ giúp Oracle Database

quyết định chính xác khi nào nên dùng index.

- Oracle Database SQL Language Reference

Khuyến nghị cá nhân của tôi còn đi xa hơn: sau mỗi thay đổi index, hãy cập nhật statistics cho base table và mọi index của nó. Tuy vậy, điều đó cũng có thể dẫn đến side effect không mong muốn. Hãy phối hợp việc này với database administrator (DBA) và backup statistics gốc.

User-Defined Function

Function-based indexing là cách tiếp cận rất tổng quát. Ngoài function như UPPER, bạn còn có thể index expression như A + B và thậm chí dùng user-defined function trong định nghĩa index.

Có một ngoại lệ quan trọng. Chẳng hạn, không thể tham chiếu current time trong định nghĩa index, trực tiếp hay gián tiếp, như ví dụ sau.

```
CREATE FUNCTION get_age(date_of_birth DATE)
RETURN NUMBER
AS
BEGIN
  RETURN
    TRUNC(MONTHS_BETWEEN(SYSDATE, date_of_birth)/12);
END;
/
```

Function GET_AGE dùng ngày hiện tại (SYSDATE) để tính tuổi dựa trên ngày sinh được cung cấp. Bạn có thể dùng function này ở mọi phần của SQL query, ví dụ trong select và where clause:

```
SELECT first_name, last_name, get_age(date_of_birth)
  FROM employees
 WHERE get_age(date_of_birth) = 42;
```

Query liệt kê mọi nhân viên 42 tuổi. Dùng function-based index là ý tưởng hiển nhiên để tối ưu query này, nhưng bạn không thể dùng function GET_AGE trong định nghĩa index vì nó không deterministic. Điều đó nghĩa là kết quả của function call không hoàn toàn được quyết định bởi parameter của nó. Chỉ function luôn trả về cùng kết quả cho cùng parameter, tức function deterministic, mới có thể được index.

Lý do cho hạn chế này đơn giản. Khi insert row mới, database gọi function và lưu kết quả trong index, rồi kết quả đó nằm yên, không đổi. Không có process định kỳ nào cập nhật index. Database chỉ cập nhật tuổi đã index khi ngày sinh bị thay đổi bằng update statement. Sau sinh nhật tiếp theo, tuổi được lưu trong index sẽ sai.

Ngoài việc phải deterministic, PostgreSQL và Oracle database yêu cầu function được khai báo deterministic khi dùng trong index, nên bạn phải dùng keyword DETERMINISTIC (Oracle) hoặc IMMUTABLE (PostgreSQL).

Cẩn trọng

PostgreSQL và Oracle database tin vào khai báo DETERMINISTIC hoặc IMMUTABLE, nghĩa là chúng tin developer. Bạn có thể khai báo function GET_AGE là deterministic và dùng nó trong định nghĩa index. Bất kể khai báo đó, nó sẽ không hoạt động như dự định vì tuổi lưu trong index sẽ không tăng theo năm tháng; nhân viên sẽ không già đi, ít nhất là trong index.

Các ví dụ khác về function không thể được "index" là random number generator và function phụ thuộc environment variable.

Hãy suy nghĩ

Làm sao vẫn có thể dùng index để tối ưu query cho tất cả nhân viên 42 tuổi?

Over-Indexing

Nếu khái niệm function-based indexing còn mới, bạn có thể bị cám dỗ index mọi thứ, nhưng thực tế đây là điều cuối cùng bạn nên làm. Lý do là mọi index đều tạo ra maintenance liên tục. Function-based index đặc biệt rắc rối vì chúng khiến việc tạo index dư thừa rất dễ dàng.

Tìm kiếm không phân biệt hoa thường ở trên cũng có thể triển khai bằng function LOWER:

SELECT first_name, last_name, phone_number

FROM employees

WHERE LOWER(last_name) = LOWER('winand');

Một index không thể hỗ trợ cả hai cách bỏ qua chữ hoa thường. Dĩ nhiên, chúng ta có thể tạo index thứ hai trên LOWER(last_name) cho query này, nhưng điều đó nghĩa là database phải duy trì hai index cho mỗi statement insert, update và delete (xem thêm Chương 8, "Modifying Data"). Để một index là đủ, bạn nên dùng nhất quán cùng một function trong toàn application.

Mẹo

Hợp nhất access path để một index có thể được dùng bởi nhiều query.

Mẹo

Luôn hướng đến việc index dữ liệu gốc vì đó thường là thông tin hữu ích nhất có thể đưa vào index.

Parameterized Queries

Phần này đề cập đến một chủ đề bị bỏ qua trong phần lớn textbook về SQL: parameterized query và bind parameter.

Bind parameter, còn gọi là dynamic parameter hoặc bind variable, là một cách khác để truyền dữ liệu vào database. Thay vì đặt giá trị trực tiếp vào SQL statement, bạn chỉ cần dùng một placeholder như ?, :name hoặc @name rồi cung cấp giá trị thực qua một API call riêng.

Không có gì sai khi viết giá trị trực tiếp vào các ad-hoc statement; tuy nhiên, có hai lý do chính đáng để dùng bind parameter trong program:

Security

Bind variable là cách tốt nhất để ngăn SQL injection.[1]

[1] http://en.wikipedia.org/wiki/SQL_injection

Performance

Database có execution plan cache như SQL Server và Oracle database có thể tái sử dụng execution plan khi thực thi cùng một statement nhiều lần. Cách này tiết kiệm công sức xây dựng lại execution plan, nhưng chỉ hoạt động khi SQL statement hoàn toàn giống nhau. Nếu đặt các giá trị khác nhau vào SQL statement, database sẽ xử lý nó như một statement khác và tạo lại execution plan.

Khi dùng bind parameter, bạn không viết giá trị thực mà thay vào đó chèn placeholder vào SQL statement. Nhờ vậy, statement không thay đổi khi được thực thi với các giá trị khác nhau.

```
99 rows selected.

SELECT first_name, last_name
  FROM employees
 WHERE subsidiary_id = 20;

-------------------------------------------------------------------
|Id | Operation                    | Name        | Rows | Cost |
-------------------------------------------------------------------
| 0 | SELECT STATEMENT             |             |   99 |   70 |
| 1 |  TABLE ACCESS BY INDEX ROWID| EMPLOYEES   |   99 |   70 |
|*2 |   INDEX RANGE SCAN           | EMPLOYEE_PK |   99 |    2 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   2 - access("SUBSIDIARY_ID"=20)
```

Index lookup cho performance tốt nhất với các subsidiary nhỏ, nhưng TABLE ACCESS FULL có thể nhanh hơn index với các subsidiary lớn:

```
1000 rows selected.

SELECT first_name, last_name
  FROM employees
 WHERE subsidiary_id = 30;
```

```
-------------------------------------------------
| Id | Operation         | Name      | Rows | Cost |
-------------------------------------------------
|  0 | SELECT STATEMENT  |           | 1000 |  478 |
|* 1 |  TABLE ACCESS FULL| EMPLOYEES | 1000 |  478 |
-------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   1 - filter("SUBSIDIARY_ID"=30)
```

Trong trường hợp này, histogram trên SUBSIDIARY_ID phát huy đúng mục đích. Optimizer dùng nó để xác định frequency của subsidiary ID được nhắc đến trong SQL query. Vì vậy, nó nhận được hai row count estimate khác nhau cho hai query.

Do đó, bước tính cost tiếp theo sẽ cho ra hai cost value khác nhau. Khi optimizer cuối cùng chọn execution plan, nó chọn plan có cost value thấp nhất. Với subsidiary nhỏ hơn, đó là plan dùng index.

Cost của operation TABLE ACCESS BY INDEX ROWID rất nhạy với row count estimate. Chọn số row nhiều gấp mười lần sẽ làm cost value tăng theo hệ số đó. Khi ấy overall cost dùng index còn cao hơn full table scan. Vì vậy, optimizer sẽ chọn execution plan còn lại cho subsidiary lớn hơn.

Khi dùng bind parameter, optimizer không có giá trị cụ thể để xác định frequency của chúng. Nó chỉ giả định phân phối đồng đều và luôn nhận được cùng row count estimate và cost value. Cuối cùng, nó sẽ luôn chọn cùng một execution plan.

> Mẹo
>
> Column histogram hữu ích nhất khi các giá trị không được phân phối đồng đều.
>
> Với column có phân phối đồng đều, thường chỉ cần chia số giá trị distinct cho số row trong table. Phương pháp này cũng hoạt động khi dùng bind parameter.

Nếu so optimizer với compiler, bind variable giống như program variable, còn khi viết giá trị trực tiếp vào statement thì chúng giống constant hơn. Database có thể dùng các giá trị từ SQL statement trong lúc optimization, cũng như compiler có thể evaluate constant expression trong lúc compilation. Nói ngắn gọn, bind parameter không visible với optimizer, giống như runtime value của variable không được compiler biết đến.

Từ góc nhìn này, có một nghịch lý nhỏ: bind parameter có thể cải thiện performance nếu không dùng bind parameter cho phép optimizer luôn chọn execution plan tốt nhất. Nhưng cái giá là gì? Việc generate và evaluate mọi execution plan variant là một effort rất lớn, không đáng nếu cuối cùng bạn vẫn nhận cùng một kết quả.

> Mẹo
>
> Không dùng bind parameter cũng giống như compile lại một program mỗi lần.

Quyết định xây dựng specialized execution plan hay generic execution plan đặt database trước một dilemma. Hoặc database bỏ effort để evaluate mọi plan variant có thể có ở mỗi lần execution nhằm luôn có execution plan tốt nhất, hoặc tiết kiệm optimization overhead và dùng cached execution plan bất cứ khi nào có thể, chấp nhận rủi ro dùng execution plan suboptimal. Vấn đề là database không biết full optimization cycle có tạo ra một execution plan khác

hay không nếu chưa thực sự thực hiện full optimization. Database vendor cố giải quyết dilemma này bằng các heuristic method, nhưng thành công rất hạn chế.

Ở vai trò developer, bạn có thể chủ động dùng bind parameter để giúp giải quyết dilemma này. Cụ thể, bạn nên luôn dùng bind parameter, ngoại trừ những giá trị cần ảnh hưởng đến execution plan.

Status code phân phối không đồng đều như "todo" và "done" là một ví dụ tốt. Số entry "done" thường nhiều hơn record "todo" cả một order of magnitude. Trong trường hợp đó, dùng index chỉ có ý nghĩa khi tìm entry "todo". Partitioning là một ví dụ khác, tức là chia table và index trên nhiều storage area. Khi đó, giá trị thực có thể ảnh hưởng đến partition nào phải được scan. Performance của LIKE query cũng có thể bị ảnh hưởng bởi bind parameter, như chúng ta sẽ thấy trong phần tiếp theo.

> Mẹo
>
> Trên thực tế, chỉ có một vài trường hợp mà giá trị thực ảnh hưởng đến execution plan. Vì vậy, nếu còn phân vân, bạn nên dùng bind parameter, chỉ để ngăn SQL injection.

Các đoạn code sau minh họa cách dùng bind parameter trong nhiều programming language khác nhau.

C#

Without bind parameters:

```
int subsidiary_id;
SqlCommand cmd = new SqlCommand(
                   "select first_name, last_name"
                 + "  from employees"
                 + " where subsidiary_id = " + subsidiary_id
                 , connection);
```

Using a bind parameter:

```
int subsidiary_id;
SqlCommand cmd =
       new SqlCommand(
                      "select first_name, last_name"
                    + "  from employees"
                    + " where subsidiary_id = @subsidiary_id
                    , connection);
cmd.Parameters.AddWithValue("@subsidiary_id", subsidiary_id);
```

Xem thêm: tài liệu class SqlParameterCollection.

Java

Without bind parameters:

```
int subsidiary_id;
Statement command = connection.createStatement(
                    "select first_name, last_name"
                  + "  from employees"
                  + " where subsidiary_id = " + subsidiary_id
                  );
```

Using a bind parameter:

```
int subsidiary_id;
PreparedStatement command = connection.prepareStatement(
                    "select first_name, last_name"
                  + "  from employees"
                  + " where subsidiary_id = ?"
                  );
command.setInt(1, subsidiary_id);
```

Xem thêm: tài liệu class PreparedStatement.

Perl

Without bind parameters:

```
my $subsidiary_id;
my $sth = $dbh->prepare(
                  "select first_name, last_name"
                . "  from employees"
                . " where subsidiary_id = $subsidiary_id"
                );
$sth->execute();
```

Using a bind parameter:

```
my $subsidiary_id;
my $sth = $dbh->prepare(
                    "select first_name, last_name"
                  . "  from employees"
                  . " where subsidiary_id = ?"
                  );
$sth->execute($subsidiary_id);
```

Xem: Programming the Perl DBI.

PHP

Using MySQL, without bind parameters:

```
$mysqli->query("select first_name, last_name"
             . "  from employees"
             . " where subsidiary_id = " . $subsidiary_id);
```

Using a bind parameter:

```
if ($stmt = $mysqli->prepare("select first_name, last_name"
                           . "  from employees"
                           . " where subsidiary_id = ?"))
{
   $stmt->bind_param("i", $subsidiary_id);
   $stmt->execute();
} else {
  /* handle SQL error */
}
```

Xem thêm: tài liệu class mysqli_stmt::bind_param và "Prepared statements and stored procedures" trong tài liệu PDO.

Ruby

Without bind parameters:

```
dbh.execute("select first_name, last_name"
          + "  from employees"
          + " where subsidiary_id = #{subsidiary_id}");
```

Using a bind parameter:

```
dbh.prepare("select first_name, last_name"
          + "  from employees"
          + " where subsidiary_id = ?");
dbh.execute(subsidiary_id);
```

Xem thêm: "Quoting, Placeholders, and Parameter Binding" trong Ruby DBI Tutorial.

Dấu hỏi (?) là placeholder duy nhất được SQL standard định nghĩa. Dấu hỏi là positional parameter. Điều đó nghĩa là các dấu hỏi được đánh số từ trái sang phải. Để bind một giá trị vào một dấu hỏi cụ thể, bạn phải chỉ định số của nó. Tuy nhiên, cách này có thể rất bất tiện vì số thứ tự thay đổi khi thêm hoặc xóa placeholder. Nhiều database

cung cấp extension proprietary cho named parameter để giải quyết vấn đề này, chẳng hạn dùng ký hiệu "at" (@name) hoặc dấu hai chấm (:name).

> Lưu ý
>
> Bind parameter không thể thay đổi structure của SQL statement. Điều đó nghĩa là bạn không thể dùng bind parameter cho table name hoặc column name. Các bind parameter sau không hoạt động:

```
String sql = prepare("SELECT * FROM ? WHERE ?");

sql.execute('employees', 'employee_id = 1');
```

Nếu cần thay đổi structure của SQL statement trong runtime, hãy dùng dynamic SQL.

Cursor Sharing and Auto Parameterization

Optimizer và SQL query càng phức tạp thì việc caching execution plan càng quan trọng. SQL Server và Oracle database có các feature tự động thay literal value trong SQL string bằng bind parameter. Các feature này lần lượt được gọi là CURSOR_SHARING (Oracle) và forced parameterization (SQL Server).

Cả hai feature đều là workaround cho application hoàn toàn không dùng bind parameter. Bật các feature này khiến developer không thể chủ động dùng literal value.

Searching for Ranges

Các inequality operator như <, > và between có thể dùng index giống như equals operator đã giải thích ở trên. Ngay cả LIKE filter cũng có thể, trong một số điều kiện, dùng index như range condition.

Việc dùng các operation này giới hạn lựa chọn về thứ tự column trong multicolumn index. Giới hạn này thậm chí có thể loại bỏ mọi option indexing tối ưu, vì có những query mà bạn đơn giản không thể định nghĩa một thứ tự column "đúng".

Greater, Less and BETWEEN

Câu hỏi này dễ trả lời nếu SQL statement nêu rõ start condition và stop condition:

```
SELECT first_name, last_name, date_of_birth
  FROM employees
 WHERE date_of_birth >= TO_DATE(?, 'YYYY-MM-DD')
   AND date_of_birth <= TO_DATE(?, 'YYYY-MM-DD')
```

Index trên DATE_OF_BIRTH chỉ được scan trong range đã chỉ định. Scan bắt đầu ở date thứ nhất và kết thúc ở date thứ hai. Chúng ta không thể thu hẹp scanned index range thêm nữa.

Start condition và stop condition khó thấy hơn khi có thêm column thứ hai:

```
SELECT first_name, last_name, date_of_birth
  FROM employees
 WHERE date_of_birth >= TO_DATE(?, 'YYYY-MM-DD')
   AND date_of_birth <= TO_DATE(?, 'YYYY-MM-DD')
   AND subsidiary_id  = ?
```

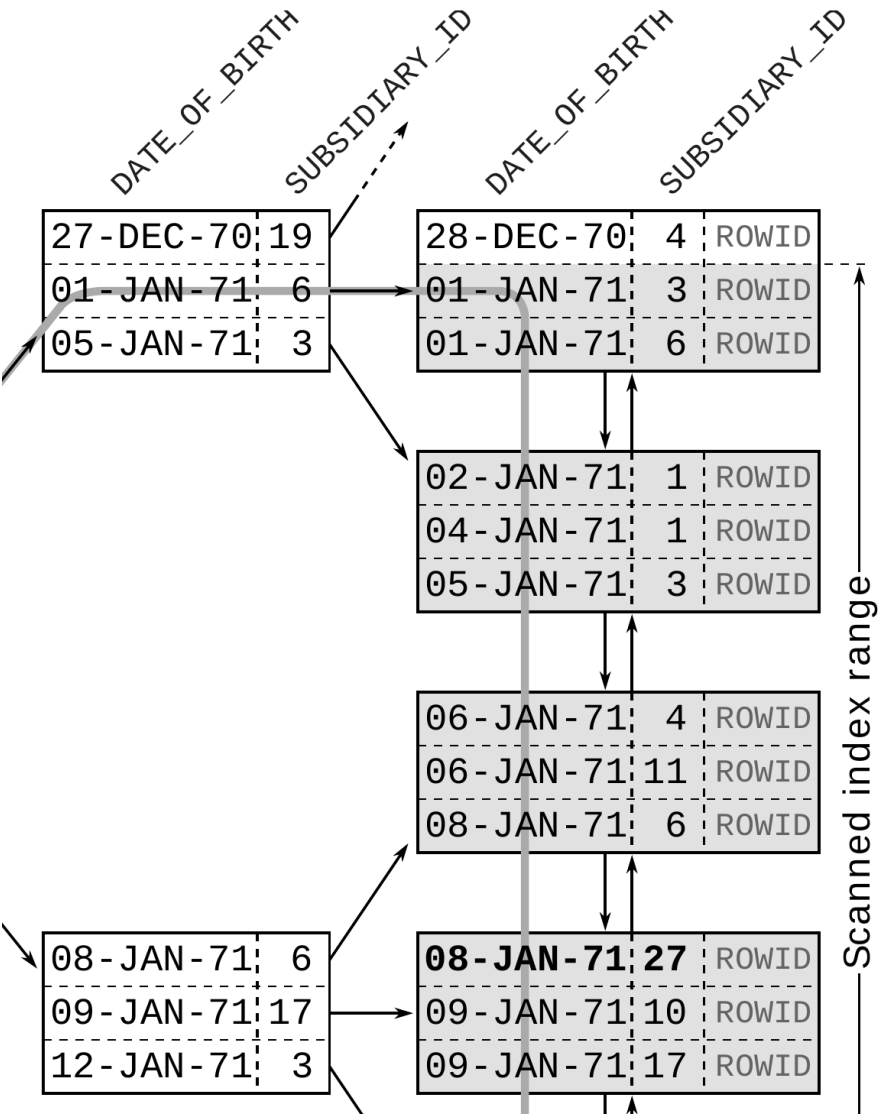
Dĩ nhiên, index lý tưởng phải cover cả hai column, nhưng câu hỏi là thứ tự nào?

Các figure sau cho thấy thứ tự column ảnh hưởng thế nào đến scanned index range. Trong minh họa này, chúng ta tìm tất cả nhân viên của subsidiary 27 sinh trong khoảng từ ngày 1 tháng 1 đến ngày 9 tháng 1 năm 1971.

Hình 2.2 mô tả chi tiết một index trên DATE_OF_BIRTH và SUBSIDIARY_ID, theo đúng thứ tự đó. Database sẽ bắt đầu đi theo leaf node chain ở đâu, hay nói cách khác, tree traversal sẽ kết thúc ở đâu?

Index được sắp xếp theo birth date trước. Chỉ khi hai nhân viên sinh cùng ngày thì SUBSIDIARY_ID mới được dùng để sắp xếp các record đó. Tuy nhiên, query bao phủ một date range. Vì vậy, thứ tự của SUBSIDIARY_ID vô dụng trong tree traversal. Điều này trở nên rõ ràng nếu nhận ra rằng không có entry cho subsidiary 27 trong branch node, dù có entry như vậy trong leaf node. Do đó, filter trên DATE_OF_BIRTH là điều kiện duy nhất giới hạn scanned index range. Scan bắt đầu từ entry đầu tiên khớp date range và kết thúc ở entry cuối cùng, tức toàn bộ năm leaf node được hiển thị trong Hình 2.2.

Hình 2.2. Range Scan trong index DATE_OF_BIRTH, SUBSIDIARY_ID

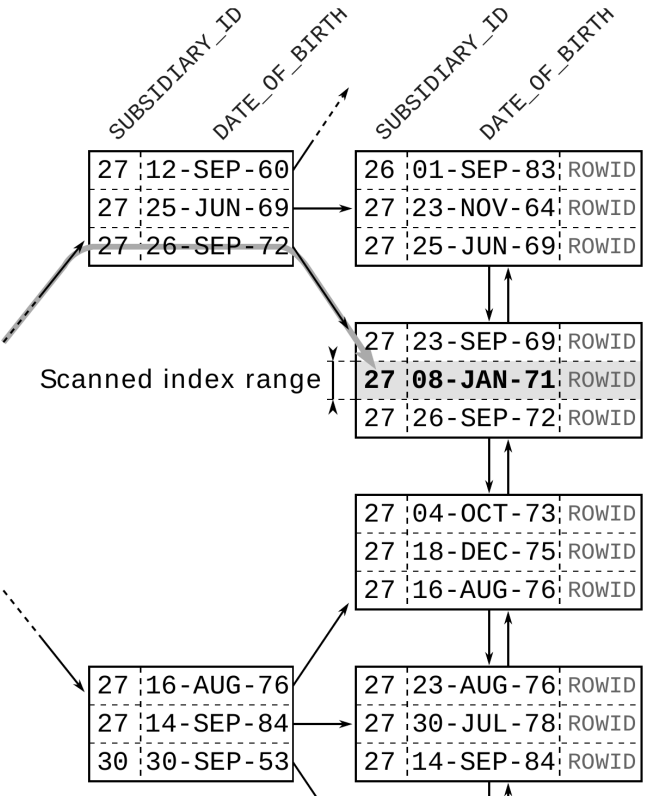


Bức tranh hoàn toàn khác khi đảo thứ tự column. Hình 2.3 minh họa scan khi index bắt đầu bằng column SUBSIDIARY_ID.

Điểm khác biệt là equals operator giới hạn column index đầu tiên vào một giá trị duy nhất. Trong range của giá trị này (SUBSIDIARY_ID 27), index được sắp xếp theo column thứ hai, tức

date of birth, nên không cần đi qua leaf node đầu tiên vì branch node đã cho biết không có employee nào của subsidiary 27 sinh sau ngày 25 tháng 6 năm 1969 trong leaf node đầu tiên.

Hình 2.3. Range Scan trong index SUBSIDIARY_ID, DATE_OF_BIRTH



Tree traversal dẫn trực tiếp đến leaf node thứ hai. Trong trường hợp này, mọi điều kiện của where clause đều giới hạn scanned index range, nên scan kết thúc ngay tại leaf node đó.

> Mẹo
>
> Quy tắc kinh nghiệm: index cho equality trước, rồi đến range.
>
> Performance difference thực tế phụ thuộc vào data và search criteria. Difference có thể không đáng kể nếu filter trên DATE_OF_BIRTH tự nó đã rất selective. Date range càng lớn thì performance difference càng lớn.

Với ví dụ này, chúng ta cũng có thể bác bỏ myth rằng column có selectivity cao nhất phải nằm ở vị trí ngoài cùng bên trái của index. Nếu nhìn vào các figure và chỉ xét selectivity của column đầu tiên, ta thấy cả hai điều kiện đều khớp 13 record. Điều này đúng bất kể filter chỉ theo DATE_OF_BIRTH hay chỉ theo SUBSIDIARY_ID. Selectivity không có ích ở đây, nhưng một thứ tự column vẫn tốt hơn thứ tự còn lại.

Để tối ưu performance, việc biết scanned index range rất quan trọng. Với phần lớn database, bạn thậm chí có thể thấy range này trong execution plan, chỉ cần biết phải tìm gì. Execution plan sau của Oracle database chỉ rõ rằng index EMP_TEST bắt đầu bằng column DATE_OF_BIRTH.

```
-------------------------------------------------------------------
|Id | Operation                     | Name      | Rows | Cost |
-------------------------------------------------------------------
| 0 | SELECT STATEMENT              |           |    1 |    4 |
|*1 |  FILTER                       |           |      |      |
| 2 |   TABLE ACCESS BY INDEX ROWID| EMPLOYEES |    1 |    4 |
|*3 |    INDEX RANGE SCAN           | EMP_TEST  |    2 |    2 |
-------------------------------------------------------------------
```

Predicate information của INDEX RANGE SCAN đưa ra manh mối then chốt. Nó xác định các điều kiện của where clause là access predicate hoặc filter predicate. Đây là cách database cho biết nó sử dụng từng điều kiện như thế nào.

> Lưu ý
>
> Execution plan đã được đơn giản hóa để dễ đọc. Appendix ở trang 170 giải thích chi tiết phần “Predicate Information” trong execution plan của Oracle.

Các điều kiện trên column DATE_OF_BIRTH là những điều kiện duy nhất được liệt kê dưới dạng access predicate; chúng giới hạn scanned index range. Vì vậy, DATE_OF_BIRTH là column đầu tiên trong index EMP_TEST. Column SUBSIDIARY_ID chỉ được dùng làm filter.

> Quan trọng
>
> Access predicate là start condition và stop condition của index lookup. Chúng định nghĩa scanned index range.
>
> Index filter predicate chỉ được áp dụng trong lúc leaf node traversal. Chúng không thu hẹp scanned index range.
>
> Appendix A giải thích cách nhận diện access predicate trong các database khác.

Database có thể dùng mọi điều kiện làm access predicate nếu chúng ta đảo định nghĩa index:

```
-------------------------------------------------------------------
| Id | Operation                    | Name      | Rows | Cost |
-------------------------------------------------------------------
|  0 | SELECT STATEMENT             |           |    1 |    3 |
|* 1 |  FILTER                      |           |      |      |
|  2 |   TABLE ACCESS BY INDEX ROWID| EMPLOYEES |    1 |    3 |
|* 3 |    INDEX RANGE SCAN          | EMP_TEST2 |    1 |    2 |
-------------------------------------------------------------------
```

Cuối cùng là between operator. Nó cho phép chỉ định upper bound và lower bound trong một condition duy nhất:

```
DATE_OF_BIRTH BETWEEN '01-JAN-71'
                  AND '10-JAN-71'
```

Lưu ý rằng between luôn bao gồm các giá trị được chỉ định, giống như khi dùng operator less than or equal to (<=) và greater than or equal to (>=):

```
    DATE_OF_BIRTH >= '01-JAN-71'
AND DATE_OF_BIRTH <= '10-JAN-71'
```

Indexing LIKE Filters

SQL LIKE operator rất thường gây ra behavior performance bất ngờ vì một số search term ngăn database dùng index hiệu quả. Điều đó nghĩa là có search term được index rất tốt, nhưng có search term thì không. Vị trí của wildcard character tạo ra toàn bộ khác biệt.

Ví dụ sau dùng wildcard % ở giữa search term:

```
SELECT first_name, last_name, date_of_birth
  FROM employees
 WHERE UPPER(last_name) LIKE 'WIN%D'
```

```
-------------------------------------------------------------------
|Id | Operation                    | Name         | Rows | Cost |
-------------------------------------------------------------------
| 0 | SELECT STATEMENT             |              |    1 |    4 |
| 1 |  TABLE ACCESS BY INDEX ROWID| EMPLOYEES    |    1 |    4 |
|*2 |   INDEX RANGE SCAN           | EMP_UP_NAME  |    1 |    2 |
-------------------------------------------------------------------
```

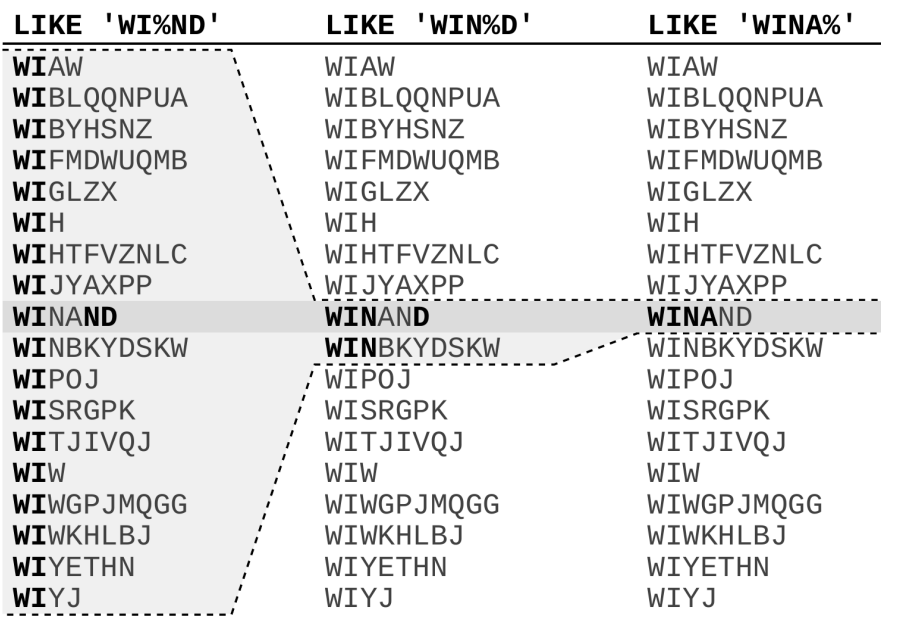
LIKE filter chỉ có thể dùng các character trước wildcard đầu tiên trong lúc tree traversal. Các character còn lại chỉ là filter predicate và không thu hẹp scanned index range. Vì vậy, một LIKE expression

có thể chứa hai loại predicate: (1) phần trước wildcard đầu tiên là access predicate; (2) các character còn lại là filter predicate.

> Cẩn trọng
>
> Với PostgreSQL database, bạn có thể cần chỉ định operator class (ví dụ varchar_pattern_ops) để dùng LIKE expression làm access predicate. Xem “Operator Classes and Operator Families” trong tài liệu PostgreSQL để biết thêm chi tiết.

Prefix trước wildcard đầu tiên càng selective thì scanned index range càng nhỏ. Đổi lại, index lookup càng nhanh. Hình 2.4 minh họa quan hệ này bằng ba LIKE expression khác nhau. Cả ba đều chọn cùng một row, nhưng scanned index range, và do đó performance, rất khác nhau.

Hình 2.4. Các LIKE search khác nhau



Expression đầu tiên có hai character trước wildcard. Chúng giới hạn scanned index range còn 18 row. Chỉ một row trong số đó khớp toàn bộ LIKE expression, còn 17 row kia được fetch rồi loại bỏ. Expression thứ hai có prefix dài hơn, thu hẹp scanned index range còn hai row. Với expression này, database chỉ đọc thêm một row không liên quan đến kết quả. Expression cuối cùng hoàn toàn không có filter predicate: database chỉ đọc entry khớp toàn bộ LIKE expression.

> Quan trọng
>
> Chỉ phần trước wildcard đầu tiên mới đóng vai trò access predicate. Các character còn lại không thu hẹp scanned index range; các entry không khớp chỉ bị loại khỏi result.

Trường hợp ngược lại cũng có thể xảy ra: LIKE expression bắt đầu bằng wildcard. LIKE expression như vậy không thể làm access predicate. Database phải scan toàn bộ table nếu không có condition khác cung cấp access predicate.

> Mẹo
>
> Tránh LIKE expression có leading wildcard, ví dụ '%TERM'.

Vị trí của wildcard character ảnh hưởng đến việc dùng index, ít nhất là trên lý thuyết. Trên thực tế, optimizer tạo generic execution plan khi search term được cung cấp qua bind parameter. Khi đó, optimizer phải đoán liệu phần lớn execution có leading wildcard hay không.

Phần lớn database chỉ giả định rằng không có leading wildcard khi optimize LIKE condition với bind parameter, nhưng giả định này sai nếu LIKE expression được dùng cho full-text search. Đáng tiếc là không có cách trực tiếp để đánh dấu một LIKE condition là full-text search. Box “Labeling Full-Text LIKE Expressions” trình bày một cách thử nhưng không hoạt động. Cách hiển nhiên nhất là chỉ định search term không dùng bind parameter, nhưng điều đó làm tăng

optimization overhead và mở ra lỗ hổng SQL injection. Một giải pháp hiệu quả, vẫn secure và portable, là cố ý obfuscate LIKE condition. “Combining Columns” ở trang 70 giải thích chi tiết vấn đề này.

> Labeling Full-Text LIKE Expressions
>
> Khi dùng LIKE operator cho full-text search, chúng ta có thể tách wildcard khỏi search term:

```
WHERE text_column LIKE '%' || ? || '%'
```

Wildcard được viết trực tiếp vào SQL statement, nhưng chúng ta dùng bind parameter cho search term. Database tự xây dựng LIKE expression cuối cùng bằng string concatenation operator || (Oracle, PostgreSQL). Dù dùng bind parameter, LIKE expression cuối cùng luôn bắt đầu bằng wildcard. Đáng tiếc là database không nhận ra điều đó.

Với PostgreSQL database, vấn đề khác đi vì PostgreSQL giả định có leading wildcard khi dùng bind parameter cho LIKE expression. PostgreSQL đơn giản là không dùng index trong trường hợp đó. Cách duy nhất để có index access cho LIKE expression là làm cho search term thực tế visible với optimizer.

Nếu không dùng bind parameter mà đặt search term trực tiếp vào SQL statement, bạn phải thực hiện các biện pháp phòng ngừa khác để chống SQL injection attack!

Ngay cả khi database optimize execution plan cho leading wildcard, performance vẫn có thể không đủ. Trong trường hợp đó, bạn có thể dùng phần khác của where clause để access dữ liệu hiệu quả, xem thêm “Index Filter Predicates Used Intentionally” ở trang 112. Nếu không có access path nào khác, bạn có thể dùng một trong các full-text index solution proprietary sau.

MySQL

MySQL cung cấp keyword match và against cho full-text search. Từ MySQL 5.6, bạn cũng có thể tạo full-text index cho table InnoDB; trước đó, việc này chỉ có thể thực hiện với table MyISAM. Xem “Full-Text Search Functions” trong tài liệu MySQL.

Oracle Database

Oracle database cung cấp keyword contains. Xem “Oracle Text Application Developer’s Guide”.

PostgreSQL

PostgreSQL cung cấp operator @@ để triển khai full-text search. Xem “Full Text Search” trong tài liệu PostgreSQL.

[2] http://www.sai.msu.su/~megera/wiki/wildspeed

Một lựa chọn khác là dùng extension WildSpeed để optimize LIKE expression trực tiếp. Extension này lưu text theo mọi rotation có thể để mỗi character lần lượt nằm ở đầu. Điều đó nghĩa là indexed text không chỉ được lưu một lần mà được lưu nhiều lần bằng số character trong string, nên cần rất nhiều space.

SQL Server

SQL Server cung cấp keyword contains. Xem “Full-Text Search” trong tài liệu SQL Server.

> Hãy suy nghĩ
>
> Làm thế nào để index một LIKE search chỉ có một wildcard ở đầu search term ('%TERM')?

Index Merge

Một trong những câu hỏi phổ biến nhất về indexing là: nên tạo một index cho mỗi column hay một index duy nhất cho tất cả column trong where clause? Trong phần lớn trường hợp, câu trả lời rất đơn giản: một index có nhiều column tốt hơn.

Tuy nhiên, có những query mà một index duy nhất không thể xử lý hoàn hảo, bất kể bạn định nghĩa index thế nào; chẳng hạn query có từ hai range condition độc lập trở lên như ví dụ sau:

```
SELECT first_name, last_name, date_of_birth
  FROM employees
 WHERE UPPER(last_name) < ?
   AND date_of_birth    < ?
```

Không thể định nghĩa B-tree index hỗ trợ query này mà không có filter predicate. Để giải thích, chỉ cần nhớ rằng index là một linked list.

Nếu định nghĩa index là UPPER(LAST_NAME), DATE_OF_BIRTH theo thứ tự đó, list sẽ bắt đầu bằng A và kết thúc bằng Z. Date of birth chỉ được xét khi có hai employee cùng tên. Nếu định nghĩa index theo thứ tự ngược lại, nó sẽ bắt đầu bằng employee lớn tuổi nhất và kết thúc bằng employee trẻ tuổi nhất. Khi đó, name chỉ có ảnh hưởng nhỏ đến sort order.

Dù xoay chuyển định nghĩa index thế nào, các entry vẫn luôn được sắp xếp dọc theo một chain. Một đầu chứa entry nhỏ và đầu kia chứa entry lớn. Vì vậy, index chỉ có thể hỗ trợ một range condition dưới dạng access predicate. Để hỗ trợ hai range condition độc lập cần có trục thứ hai, chẳng hạn như bàn cờ. Query ở trên khi đó sẽ khớp tất cả entry từ một góc của bàn cờ, nhưng index không giống bàn cờ mà giống một chain. Nó không có góc.

Dĩ nhiên, bạn vẫn có thể chấp nhận filter predicate và dùng multicolumn index. Trong nhiều trường hợp, đó vẫn là giải pháp tốt nhất. Khi đó, định nghĩa index nên đặt column selective hơn lên trước để column này có thể được dùng làm access predicate. Đây có thể là nguồn gốc của myth "most selective first", nhưng quy tắc này chỉ đúng khi bạn không thể tránh filter predicate.

Lựa chọn còn lại là dùng hai index riêng, mỗi index cho một column. Khi đó database trước tiên phải scan cả hai index rồi kết hợp kết quả.

> Lưu ý
>
> Một index scan nhanh hơn hai index scan.

Database dùng hai phương pháp để kết hợp index. Thứ nhất là index join. Chương 4, "The Join Operation", giải thích chi tiết các related algorithm. Cách thứ hai sử dụng functionality từ thế giới data warehouse.

Data warehouse là mẹ của mọi ad-hoc query. Nó chỉ cần vài cú click để kết hợp các condition tùy ý thành query bạn muốn. Không thể dự đoán những tổ hợp column nào có thể xuất hiện trong where clause, khiến việc indexing như đã giải thích từ đầu gần như không thể.

Data warehouse dùng một loại index chuyên dụng để giải quyết vấn đề đó: bitmap index. Ưu điểm của bitmap index là chúng có thể được kết hợp khá dễ dàng. Vì vậy, bạn có performance khá tốt khi index từng column riêng lẻ. Ngược lại, nếu biết query từ trước để tạo một tailored multicolumn B-tree index, index đó vẫn nhanh hơn việc kết hợp nhiều bitmap index.

Điểm yếu lớn nhất của bitmap index là khả năng scale rất tệ với insert, update và delete. Concurrent write operation gần như không thể thực hiện. Điều đó không thành vấn đề trong data warehouse vì các load process được schedule lần lượt. Trong online application, bitmap index phần lớn là vô dụng.

> Quan trọng
>
> Bitmap index gần như không thể dùng cho online transaction processing (OLTP).

Nhiều database product cung cấp giải pháp hybrid giữa B-tree và bitmap index. Khi không có access path tốt hơn, chúng chuyển kết quả của một số B-tree scan thành bitmap structure trong memory. Các structure này có thể được kết hợp hiệu quả. Bitmap structure không được lưu persistent mà bị loại bỏ sau khi statement thực thi, qua đó tránh vấn đề write scalability kém. Nhược điểm là cần nhiều memory và CPU time. Xét cho cùng, phương pháp này là một hành động tuyệt vọng của optimizer.

Partial Indexes

Cho đến đây, chúng ta chỉ bàn về những column được thêm vào index. Với partial index (PostgreSQL) hoặc filtered index (SQL Server), bạn cũng có thể chỉ định những row được index.

> Cẩn trọng
>
> Oracle database có cách tiếp cận riêng đối với partial indexing. Phần tiếp theo sẽ giải thích cách đó dựa trên phần này.

Partial index hữu ích cho các where condition thường dùng với constant value, chẳng hạn status code trong ví dụ sau:

```
SELECT message
  FROM messages
 WHERE processed = 'N'
   AND receiver  = ?
```

Các query như vậy rất phổ biến trong queuing system. Query lấy tất cả message chưa được process của một recipient cụ thể. Message đã được process hiếm khi cần đến. Nếu cần, chúng thường được truy cập bằng một criteria cụ thể hơn như primary key.

Chúng ta có thể optimize query này bằng một two-column index. Chỉ xét query này, thứ tự column không quan trọng vì không có range condition.

```
CREATE INDEX messages_todo
          ON messages (receiver, processed)
```

Index hoàn thành mục đích, nhưng nó chứa nhiều row không bao giờ được tìm kiếm, cụ thể là tất cả message đã được process. Dù vậy, nhờ logarithmic scalability, index vẫn làm query rất nhanh, dù lãng phí nhiều disk space.

Với partial indexing, bạn có thể giới hạn index để chỉ chứa message chưa được process. Syntax cho việc này đơn giản đến bất ngờ: một where clause.

```
CREATE INDEX messages_todo
          ON messages (receiver)
       WHERE processed = 'N'
```

Index chỉ chứa những row thỏa mãn where clause. Trong trường hợp cụ thể này, chúng ta thậm chí có thể bỏ column PROCESSED vì nó luôn là 'N'. Điều đó có nghĩa index giảm kích thước theo hai chiều: theo chiều dọc vì chứa ít row hơn; theo chiều ngang vì đã bỏ một column.

Vì vậy, index rất nhỏ. Với một queue, điều này thậm chí có thể nghĩa là kích thước index không đổi dù table tăng không giới hạn. Index không chứa mọi message, chỉ chứa message chưa được process.

Where clause của partial index có thể phức tạp tùy ý. Giới hạn căn bản duy nhất liên quan đến function: bạn chỉ có thể dùng deterministic function, giống như ở mọi nơi khác trong index definition. Tuy nhiên, SQL Server có rule hạn chế hơn và không cho phép function cũng như OR operator trong index predicate.

Database có thể dùng partial index bất cứ khi nào where clause xuất hiện trong query.

> Hãy suy nghĩ
>
> Index nhỏ nhất có đặc điểm gì với query sau?

```
SELECT message
  FROM messages
 WHERE processed = 'N';
```

NULL trong Oracle Database

NULL trong SQL thường gây nhầm lẫn. Ý tưởng cơ bản của NULL — biểu diễn dữ liệu bị thiếu — khá đơn giản, nhưng có một số điểm đặc biệt. Chẳng hạn, bạn phải dùng IS NULL thay vì = NULL. Oracle database còn có thêm những điểm khác thường về NULL, một mặt vì nó không phải lúc nào cũng xử lý NULL theo đúng standard, mặt khác vì nó xử lý NULL trong index theo một cách rất "đặc biệt".

SQL standard không định nghĩa NULL là một value mà là placeholder cho một value bị thiếu hoặc chưa biết. Vì vậy, không có value nào có thể là NULL. Thay vào đó, Oracle database xử lý empty string như NULL:

```
   SELECT     '0 IS NULL???' AS "what is NULL?" FROM dual
    WHERE      0 IS NULL
UNION ALL
   SELECT    '0 is not null' FROM dual
    WHERE     0 IS NOT NULL
UNION ALL
   SELECT '''''  IS NULL???'  FROM dual
    WHERE    '' IS NULL
UNION ALL
   SELECT '''''  is not null' FROM dual
    WHERE    '' IS NOT NULL;

what is NULL?
--------------
0 is not null
'' IS NULL???
```

Để vấn đề thêm rối, có một trường hợp Oracle database thậm chí xử lý NULL như empty string:

```
SELECT dummy
     , dummy || ''
     , dummy || NULL
  FROM dual;

D D D
- - -
X X X
```

Nối column DUMMY (luôn chứa 'X') với NULL đáng lẽ phải trả về NULL.

Concept NULL được dùng trong nhiều programming language. Dù nhìn ở đâu, empty string cũng không bao giờ là NULL… ngoại trừ trong Oracle database. Trên thực tế, không thể lưu empty string vào field VARCHAR2. Nếu thử làm vậy, Oracle database chỉ lưu NULL.

Điểm đặc biệt này không chỉ kỳ lạ mà còn nguy hiểm. Ngoài ra, điểm khác thường của Oracle database với NULL chưa dừng ở đó — nó tiếp tục ảnh hưởng đến indexing.

Indexing NULL

Oracle database không đưa row vào index nếu tất cả column được index đều là NULL. Điều đó có nghĩa mọi index đều là một partial index — tương đương với việc có where clause:

```
CREATE INDEX idx
          ON tbl (A, B, C, ...)
       WHERE A IS NOT NULL
          OR B IS NOT NULL
          OR C IS NOT NULL
             ...;
```

Hãy xét index EMP_DOB. Nó chỉ có một column là DATE_OF_BIRTH. Một row không có value DATE_OF_BIRTH sẽ không được thêm vào index này.

```
INSERT INTO employees ( subsidiary_id, employee_id
                      , first_name   , last_name
                      , phone_number)
              VALUES ( ?, ?, ?, ?, ? );
```

Insert statement không set DATE_OF_BIRTH nên value mặc định là NULL — vì vậy record không được thêm vào index EMP_DOB. Do đó, index không thể hỗ trợ query tìm các record có DATE_OF_BIRTH IS NULL:

```
SELECT first_name, last_name
  FROM employees
 WHERE date_of_birth IS NULL;
```

```
-----------------------------------------------------
| Id | Operation         | Name      | Rows | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT  |           |    1 |  477 |
|* 1 |  TABLE ACCESS FULL| EMPLOYEES |    1 |  477 |
-----------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - filter("DATE_OF_BIRTH" IS NULL)
```

Tuy nhiên, record vẫn được đưa vào concatenated index nếu ít nhất một column của index không phải NULL:

```
CREATE INDEX demo_null
          ON employees (subsidiary_id, date_of_birth);
```

Row vừa tạo được thêm vào index vì SUBSIDIARY_ID không phải NULL. Vì vậy, index này có thể hỗ trợ query tìm tất cả employee thuộc một subsidiary cụ thể nhưng không có value DATE_OF_BIRTH:

```
SELECT first_name, last_name
  FROM employees
 WHERE subsidiary_id = ?
   AND date_of_birth IS NULL;
```

```
-------------------------------------------------------------
| Id | Operation                    | Name      | Rows | Cost |
-------------------------------------------------------------
|  0 | SELECT STATEMENT             |           |    1 |    2 |
|  1 |  TABLE ACCESS BY INDEX ROWID| EMPLOYEES |    1 |    2 |
|* 2 |   INDEX RANGE SCAN           | DEMO_NULL |    1 |    1 |
-------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   2 - access("SUBSIDIARY_ID"=TO_NUMBER(?)
          AND "DATE_OF_BIRTH" IS NULL)
```

Lưu ý rằng index cover toàn bộ where clause; mọi filter đều được dùng làm access predicate trong INDEX RANGE SCAN.

Chúng ta có thể mở rộng concept này cho query ban đầu để tìm mọi record có DATE_OF_BIRTH IS NULL. Khi đó, column DATE_OF_BIRTH phải là column ngoài cùng bên trái trong index để có thể được dùng làm access predicate. Dù bản thân query không cần column index thứ hai, chúng ta thêm một column không bao giờ là NULL để bảo đảm index có đủ mọi row.

Có thể dùng bất kỳ column nào có NOT NULL constraint, chẳng hạn SUBSIDIARY_ID, cho mục đích đó.

Một cách khác là dùng constant expression không bao giờ là NULL. Nhờ vậy, index sẽ có đủ mọi row — kể cả khi DATE_OF_BIRTH là NULL.

```
DROP   INDEX emp_dob;
CREATE INDEX emp_dob ON employees (date_of_birth, '1');
```

Về mặt kỹ thuật, đây là một function-based index. Ví dụ này cũng bác bỏ myth rằng Oracle database không thể index NULL.

> Mẹo
>
> Thêm một column không thể là NULL để index NULL như mọi value khác.

NOT NULL Constraints

Để index một điều kiện IS NULL trong Oracle database, index phải có một column không bao giờ có thể là NULL.

Tuy vậy, chỉ không có entry NULL là chưa đủ. Database phải chắc chắn rằng không bao giờ có entry NULL; nếu không, database phải giả định table có những row không nằm trong index.

Index sau chỉ hỗ trợ query nếu column LAST_NAME có NOT NULL constraint:

```
DROP INDEX emp_dob;
CREATE INDEX emp_dob_name
          ON employees (date_of_birth, last_name);

SELECT *
  FROM employees
 WHERE date_of_birth IS NULL;
```

```
-------------------------------------------------------------------
|Id |Operation                    | Name         | Rows | Cost |
-------------------------------------------------------------------
| 0 |SELECT STATEMENT             |              |    1 |    3 |
| 1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES     |    1 |    3 |
|*2 |  INDEX RANGE SCAN           | EMP_DOB_NAME |    1 |    2 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   2 - access("DATE_OF_BIRTH" IS NULL)
```

Gỡ NOT NULL constraint khiến index không thể dùng cho query này:

```
ALTER TABLE employees MODIFY last_name NULL;

SELECT *
  FROM employees
 WHERE date_of_birth IS NULL;
```

```
-----------------------------------------------------
| Id | Operation          | Name      | Rows | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT   |           |    1 |  477 |
|* 1 |  TABLE ACCESS FULL| EMPLOYEES  |    1 |  477 |
-----------------------------------------------------
```

> Mẹo
>
> Thiếu NOT NULL constraint có thể ngăn index được sử dụng trong Oracle database — đặc biệt với query count(*).

Ngoài NOT NULL constraint, database cũng biết các constant expression như ở phần trước không thể trở thành NULL.

Tuy nhiên, index trên user-defined function không áp đặt NOT NULL constraint lên index expression:

```
CREATE OR REPLACE FUNCTION blackbox(id IN NUMBER) RETURN NUMBER
DETERMINISTIC
IS BEGIN
   RETURN id;
END;

DROP INDEX emp_dob_name;
CREATE INDEX emp_dob_bb
    ON employees (date_of_birth, blackbox(employee_id));
```

```
SELECT *
  FROM employees
 WHERE date_of_birth IS NULL;
```

```
-----------------------------------------------------
| Id | Operation         | Name      | Rows | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT  |           |    1 |  477 |
|* 1 |  TABLE ACCESS FULL| EMPLOYEES |    1 |  477 |
-----------------------------------------------------
```

Tên function BLACKBOX nhấn mạnh rằng optimizer không biết function làm gì. Chúng ta thấy function truyền thẳng input value, nhưng với database, đó chỉ là một function trả về number. Tính chất NOT NULL của parameter bị mất. Dù index phải có mọi row, database không biết điều đó nên không thể dùng index cho query.

Nếu biết function không bao giờ trả về NULL như trong ví dụ này, bạn có thể thay đổi query để phản ánh điều đó:

```
SELECT *
  FROM employees
 WHERE date_of_birth IS NULL
   AND blackbox(employee_id) IS NOT NULL;
```

```
-------------------------------------------------------------
|Id |Operation                    | Name        | Rows | Cost |
-------------------------------------------------------------
| 0 |SELECT STATEMENT             |             |    1 |    3 |
| 1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES   |    1 |    3 |
|*2 |  INDEX RANGE SCAN           | EMP_DOB_BB  |    1 |    2 |
-------------------------------------------------------------
```

Điều kiện bổ sung trong where clause luôn đúng nên không thay đổi kết quả. Tuy nhiên, Oracle database nhận ra rằng query chỉ lấy các row chắc chắn phải nằm trong index theo định nghĩa.

Đáng tiếc là không có cách nào đánh dấu một function không bao giờ trả về NULL, nhưng bạn có thể chuyển lời gọi function vào một virtual column (từ 11g) và đặt NOT NULL constraint lên column đó.

```
ALTER TABLE employees ADD bb_expression
      GENERATED ALWAYS AS (blackbox(employee_id)) NOT NULL;

DROP   INDEX emp_dob_bb;
CREATE INDEX emp_dob_bb
    ON employees (date_of_birth, bb_expression);
```

```
SELECT *
  FROM employees
 WHERE date_of_birth IS NULL;
```

```
-------------------------------------------------------------
|Id |Operation                    | Name       | Rows | Cost |
-------------------------------------------------------------
| 0 |SELECT STATEMENT             |            |    1 |    3 |
| 1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES  |    1 |    3 |
|*2 |  INDEX RANGE SCAN           | EMP_DOB_BB |    1 |    2 |
-------------------------------------------------------------
```

Oracle database biết rằng một số internal function chỉ trả về NULL nếu input được cung cấp là NULL.

```
DROP INDEX emp_dob_bb;

CREATE INDEX emp_dob_upname
    ON employees (date_of_birth, upper(last_name));

SELECT *
  FROM employees
 WHERE date_of_birth IS NULL;
```

```
-------------------------------------------------------------
|Id |Operation                    | Name           | Cost |
-------------------------------------------------------------
| 0 |SELECT STATEMENT             |                |    3 |
| 1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES      |    3 |
|*2 |  INDEX RANGE SCAN           | EMP_DOB_UPNAME |    2 |
-------------------------------------------------------------
```

Function UPPER bảo toàn tính chất NOT NULL của column LAST_NAME. Tuy nhiên, gỡ constraint sẽ khiến index không thể dùng được:

```
ALTER TABLE employees MODIFY last_name NULL;

SELECT *
  FROM employees
 WHERE date_of_birth IS NULL;
```

```
-----------------------------------------------------
| Id | Operation         | Name      | Rows | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT  |           |    1 |  477 |
|* 1 |  TABLE ACCESS FULL| EMPLOYEES |    1 |  477 |
-----------------------------------------------------
```

Emulating Partial Indexes

Cách Oracle database xử lý NULL trong index một cách khác thường có thể được dùng để mô phỏng partial index. Để làm vậy, chúng ta chỉ cần dùng NULL cho những row không nên được index.

Để minh họa, chúng ta mô phỏng partial index sau:

```
CREATE INDEX messages_todo
          ON messages (receiver)
       WHERE processed = 'N'
```

Trước tiên, chúng ta cần một function chỉ trả về value RECEIVER khi value PROCESSED là 'N'.

```
CREATE OR REPLACE
FUNCTION pi_processed(processed CHAR, receiver NUMBER)
RETURN NUMBER
DETERMINISTIC
AS BEGIN
   IF processed IN ('N') THEN
      RETURN receiver;
   ELSE
      RETURN NULL;
   END IF;
END;
/
```

Function phải deterministic để có thể dùng trong index definition.

Bây giờ chúng ta có thể tạo index chỉ chứa các row có PROCESSED='N'.

```
CREATE INDEX messages_todo
          ON messages (pi_processed(processed, receiver));
```

Để sử dụng index, bạn phải dùng indexed expression trong query:

```
SELECT message
  FROM messages
 WHERE pi_processed(processed, receiver) = ?
```

```
-------------------------------------------------------------
|Id | Operation                    | Name          | Cost |
-------------------------------------------------------------
| 0 | SELECT STATEMENT             |               | 5330 |
| 1 |  TABLE ACCESS BY INDEX ROWID| MESSAGES      | 5330 |
|*2 |   INDEX RANGE SCAN           | MESSAGES_TODO | 5303 |
-------------------------------------------------------------
```

Partial Indexes, Part II

Từ release 11g, có một cách tiếp cận thứ hai — đáng sợ không kém — để mô phỏng partial index trong Oracle database: dùng một index partition bị làm hỏng có chủ đích cùng parameter SKIP_UNUSABLE_INDEX.

Obfuscated Conditions

Các phần sau trình bày một số phương pháp phổ biến để obfuscate condition. Obfuscated condition là where clause được viết theo cách ngăn index được sử dụng đúng mức. Đây là tập hợp các anti-pattern mà mọi developer nên biết và tránh.

Date Types

Phần lớn obfuscation liên quan đến DATE type. Oracle database đặc biệt dễ gặp vấn đề ở đây vì nó chỉ có một DATE type và type này luôn bao gồm cả time component.

Dùng function TRUNC để loại bỏ time component đã trở thành một practice phổ biến. Thực ra function này không loại bỏ time mà set nó về midnight, vì Oracle database không có pure DATE type. Để bỏ qua time component khi search, bạn có thể dùng function TRUNC ở cả hai phía của phép so sánh — ví dụ để tìm sales của ngày hôm qua:

```
SELECT ...
  FROM sales
 WHERE TRUNC(sale_date) = TRUNC(sysdate - INTERVAL '1' DAY)
```

Đây là một statement hoàn toàn hợp lệ và đúng, nhưng không thể sử dụng đúng mức index trên SALE_DATE. Như đã giải thích trong “Case-Insensitive Search Using UPPER or LOWER” ở trang 24, TRUNC(sale_date) hoàn toàn khác SALE_DATE — function là black box đối với database.

Có một giải pháp khá đơn giản cho vấn đề này: function-based index.

```
CREATE INDEX index_name
          ON table_name (TRUNC(sale_date))
```

Nhưng khi đó bạn phải luôn dùng TRUNC(date_column) trong where clause. Nếu dùng không nhất quán — lúc có, lúc không có TRUNC — bạn sẽ cần hai index!

Vấn đề tương tự cũng xảy ra với database có pure date type nếu bạn search một khoảng thời gian dài hơn, như trong MySQL query sau:

```
SELECT ...
  FROM sales
 WHERE DATE_FORMAT(sale_date, "%Y-%M")
     = DATE_FORMAT(now()    , "%Y-%M')
```

Query dùng date format chỉ chứa year và month: một lần nữa, đây là query hoàn toàn đúng nhưng có cùng vấn đề như trước. Tuy nhiên, giải pháp ở trên không áp dụng được vì MySQL không có function-based index.

Giải pháp thay thế là dùng explicit range condition. Đây là một giải pháp generic hoạt động với mọi database:

```
SELECT ...
  FROM sales
 WHERE sale_date BETWEEN quarter_begin(?)
                     AND quarter_end(?)
```

Nếu đã làm bài tập, có lẽ bạn nhận ra pattern từ exercise về tất cả employee 42 tuổi.

Một index thông thường trên SALE_DATE là đủ để optimize query này. Các function QUARTER_BEGIN và QUARTER_END tính boundary date. Việc tính toán có thể hơi phức tạp vì operator between luôn bao gồm các boundary value. Vì vậy, function QUARTER_END phải trả về một time stamp ngay trước ngày đầu tiên của quarter tiếp theo nếu SALE_DATE có time component. Logic này có thể được ẩn trong function.

Các ví dụ ở những trang sau cho thấy cách implement function QUARTER_BEGIN và QUARTER_END cho nhiều database khác nhau.

MySQL

```
CREATE FUNCTION quarter_begin(dt DATETIME)
RETURNS DATETIME DETERMINISTIC
RETURN CONVERT
       (
         CONCAT
         ( CONVERT(YEAR(dt),CHAR(4))
         , '-'
         , CONVERT(QUARTER(dt)*3-2,CHAR(2))
         , '-01'
         )
       , datetime
       );

CREATE FUNCTION quarter_end(dt DATETIME)
RETURNS DATETIME DETERMINISTIC
RETURN DATE_ADD
       ( DATE_ADD ( quarter_begin(dt), INTERVAL 3 MONTH )
       , INTERVAL -1 MICROSECOND);
```

Oracle Database

```
CREATE FUNCTION quarter_begin(dt IN DATE)
RETURN DATE
AS
BEGIN
   RETURN TRUNC(dt, 'Q');
END;
/

CREATE FUNCTION quarter_end(dt IN DATE)
RETURN DATE
AS
BEGIN
   -- the Oracle DATE type has seconds resolution
   -- subtract one second from the first
   -- day of the following quarter
   RETURN TRUNC(ADD_MONTHS(dt, +3), 'Q')
        - (1/(24*60*60));
END;
/
```

PostgreSQL

```
CREATE FUNCTION quarter_begin(dt timestamp with time zone)
RETURNS timestamp with time zone AS $$
BEGIN
    RETURN date_trunc('quarter', dt);
END;
$$ LANGUAGE plpgsql;

CREATE FUNCTION quarter_end(dt timestamp with time zone)
RETURNS timestamp with time zone AS $$
BEGIN
   RETURN   date_trunc('quarter', dt)
          + interval '3 month'
          - interval '1 microsecond';
END;
$$ LANGUAGE plpgsql;
```

SQL Server

```
CREATE FUNCTION quarter_begin (@dt DATETIME )
RETURNS DATETIME
BEGIN
  RETURN DATEADD (qq, DATEDIFF (qq, 0, @dt), 0)
END
GO

CREATE FUNCTION quarter_end (@dt DATETIME )
RETURNS DATETIME
BEGIN
  RETURN DATEADD
          ( ms
          , -3
          , DATEADD(mm, 3, dbo.quarter_begin(@dt))
          );
END
GO
```

Bạn có thể dùng các auxiliary function tương tự cho những period khác — phần lớn sẽ đơn giản hơn các ví dụ trên, đặc biệt khi dùng condition lớn hơn hoặc bằng (>=) và nhỏ hơn (<) thay cho operator between. Tất nhiên, nếu muốn bạn cũng có thể tính boundary date trong application.

> Mẹo
>
> Viết query cho các period liên tục dưới dạng explicit range condition. Hãy làm vậy ngay cả với một ngày — ví dụ, trong Oracle database:

```
    sale_date >= TRUNC(sysdate)
AND sale_date <  TRUNC(sysdate + INTERVAL '1' DAY)
```

Một obfuscation phổ biến khác là so sánh date dưới dạng string, như trong PostgreSQL example sau:

```
SELECT ...
  FROM sales
 WHERE TO_CHAR(sale_Date, 'YYYY-MM-DD') = '1970-01-01'
```

Vấn đề một lần nữa là việc convert DATE_COLUMN. Những condition như vậy thường được tạo ra vì nghĩ rằng database chỉ nhận được type number và string. Tuy nhiên, bind parameter hỗ trợ mọi data type. Chẳng hạn, bạn có thể dùng object java.util.Date làm bind parameter. Đây là một lợi ích khác của bind parameter.

Nếu không thể làm vậy, bạn chỉ cần convert search term thay vì table column:

```
SELECT ...
  FROM sales
 WHERE sale_date = TO_DATE('1970-01-01', 'YYYY-MM-DD')
```

Query này có thể dùng một index thông thường trên SALE_DATE. Hơn nữa, nó chỉ convert input string một lần. Statement trước phải convert mọi date được lưu trong table trước khi có thể so sánh chúng với search term.

Dù thay đổi theo cách nào — dùng bind parameter hay convert phía còn lại của phép so sánh — bạn đều có thể dễ dàng tạo bug nếu SALE_DATE có time component. Khi đó, bạn phải dùng explicit range condition:

```
SELECT ...
  FROM sales
 WHERE sale_date >= TO_DATE('1970-01-01', 'YYYY-MM-DD')
   AND sale_date <  TO_DATE('1970-01-01', 'YYYY-MM-DD')
                  + INTERVAL '1' DAY
```

Luôn cân nhắc dùng explicit range condition khi so sánh date.

LIKE on Date Types

Obfuscation sau đây đặc biệt khó nhận ra:

```
sale_date LIKE SYSDATE
```

Thoạt nhìn, nó không giống obfuscation vì không dùng function nào.

Tuy nhiên, operator LIKE bắt buộc phải so sánh string. Tùy database, điều đó có thể gây error hoặc tạo implicit type conversion ở cả hai phía. Phần “Predicate Information” của execution plan cho thấy Oracle database xử lý thế nào:

```
filter( INTERNAL_FUNCTION(SALE_DATE)
   LIKE TO_CHAR(SYSDATE@!))
```

Function INTERNAL_FUNCTION convert type của column SALE_DATE. Một tác dụng phụ là nó cũng ngăn việc dùng index thông thường trên DATE_COLUMN, giống như mọi function khác.

Numeric Strings

Numeric string là number được lưu trong text column. Dù đây là một practice rất tệ, nó không tự động làm index vô dụng nếu bạn nhất quán xử lý value đó như string:

```
SELECT ...
  FROM ...
 WHERE numeric_string = '42'
```

Dĩ nhiên statement này có thể dùng index trên NUMERIC_STRING. Tuy nhiên, nếu so sánh bằng number thì database không còn dùng được condition này làm access predicate.

```
SELECT ...
  FROM ...
 WHERE numeric_string = 42
```

Hãy chú ý dấu nháy bị thiếu. Dù một số database báo error (ví dụ PostgreSQL), nhiều database chỉ thêm implicit type conversion.

```
SELECT ...
  FROM ...
 WHERE TO_NUMBER(numeric_string) = 42
```

Đây là cùng vấn đề như trước. Index trên NUMERIC_STRING không thể dùng được do lời gọi function. Giải pháp cũng giống như trước: đừng convert table column, hãy convert search term.

```
SELECT ...
  FROM ...
 WHERE numeric_string = TO_CHAR(42)
```

Bạn có thể thắc mắc tại sao database không tự làm theo cách này. Lý do là convert string thành number luôn cho một kết quả không mơ hồ. Chiều ngược lại thì không đúng. Một number được format thành text có thể chứa space, punctuation và leading zero. Một value có thể được viết theo nhiều cách:

```
42
042
0042
00042
...
```

Numeric Strings

Database không thể biết number format được dùng trong column NUMERIC_STRING nên nó làm theo chiều ngược lại: database convert string thành number — đây là phép biến đổi không mơ hồ.

Function TO_CHAR chỉ trả về một string representation của number. Vì vậy, nó chỉ match string đầu tiên trong danh sách trên. Nếu dùng TO_NUMBER, nó match tất cả string. Điều đó có nghĩa hai biến thể không chỉ khác nhau về performance mà còn khác nhau về semantic!

Dùng numeric string nhìn chung gây nhiều vấn đề: quan trọng nhất là nó gây performance problem do implicit conversion và tạo rủi ro gặp conversion error vì number không hợp lệ. Ngay cả query đơn giản nhất, không dùng function nào trong where clause, cũng có thể abort với conversion error nếu chỉ cần một invalid number được lưu trong table.

> Mẹo
>
> Dùng numeric type để lưu number.

Lưu ý rằng vấn đề không xảy ra theo chiều ngược lại:

```
SELECT ...
  FROM ...
 WHERE numeric_number = '42'
```

Database sẽ nhất quán transform string thành number. Nó không áp dụng function lên column có thể được index, vì vậy regular index vẫn hoạt động. Tuy nhiên, bạn vẫn có thể tự convert theo cách sai:

```
SELECT ...
  FROM ...
 WHERE TO_CHAR(numeric_number) = '42'
```

Combining Columns

Phần này nói về một obfuscation phổ biến ảnh hưởng đến concatenated index.

Ví dụ đầu tiên lại nói về date và time type nhưng theo chiều ngược lại. MySQL query sau kết hợp một date column và một time column để áp dụng range filter lên cả hai:

```
SELECT ...
  FROM ...
 WHERE ADDTIME(date_column, time_column)
     > DATE_ADD(now(), INTERVAL -1 DAY)
```

Nó chọn mọi record trong 24 giờ vừa qua. Query không thể dùng đúng mức concatenated index trên (DATE_COLUMN, TIME_COLUMN) vì search không thực hiện trên các column được index mà trên derived data.

Bạn có thể tránh vấn đề này bằng data type có cả date và time component (ví dụ MySQL DATETIME). Khi đó, bạn có thể dùng column này mà không cần function call:

```
SELECT ...
  FROM ...
 WHERE datetime_column
     > DATE_ADD(now(), INTERVAL -1 DAY)
```

Đáng tiếc là khi gặp vấn đề này, thường không thể thay đổi table.

Lựa chọn tiếp theo là function-based index nếu database hỗ trợ — dù cách này có mọi drawback đã bàn trước đó. Với MySQL, function-based index vốn không phải là một option.

Tuy vậy, vẫn có thể viết query để database dùng concatenated index trên DATE_COLUMN, TIME_COLUMN với access predicate — ít nhất là một phần. Để làm vậy, chúng ta thêm một condition vào DATE_COLUMN.

```
 WHERE ADDTIME(date_column, time_column)
     > DATE_ADD(now(), INTERVAL -1 DAY)
   AND date_column
    >= DATE(DATE_ADD(now(), INTERVAL -1 DAY))
```

Condition mới hoàn toàn redundant nhưng là một filter trực tiếp trên DATE_COLUMN và có thể được dùng làm access predicate. Dù kỹ thuật này không hoàn hảo, nó thường là một approximation đủ tốt.

> Mẹo
>
> Dùng một redundant condition trên column có significance cao nhất khi một range condition kết hợp nhiều column.
>
> Với PostgreSQL, nên dùng row values syntax được mô tả ở trang 151.

Bạn cũng có thể dùng kỹ thuật này khi lưu date và time trong text column, nhưng phải dùng date format và time format tạo ra thứ tự chronological khi sort theo lexical order — ví dụ format được ISO 8601 gợi ý (YYYY-MM-DD HH:MM:SS). Ví dụ sau dùng function TO_CHAR của Oracle database cho mục đích đó:

```
SELECT ...
  FROM ...
 WHERE date_string || time_string
     > TO_CHAR(sysdate - 1, 'YYYY-MM-DD HH24:MI:SS')
   AND date_string
    >= TO_CHAR(sysdate - 1, 'YYYY-MM-DD')
```

Chúng ta sẽ lại gặp vấn đề áp dụng range condition trên nhiều column trong phần “Paging Through Results”. Chúng ta cũng sẽ dùng cùng phương pháp approximation để giảm nhẹ vấn đề.

Đôi khi chúng ta gặp trường hợp ngược lại và muốn cố ý obfuscate condition để nó không còn được dùng làm access predicate. Chúng ta đã xem vấn đề này khi bàn về ảnh hưởng của bind parameter lên LIKE condition. Hãy xét ví dụ sau:

```
SELECT last_name, first_name, employee_id
  FROM employees
 WHERE subsidiary_id = ?
   AND last_name LIKE ?
```

Giả sử có một index trên SUBSIDIARY_ID và một index khác trên LAST_NAME, index nào tốt hơn cho query này?

Không biết vị trí wildcard trong search term thì không thể đưa ra câu trả lời có cơ sở. Optimizer không còn lựa chọn nào khác ngoài việc "đoán". Nếu biết luôn có leading wildcard, bạn có thể cố ý obfuscate LIKE condition để optimizer không còn xem xét index trên LAST_NAME.

```
SELECT last_name, first_name, employee_id
  FROM employees
 WHERE subsidiary_id = ?
   AND last_name || '' LIKE ?
```

Chỉ cần append một empty string vào column LAST_NAME. Tuy nhiên, đây là option cuối cùng. Chỉ dùng khi thật sự cần thiết.

Smart Logic

Một trong những tính năng cốt lõi của SQL database là hỗ trợ ad-hoc query: query mới có thể được thực thi bất cứ lúc nào. Điều này chỉ khả thi vì query optimizer (query planner) hoạt động ở runtime; nó phân tích từng statement khi nhận được và lập tức tạo execution plan hợp lý. Bind parameter có thể giảm overhead do runtime optimization gây ra.

Tóm lại, database được tối ưu cho dynamic SQL — vì vậy hãy dùng nó nếu cần.

Tuy nhiên, có một practice phổ biến là tránh dynamic SQL để dùng static SQL — thường vì myth "dynamic SQL chậm". Practice này gây hại nhiều hơn lợi nếu database dùng shared execution plan cache như DB2, Oracle database hoặc SQL Server.

Để minh họa, hãy hình dung một application query table EMPLOYEES. Application cho phép search subsidiary id, employee id và last name (case-insensitive) theo mọi tổ hợp. Vẫn có thể viết một

query duy nhất bao phủ mọi trường hợp bằng “smart” logic.

Smart Logic

```
SELECT first_name, last_name, subsidiary_id, employee_id
  FROM employees
 WHERE ( subsidiary_id    = :sub_id OR :sub_id IS NULL )
   AND ( employee_id      = :emp_id OR :emp_id IS NULL )
   AND ( UPPER(last_name) = :name   OR :name   IS NULL )
```

Query dùng named bind variable để dễ đọc hơn. Mọi filter expression có thể có đều được viết tĩnh trong statement. Khi không cần một filter, bạn chỉ cần dùng NULL thay cho search term: logic OR sẽ disable condition đó.

Đây là một SQL statement hoàn toàn hợp lý. Việc dùng NULL thậm chí còn phù hợp với định nghĩa của nó theo three-valued logic của SQL. Tuy nhiên, đây lại là một trong những performance anti-pattern tệ nhất.

Database không thể optimize execution plan cho một filter cụ thể vì bất kỳ filter nào cũng có thể bị hủy ở runtime. Database phải chuẩn bị cho trường hợp xấu nhất — khi tất cả filter đều bị disable:

```
-----------------------------------------------------
| Id | Operation         | Name      | Rows | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT  |           |    2 |  478 |
|* 1 |  TABLE ACCESS FULL| EMPLOYEES |    2 |  478 |
-----------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
1 - filter((:NAME   IS NULL OR UPPER("LAST_NAME")=:NAME)
       AND (:EMP_ID IS NULL OR "EMPLOYEE_ID"=:EMP_ID)
       AND (:SUB_ID IS NULL OR "SUBSIDIARY_ID"=:SUB_ID))
```

Vì vậy, database dùng full table scan ngay cả khi có index cho từng column.

Không phải database không thể resolve “smart” logic. Database tạo generic execution plan vì bind parameter cho phép plan được cache và re-use với các value khác về sau. Nếu không dùng bind parameter mà viết actual value vào SQL statement, optimizer sẽ chọn index phù hợp cho filter đang active:

```
SELECT first_name, last_name, subsidiary_id, employee_id
  FROM employees
 WHERE( subsidiary_id    = NULL     OR NULL IS NULL )
   AND( employee_id      = NULL     OR NULL IS NULL )
   AND( UPPER(last_name) = 'WINAND' OR 'WINAND' IS NULL )
```

```
-------------------------------------------------------------------
|Id | Operation                    | Name        | Rows | Cost |
-------------------------------------------------------------------
| 0 | SELECT STATEMENT             |             |    1 |    2 |
| 1 |  TABLE ACCESS BY INDEX ROWID| EMPLOYEES   |    1 |    2 |
|*2 |   INDEX RANGE SCAN           | EMP_UP_NAME |    1 |    1 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
  2 - access(UPPER("LAST_NAME")='WINAND')
```

Tuy nhiên, đây không phải là giải pháp. Nó chỉ chứng minh rằng database có thể resolve các condition này.

> Warning
>
> Dùng literal value khiến application của bạn dễ bị SQL injection attack và có thể gây performance problem do optimization overhead tăng lên.

Giải pháp hiển nhiên cho dynamic query là dynamic SQL. Theo nguyên tắc KISS, hãy chỉ nói cho database biết bạn cần gì ngay lúc này — và không gì khác.

```
SELECT first_name, last_name, subsidiary_id, employee_id
  FROM employees
 WHERE UPPER(last_name) = :name
```

Lưu ý rằng query này dùng một bind parameter.

> Tip
>
> Dùng dynamic SQL nếu bạn cần dynamic where clause.
>
> Vẫn dùng bind parameter khi generate dynamic SQL — nếu không, myth "dynamic SQL chậm" sẽ trở thành sự thật.

Vấn đề được mô tả trong phần này rất phổ biến. Mọi database dùng shared execution plan cache đều có một feature để xử lý nó — thường lại đưa vào những problem và bug mới.

3

http://en.wikipedia.org/wiki/KISS_principle

Smart Logic

MySQL

MySQL không gặp vấn đề cụ thể này vì nó hoàn toàn không có execution plan cache. Một feature request từ năm 2009 bàn về ảnh hưởng của execution plan caching. Có vẻ optimizer của MySQL đủ đơn giản để execution plan caching không mang lại lợi ích.

Oracle Database

Oracle database dùng shared execution plan cache ("SQL area") và hoàn toàn chịu ảnh hưởng của vấn đề được mô tả trong phần này.

Oracle giới thiệu bind peeking từ release 9i. Bind peeking cho phép optimizer dùng bind value thực tế của lần execute đầu tiên khi chuẩn bị execution plan. Vấn đề của cách tiếp cận này là behavior

không deterministic: value của lần execute đầu tiên ảnh hưởng đến mọi lần execute. Execution plan có thể thay đổi khi database restart hoặc, khó đoán hơn, khi cached plan hết hạn và optimizer tạo lại nó bằng các value khác vào lần statement được execute tiếp theo.

Release 11g giới thiệu adaptive cursor sharing để cải thiện tình hình hơn nữa. Feature này cho phép database cache nhiều execution plan cho cùng một SQL statement. Ngoài ra, optimizer peek bind parameter và lưu estimated selectivity của chúng cùng với execution plan.

Khi cache được truy cập sau đó, selectivity của bind value hiện tại phải nằm trong các selectivity range của một cached execution plan thì plan đó mới được re-use. Nếu không, optimizer tạo execution plan mới và so sánh nó với các execution plan đã cache cho query này. Nếu đã có execution plan như vậy, database thay thế nó bằng execution plan mới cũng bao phủ các selectivity estimate của bind value hiện tại. Nếu chưa có, database cache một execution plan variant mới cho query này — dĩ nhiên kèm theo các selectivity estimate.

PostgreSQL

PostgreSQL query plan cache chỉ hoạt động với statement đang mở — tức là chừng nào bạn còn giữ PreparedStatement mở. Vấn đề được mô tả ở trên chỉ xảy ra khi re-use statement handle. Lưu ý rằng JDBC driver của PostgresSQL chỉ enable cache sau lần execute thứ năm.

SQL Server

SQL Server dùng parameter sniffing. Parameter sniffing cho phép optimizer dùng bind value thực tế của lần execute đầu tiên trong quá trình parsing. Vấn đề của cách tiếp cận này là behavior không deterministic: value của lần execute đầu tiên ảnh hưởng đến mọi lần

execute. Execution plan có thể thay đổi khi database restart hoặc, khó đoán hơn, khi cached plan hết hạn và optimizer tạo lại nó bằng các value khác vào lần statement được execute tiếp theo.

SQL Server 2005 bổ sung các query hint mới để kiểm soát parameter sniffing và việc recompile tốt hơn. Query hint RECOMPILE bypass plan cache cho một statement được chọn. OPTIMIZE FOR cho phép chỉ định các parameter value được dùng riêng cho optimization. Cuối cùng, bạn có thể cung cấp toàn bộ execution plan bằng hint USE PLAN.

Implementation ban đầu của hint OPTION(RECOMPILE) có một bug nên không xem xét tất cả bind variable. Implementation mới được giới thiệu trong SQL Server 2008 lại có một bug khác, khiến tình hình trở nên rất confusing. Erland Sommarskog đã tập hợp toàn bộ thông tin liên quan cho tất cả release của SQL Server.

4

http://www.sommarskog.se/dyn-search-2008.html

Các heuristic method có thể cải thiện vấn đề “smart logic” ở một mức độ nhất định, nhưng thực ra chúng được xây dựng để xử lý vấn đề của bind parameter liên quan đến column histogram và LIKE expression.

Phương pháp đáng tin cậy nhất để đạt được execution plan tốt nhất là tránh các filter không cần thiết trong SQL statement.

Math

Math

Còn một nhóm obfuscation nữa cũng rất “smart” và ngăn index được sử dụng đúng. Thay vì dùng logic expression, nó dùng phép tính.

Hãy xét statement sau. Nó có thể dùng index trên NUMERIC_NUMBER không?

```
SELECT numeric_number
  FROM table_name
 WHERE numeric_number - 1000 > ?
```

Tương tự, statement sau có thể dùng index trên A và B — bạn tự chọn thứ tự — không?

```
SELECT a, b
  FROM table_name
 WHERE 3*a + 5 = b
```

Hãy đặt các câu hỏi này dưới một góc nhìn khác; nếu bạn đang phát triển một SQL database, bạn có bổ sung equation solver không? Phần lớn database vendor chỉ trả lời "No!" và vì vậy, cả hai ví dụ đều không dùng được index.

Bạn thậm chí có thể cố ý dùng phép toán để obfuscate một condition — như chúng ta đã làm trước đó với full text LIKE search. Chỉ cần thêm zero là đủ, chẳng hạn:

```
SELECT numeric_number
  FROM table_name
 WHERE numeric_number + 0 = ?
```

Tuy vậy, chúng ta có thể index các expression này bằng function-based index nếu dùng phép tính theo cách thông minh và biến where clause thành một phương trình:

```
SELECT a, b
  FROM table_name
 WHERE 3*a - b = -5
```

Chúng ta chỉ chuyển các table reference sang một vế và constant sang vế còn lại. Sau đó có thể tạo function-based index cho vế trái của phương trình:

```
CREATE INDEX math ON table_name (3*a - b)
```

Chapter 3

Performance and Scalability

Chương này nói về performance và scalability của database.

Trong ngữ cảnh này, tôi dùng định nghĩa sau cho scalability:

Scalability là khả năng của một system, network hoặc process xử lý một lượng công việc ngày càng tăng theo cách có năng lực,

hoặc

khả năng được mở rộng để đáp ứng mức tăng trưởng đó.

—Wikipedia

Bạn thấy rằng thực ra có hai định nghĩa. Định nghĩa thứ nhất nói về tác động của tải tăng lên một system, còn định nghĩa thứ hai nói về việc mở rộng system để xử lý nhiều tải hơn.

Định nghĩa thứ hai phổ biến hơn nhiều so với định nghĩa thứ nhất. Mỗi khi ai đó nói về scalability, gần như lúc nào họ cũng nói về việc dùng thêm hardware. Scale-up và scale-out là hai keyword tương ứng, gần đây được bổ sung bởi những buzzword mới như web-scale.

Nói rộng ra, scalability nói về tác động performance của các thay đổi trong môi trường. Hardware chỉ là một parameter môi trường có thể thay đổi. Chương này cũng đề cập đến các parameter khác như data volume và system load.

http://en.wikipedia.org/wiki/Scalability

Performance Impacts of Data Volume

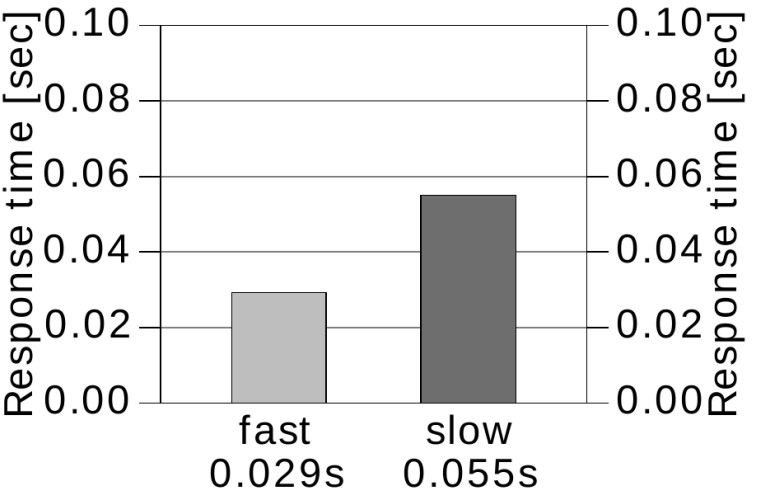
Lượng data được lưu trong database có ảnh hưởng lớn đến performance. Thông thường người ta chấp nhận rằng query sẽ chậm hơn khi database có thêm data. Nhưng performance bị ảnh hưởng nhiều đến mức nào nếu data volume tăng gấp đôi? Và làm thế nào để cải thiện tỷ lệ này? Đây là những câu hỏi then chốt khi bàn về database scalability.

Để làm ví dụ, chúng ta phân tích response time của query sau khi dùng hai index khác nhau. Tạm thời chưa biết các index definition — chúng sẽ được tiết lộ trong quá trình thảo luận.

```
SELECT count(*)
  FROM scale_data
 WHERE section = ?
   AND id2 = ?
```

Column SECTION có một mục đích đặc biệt trong query này: nó kiểm soát data volume. SECTION càng lớn thì query càng select nhiều row. Figure 3.1 cho thấy response time với một SECTION nhỏ.

Figure 3.1. So sánh performance

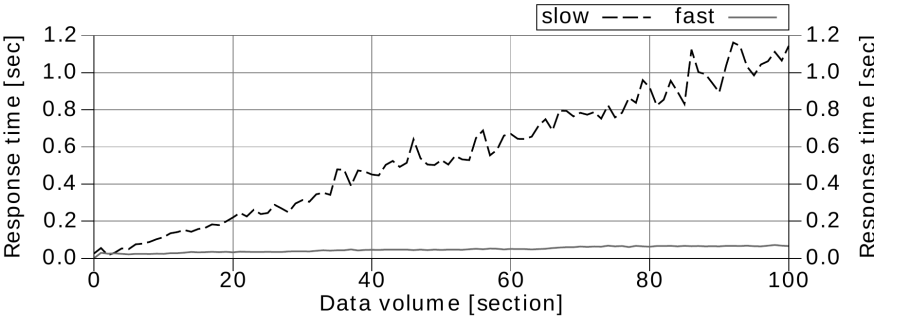


Có khác biệt đáng kể về performance giữa hai variant của index. Cả hai response time vẫn thấp hơn nhiều so với một phần mười giây, nên trong hầu hết trường hợp query chậm hơn có lẽ vẫn đủ nhanh. Tuy nhiên, biểu đồ performance chỉ cho thấy một test point. Bàn về scalability nghĩa là phải xem tác động performance khi thay đổi các parameter môi trường — chẳng hạn data volume.

> Important
>
> Scalability cho thấy dependency của performance vào các factor như data volume.
>
> Một performance value chỉ là một data point trên scalability chart.

Figure 3.2 cho thấy response time theo số SECTION — tức là theo data volume đang tăng.

Figure 3.2. Scalability theo Data Volume



Biểu đồ cho thấy response time tăng với cả hai index. Ở phía bên phải biểu đồ, khi data volume cao gấp một trăm lần, query nhanh hơn cần thời gian lâu hơn ban đầu hơn hai lần, trong khi response time của query chậm hơn tăng gấp 20 lần lên hơn một giây.

Response time của một SQL query phụ thuộc vào nhiều factor. Data volume là một trong số đó. Nếu query đủ nhanh trong một số điều kiện test nhất định, điều đó không có nghĩa là nó sẽ đủ nhanh trong production. Điều này đặc biệt đúng với môi trường development chỉ có một phần nhỏ data so với production system.

Tuy nhiên, query chậm hơn khi data volume tăng không có gì bất ngờ. Điều hơi bất ngờ là khoảng cách lớn giữa hai index. Lý do khiến tốc độ tăng khác nhau là gì?

So sánh hai execution plan sẽ giúp tìm ra lý do khá dễ dàng.

```
-----------------------------------------------------
| Id | Operation         | Name       | Rows  | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT  |            |     1 |  972 |
|  1 |  SORT AGGREGATE   |            |     1 |      |
|* 2 |   INDEX RANGE SCAN| SCALE_SLOW |  3000 |  972 |
-----------------------------------------------------


-----------------------------------------------------
```

```
| Id   Operation          | Name       | Rows  | Cost |
-----------------------------------------------------------
|  0 | SELECT STATEMENT  |            |     1 |   13 |
|  1 |  SORT AGGREGATE   |            |     1 |      |
|* 2 |   INDEX RANGE SCAN| SCALE_FAST |  3000 |   13 |
-----------------------------------------------------------
```

Các execution plan gần như giống hệt nhau — chúng chỉ dùng một index khác nhau. Dù cost value phản ánh khác biệt về tốc độ, lý do không thể nhìn thấy trong execution plan.

Có vẻ chúng ta đang gặp một “slow index experience”; query chậm dù có dùng index. Tuy vậy, chúng ta không còn tin vào myth về “broken index”. Thay vào đó, hãy nhớ hai yếu tố khiến index lookup chậm: (1) table access và (2) scan một index range rộng.

Không execution plan nào hiển thị operation TABLE ACCESS BY INDEX ROWID, nên một execution plan chắc chắn phải scan một index range rộng hơn execution plan kia. Vậy execution plan hiển thị index range được scan ở đâu? Tất nhiên là trong predicate information!

> Tip
>
> Hãy chú ý đến predicate information.

Predicate information hoàn toàn không phải là detail không cần thiết có thể bỏ qua như ở trên. Execution plan không có predicate information là không đầy đủ. Điều đó có nghĩa bạn không thể thấy lý do khác biệt về performance trong các plan ở trên. Nếu xem execution plan đầy đủ, chúng ta sẽ thấy khác biệt.

```
-----------------------------------------------------
| Id | Operation         | Name       | Rows  | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT  |            |     1 |  972 |
|  1 |  SORT AGGREGATE   |            |     1 |      |
|* 2 |   INDEX RANGE SCAN| SCALE_SLOW |  3000 |  972 |
-----------------------------------------------------

Predicate Information (identified by operation id):
   2 - access("SECTION"=TO_NUMBER(:A))
       filter("ID2"=TO_NUMBER(:B))

-----------------------------------------------------
```

```
| Id   Operation         | Name       | Rows  | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT  |            |     1 |   13 |
|  1 |  SORT AGGREGATE   |            |     1 |      |
|* 2 |   INDEX RANGE SCAN| SCALE_FAST |  3000 |   13 |
-----------------------------------------------------

Predicate Information (identified by operation id):
   2 - access("SECTION"=TO_NUMBER(:A) AND "ID2"=TO_NUMBER(:B))
```

> Note
>
> Execution plan đã được đơn giản hóa để dễ hiểu. Appendix ở page 170 giải thích chi tiết phần “Predicate Information” trong Oracle execution plan.

Khác biệt giờ đã rõ: khi dùng index SCALE_SLOW, chỉ condition trên SECTION là access predicate. Database đọc mọi row trong section rồi loại những row không khớp filter predicate trên ID2. Response time tăng theo số row trong section. Với index SCALE_FAST, database dùng mọi condition làm access predicate. Response time tăng theo số row được select.

> Important
>
> Filter predicate giống như thiết bị nổ chưa kích hoạt. Chúng có thể phát nổ bất kỳ lúc nào.

Mảnh ghép cuối cùng trong câu đố là index definition. Chúng ta có thể suy ra index definition từ execution plan không?

Definition của index SACLE_SLOW phải bắt đầu bằng column SECTION — nếu không, nó không thể được dùng làm access predicate. Condition trên ID2 không phải access predicate — vì vậy nó không thể đứng sau SECTION trong index definition. Điều đó có nghĩa index SACLE_SLOW phải có tối thiểu ba column, trong đó SECTION là column đầu tiên và ID2 không phải column thứ hai. Đó chính xác là index definition được dùng cho test này:

```
CREATE INDEX scale_slow ON scale_data (section, id1, id2);
```

Database không thể dùng ID2 làm access predicate vì column ID1 đứng ở vị trí thứ hai.

Definition của index SCALE_FAST phải có column SECTION và ID2 ở hai vị trí đầu vì cả hai đều được dùng làm access predicate. Tuy vậy, chúng ta không thể nói gì về thứ tự của chúng. Index được dùng cho test bắt đầu bằng column SECTION và có thêm column ID1 ở vị trí thứ ba:

```
CREATE INDEX scale_fast ON scale_data (section, id2, id1);
```

Column ID1 chỉ được thêm vào để index này có cùng kích thước với SCALE_SLOW — nếu không, bạn có thể nghĩ rằng kích thước là nguyên nhân tạo ra khác biệt.

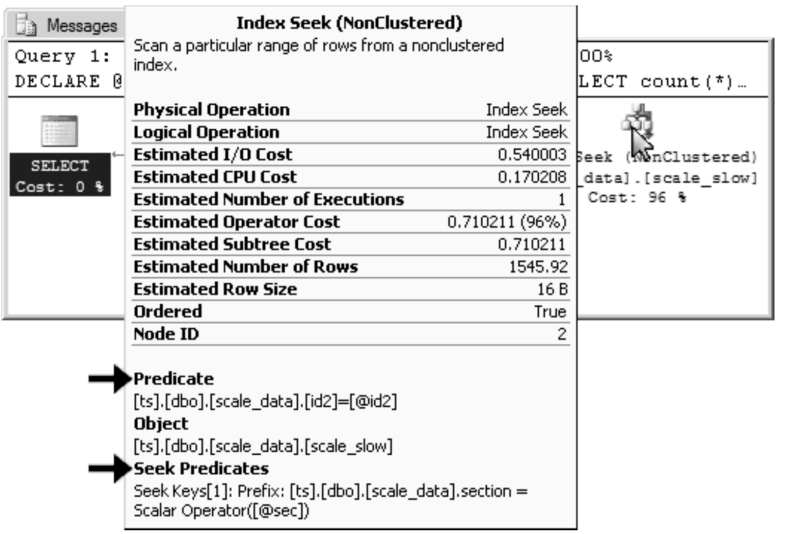
Performance Impacts of System Load

Việc cân nhắc cách định nghĩa multi-column index thường dừng lại ngay khi index được dùng cho query đang tune. Tuy nhiên, optimizer không dùng một index vì đó là index “đúng” cho query, mà vì nó hiệu quả hơn full table scan. Điều đó không có nghĩa nó là index tối ưu cho query.

Ví dụ trước cho thấy khó nhận ra column order sai trong execution plan. Predicate information thường bị ẩn khá kỹ, nên bạn phải chủ động tìm nó để kiểm tra index usage có tối ưu hay không.

Chẳng hạn, SQL Server Management Studio chỉ hiển thị predicate information dưới dạng tool tip khi di chuột lên index operation ("hover"). Execution plan sau dùng index SCALE_SLOW; vì vậy nó hiển thị condition trên ID2 dưới dạng filter predicate (chỉ là "Predicate", không có Seek).

Figure 3.3. Predicate Information dưới dạng Tool Tip

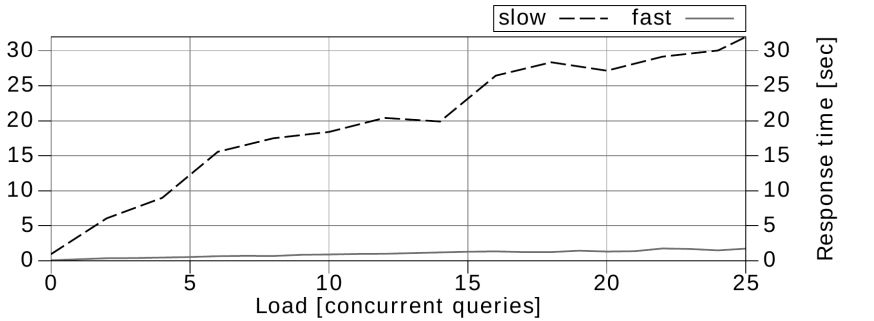


Lấy predicate information từ execution plan của MySQL hoặc PostgreSQL thậm chí còn awkward hơn. Appendix A ở page 165 có chi tiết.

Dù predicate information có vẻ không đáng kể đến đâu trong execution plan, nó vẫn có ảnh hưởng lớn đến performance — đặc biệt khi system phát triển. Hãy nhớ rằng không chỉ data volume tăng mà access rate cũng tăng. Đây là một parameter khác của scalability function.

Figure 3.4 biểu diễn response time theo access rate — data volume giữ nguyên. Nó cho thấy execution time của cùng query như trước và luôn dùng section có data volume lớn nhất. Điều đó có nghĩa point cuối cùng của Figure 3.2 ở page 81 tương ứng với point đầu tiên trong chart này.

Figure 3.4. Scalability theo System Load



Đường nét đứt biểu diễn response time khi dùng index SCALE_SLOW. Nó tăng lên đến 32 giây khi có 25 query chạy đồng thời. So với response time không có background load — như có thể xảy ra trong môi trường development — thời gian này lâu hơn 30 lần. Ngay cả khi có full copy của production database trong môi trường development, background load vẫn có thể khiến query chạy chậm hơn nhiều trong production.

Đường liền biểu diễn response time khi dùng index SCALE_FAST — index này không có filter predicate nào. Response time vẫn thấp hơn hai giây ngay cả khi có 25 query chạy đồng thời.

> Note
>
> Kiểm tra execution plan cẩn thận tạo ra độ tin cậy cao hơn benchmark hời hợt.
>
> Một full stress test vẫn đáng làm — nhưng cost rất cao.

Response Time and Throughput

Response time đáng ngờ thường bị xem nhẹ trong quá trình development. Phần lớn là vì chúng ta kỳ vọng “production hardware mạnh hơn” sẽ mang lại performance tốt hơn. Nhưng thường thì ngược lại, vì production infrastructure phức tạp hơn và tích lũy những latency không xuất hiện trong môi trường development. Ngay cả khi test trên infrastructure tương đương production, background load vẫn có thể gây ra response time khác nhau. Phần tiếp theo sẽ cho thấy nhìn chung không hợp lý khi kỳ vọng response nhanh hơn từ “hardware lớn hơn”.

Response Time and Throughput

Hardware lớn hơn không phải lúc nào cũng nhanh hơn — nhưng thường có thể xử lý nhiều load hơn. Hardware lớn hơn giống một xa lộ rộng hơn là một chiếc xe nhanh hơn: bạn không thể lái nhanh hơn — thực ra là không được phép — chỉ vì có thêm nhiều làn đường. Đó là lý do thêm hardware không tự động cải thiện SQL query chậm.

Chúng ta không còn ở những năm 1990 nữa. Khi đó, computing power của single-core CPU tăng rất nhanh. Phần lớn vấn đề response time biến mất trên hardware mới chỉ vì CPU được cải thiện. Nó giống như các model xe mới liên tục chạy nhanh gấp đôi model cũ mỗi năm! Tuy nhiên, single-core CPU power đã chạm giới hạn trong vài năm đầu của thế kỷ 21. Trục này gần như không còn cải thiện nữa. Để tiếp tục tạo ra CPU ngày càng mạnh, vendor phải chuyển sang chiến lược multi-core. Dù cho phép nhiều task chạy đồng thời, cách này không cải thiện performance nếu chỉ có một task. Performance có nhiều hơn một dimension.

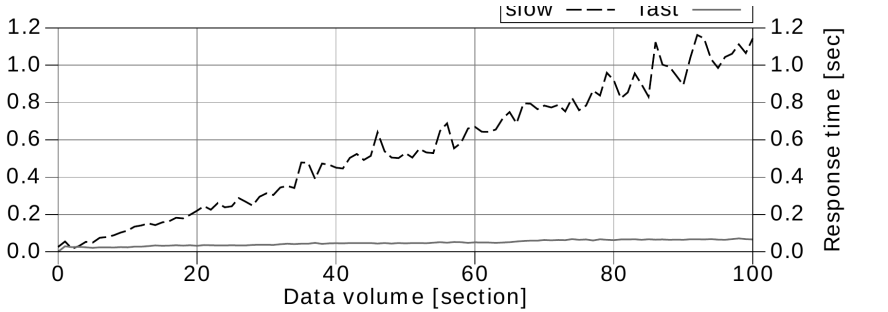
Scaling theo chiều ngang (thêm server) cũng có những giới hạn tương tự. Dù nhiều server có thể xử lý nhiều request hơn, chúng không cải thiện response time của một query cụ thể. Để tìm kiếm

nhanh hơn, bạn cần một search tree hiệu quả — kể cả trong các hệ thống non-relational như CouchDB và MongoDB.

> Important
>
> Indexing đúng cách là phương pháp tốt nhất để giảm query response time — trong cả relational SQL database lẫn non-relational system.

Indexing đúng cách nhằm khai thác trọn vẹn scalability dạng logarithmic của B-tree index. Đáng tiếc là indexing thường được thực hiện rất cẩu thả. Chart trong “Performance Impacts of Data Volume” cho thấy rõ tác động của việc indexing cẩu thả.

Figure 3.5. Response Time theo Data Volume



Khác biệt response time giữa một index cẩu thả và một index đúng cách thật đáng kinh ngạc. Gần như không thể bù đắp tác động này bằng cách thêm hardware. Ngay cả khi bạn giảm được response time bằng hardware, vẫn cần đặt câu hỏi liệu đó có phải giải pháp tốt nhất cho vấn đề này hay không.

Nhiều hệ thống được gọi là NoSQL vẫn tuyên bố có thể giải quyết mọi performance problem bằng horizontal scalability. Tuy nhiên, scalability này phần lớn bị giới hạn ở write operation và được thực hiện bằng eventual consistency model. SQL database dùng strict

consistency model làm write operation chậm hơn, nhưng điều đó không nhất thiết đồng nghĩa với throughput kém. Hãy tìm hiểu thêm trong box có tiêu đề “Eventual Consistency and the CAP Theorem”.

Thêm hardware thường không cải thiện response time. Thực tế, nó thậm chí có thể làm system chậm hơn vì complexity tăng thêm có thể tích lũy nhiều latency hơn. Network latency sẽ không thành vấn đề nếu application và database chạy trên cùng một computer, nhưng setup này khá hiếm trong production environment, nơi database và application thường được cài trên hardware riêng. Security policy thậm chí có thể yêu cầu firewall giữa application server và database — thường làm network latency tăng gấp đôi. Infrastructure càng phức tạp thì càng tích lũy nhiều latency và response càng chậm. Tác động này thường dẫn đến quan sát ngược với trực giác: production hardware đắt tiền lại chậm hơn môi trường desktop PC rẻ tiền dùng cho development.

Eventual Consistency and the CAP Theorem

Duy trì strict consistency trong một distributed system đòi hỏi synchronous coordination của mọi write operation giữa các node. Nguyên tắc này có hai side effect khó chịu: (1) thêm latency và làm response time tăng; (2) làm giảm overall availability vì nhiều member phải cùng available tại một thời điểm để hoàn thành write operation.

Distributed SQL database thường bị nhầm với computer cluster dùng shared storage system hoặc master-slave replication. Thực tế, distributed database giống một web shop tích hợp với ERP system hơn — thường là hai product khác nhau của hai vendor khác nhau. Consistency giữa hai system vẫn là một mục tiêu đáng mong muốn và thường được đạt bằng two-phase commit (2PC) protocol. Protocol này thiết lập global transaction mang lại behavior “all-or-nothing”

quen thuộc trên nhiều database. Chỉ có thể hoàn thành global transaction nếu mọi member tham gia đều available. Vì vậy, nó làm giảm overall availability.

Distributed system càng có nhiều node thì strict consistency càng trở nên khó khăn. Duy trì strict consistency gần như không thể nếu system có nhiều hơn một vài node. Ngược lại, bỏ strict consistency giải quyết vấn đề availability và loại bỏ response time tăng thêm. Ý tưởng cơ bản là tái lập global consistency sau khi hoàn thành write operation trên một subset của các node. Cách tiếp cận này chỉ còn một vấn đề chưa giải quyết: không thể ngăn conflict nếu hai node chấp nhận các thay đổi mâu thuẫn. Consistency cuối cùng đạt được bằng cách xử lý conflict, không phải ngăn conflict. Trong ngữ cảnh đó, consistency có nghĩa mọi node có cùng data — không nhất thiết là data đúng hoặc tốt nhất.

CAP Theorem của Brewer mô tả các dependency tổng quát giữa Consistency, Availability và Partition tolerance.

Một latency khác rất quan trọng là disk seek time. Spinning hard disk drive (HDD) cần khá nhiều thời gian để di chuyển các bộ phận cơ học đến vị trí có thể đọc data được yêu cầu — thường là vài millisecond. Latency này xuất hiện bốn lần khi traverse một B-tree bốn level — tổng cộng vài chục millisecond. Dù đó là cả một “thế kỷ” đối với computer, nó vẫn thấp hơn rất nhiều so với ngưỡng nhận biết của chúng ta… nếu chỉ xảy ra một lần. Tuy nhiên, một SQL statement rất dễ trigger hàng trăm hoặc thậm chí hàng nghìn disk seek, đặc biệt khi kết hợp nhiều table bằng join operation. Dù caching giảm mạnh vấn đề và công nghệ mới như SSD giảm seek time một order of magnitude, join vẫn thường bị cho là chậm. Vì vậy, chương tiếp theo sẽ giải thích cách dùng index để join table hiệu quả.

Solid State Disks (SSD) and Caching

Solid State Disk (SSD) là công nghệ mass storage không có bộ phận chuyển động. Seek time điển hình của SSD nhanh hơn seek time của HDD một order of magnitude. SSD xuất hiện trong enterprise storage khoảng năm 2010 nhưng do cost cao và lifetime có giới hạn, chúng chưa thường được dùng cho database.

Tuy nhiên, database cache data được truy cập thường xuyên trong main memory. Điều này đặc biệt hữu ích với data cần cho mọi index access — chẳng hạn index root node. Database có thể cache toàn bộ các index được dùng thường xuyên, nhờ đó một index lookup không trigger dù chỉ một disk seek.

Chapter 4

The Join Operation

Một SQL query bước vào bar và nhìn thấy hai table.

Nó bước đến gần và hỏi: “Tôi join cùng các bạn được không?”

—Source: Unknown

Join operation biến đổi data từ normalized model thành dạng denormalized phù hợp với một processing purpose cụ thể. Join đặc biệt nhạy cảm với disk seek latency vì nó kết hợp các data fragment nằm rải rác. Indexing đúng cách một lần nữa là giải pháp tốt nhất để giảm response time. Tuy nhiên, index đúng phụ thuộc vào việc query dùng thuật toán nào trong ba join algorithm phổ biến.

Tuy nhiên, có một điểm chung cho mọi join algorithm: chúng chỉ xử lý hai table mỗi lần. SQL query có nhiều table cần nhiều bước: trước tiên xây dựng một intermediate result set bằng cách join hai table, sau đó join result với table tiếp theo, cứ như vậy.

Dù join order không ảnh hưởng đến final result, nó vẫn ảnh hưởng đến performance. Vì vậy optimizer sẽ đánh giá mọi permutation có thể của join order và chọn permutation tốt nhất. Điều đó có nghĩa chỉ riêng việc optimize một complex statement cũng có thể trở thành performance problem. Càng có nhiều table cần join, càng có nhiều execution plan variant phải đánh giá — nói theo toán học là n! (factorial growth), dù đây không phải vấn đề khi dùng bind parameter.

> Important
>
> Statement càng complex thì việc dùng bind parameter càng quan trọng.
>
> Không dùng bind parameter cũng giống như recompile một program mỗi lần.

Pipelining Intermediate Results

Dù intermediate result giải thích algorithm rất rõ, điều đó không có nghĩa database phải materialize nó. Nếu vậy, database phải lưu intermediate result của join đầu tiên trước khi bắt đầu join tiếp theo. Thay vào đó, database dùng pipelining để giảm memory usage. Nghĩa là mỗi row của intermediate result được pipeline ngay lập tức sang join operation tiếp theo — tránh phải lưu intermediate result set.

Nested Loops

Nested loops join là join algorithm nền tảng nhất. Nó hoạt động giống như dùng hai nested query: outer hoặc driving query fetch result từ một table, rồi một query thứ hai cho mỗi row của driving query fetch data tương ứng từ table còn lại.

Bạn thực sự có thể tự dùng “nested select” để implement nested loops algorithm. Tuy nhiên, đó là cách tiếp cận rắc rối vì network latency xuất hiện chồng lên disk latency — khiến overall response time còn tệ hơn. “Nested select” vẫn rất phổ biến vì dễ implement mà không nhận ra mình đang làm vậy. Object-relational mapping (ORM) tool đặc biệt “helpful” ở điểm này… đến mức cái gọi là N+1 selects problem đã trở nên tai tiếng trong lĩnh vực này.

Các ví dụ sau cho thấy những join “accidental nested select” được tạo ra bởi các ORM tool khác nhau. Các ví dụ search employee có last name bắt đầu bằng 'WIN' và fetch toàn bộ SALES của những employee đó.

ORM không generate SQL join — thay vào đó chúng query table SALES bằng nested select. Hiệu ứng này được gọi là “N+1 selects problem”, hoặc ngắn hơn là “N+1 problem”, vì tổng cộng nó execute N+1 select nếu driving query trả về N row.

Java

Ví dụ JPA dùng interface CriteriaBuilder.

```
CriteriaBuilder queryBuilder = em.getCriteriaBuilder();
CriteriaQuery<Employees>
   query = queryBuilder.createQuery(Employees.class);
Root<Employees> r = query.from(Employees.class);
query.where(
  queryBuilder.like(
    queryBuilder.upper(r.get(Employees_.lastName)),
    "WIN%"
  )
);

List<Employees> emp = em.createQuery(query).getResultList();

for (Employees e: emp) {
  // process Employee
  for (Sales s: e.getSales()) {
    // process sale for Employee
  }
}
```

Hibernate JPA 3.6.0 generate N+1 select query:

```
select employees0_.subsidiary_id as subsidiary1_0_
       -- MORE COLUMNS
  from employees employees0_
 where upper(employees0_.last_name) like ?

  select sales0_.subsidiary_id as subsidiary4_0_1_
         -- MORE COLUMNS
    from sales sales0_
   where sales0_.subsidiary_id=?
     and sales0_.employee_id=?

  select sales0_.subsidiary_id as subsidiary4_0_1_
         -- MORE COLUMNS
    from sales sales0_
   where sales0_.subsidiary_id=?
     and sales0_.employee_id=?
```

Chapter 4: The Join Operation

Perl

Ví dụ sau minh họa framework DBIx::Class của Perl:

```
my @employees =
   $schema->resultset('Employees')
          ->search({'UPPER(last_name)' => {-like=>'WIN%'}});

foreach my $employee (@employees) {
   # process Employee
   foreach my $sale ($employee->sales) {
      # process Sale for Employee
   }
}
```

DBIx::Class 0.08192 generate các query select N+1:

```
SELECT me.employee_id, me.subsidiary_id
     , me.last_name, me.first_name, me.date_of_birth
  FROM employees me
 WHERE ( UPPER(last_name) LIKE ? )


   SELECT me.sale_id, me.employee_id, me.subsidiary_id
        , me.sale_date, me.eur_value
     FROM sales me
    WHERE ( ( me.employee_id = ?
      AND me.subsidiary_id = ? ) )
```

PHP

Ví dụ Doctrine dùng interface query builder:

```
$qb = $em->createQueryBuilder();
$qb->select('e')
   ->from('Employees', 'e')
   ->where("upper(e.last_name) like :last_name")
   ->setParameter('last_name', 'WIN%');
$r = $qb->getQuery()->getResult();

foreach ($r as $row) {
   // process Employee
   foreach ($row->getSales() as $sale) {
      // process Sale for Employee
   }
}
```

Nested Loops

Doctrine 2.0.5 generate các query select N+1:

```
SELECT e0_.employee_id AS employee_id0 -- MORE COLUMNS
  FROM employees e0_
 WHERE UPPER(e0_.last_name) LIKE ?

   SELECT t0.sale_id AS SALE_ID1 -- MORE COLUMNS
     FROM sales t0
    WHERE t0.subsidiary_id = ?
      AND t0.employee_id = ?

   SELECT t0.sale_id AS SALE_ID1 -- MORE COLUMNS
     FROM sales t0
    WHERE t0.subsidiary_id = ?
      AND t0.employee_id = ?
```

Enabling SQL Logging

Bật SQL logging trong quá trình development và review các SQL statement được generate.

DBIx::Class

X::Class
`export DBIC_TRACE=1` in your shell.

Doctrine

Chỉ thực hiện ở source code level — đừng quên disable setting này trong production. Hãy cân nhắc việc tự xây dựng một configurable logger.

```
$logger = new \Doctrine\DBAL\Logging\EchoSqlLogger;
$config->setSQLLogger($logger);
```

Hibernate (native)

```
<property name="show_sql">true</property> in App.config
hibernate.cfg.xml
```

JPA

Trong persistence.xml, nhưng còn tùy JPA provider:

```
<property name="eclipselink.logging.level" value="FIN
<property name="hibernate.show_sql" value="TRUE"/>
<property name="openjpa.Log" value="SQL=TRACE"/>
```

Hầu hết ORM cũng cung cấp cách programmatic để enable SQL logging. Cách này có rủi ro vô tình deploy setting đó lên production.

Chapter 4: The Join Operation

Dù cách tiếp cận "nested selects" là một anti-pattern, nó vẫn giải thích khá rõ nested loops join. Database thực thi join chính xác như các ORM tool ở trên. Vì vậy, indexing cho nested loops join cũng giống như indexing cho các select statement đã trình bày ở trên. Đó là function-based index trên table EMPLOYEES và concatenated index cho các join predicate trên table SALES:

```
CREATE INDEX emp_up_name ON employees (UPPER(last_name));
CREATE INDEX sales_emp ON sales (subsidiary_id, employee_id);
```

SQL join vẫn hiệu quả hơn cách tiếp cận nested selects — dù thực hiện cùng các index lookup — vì nó tránh được nhiều network communication. Nó thậm chí còn nhanh hơn khi tổng lượng data truyền đi lớn hơn do employee attribute bị lặp lại cho mỗi sale. Đó là vì performance có hai dimension: response time và throughput; trong computer network, chúng ta gọi chúng là latency và bandwidth. Bandwidth chỉ tác động nhỏ đến response time, còn latency có tác động rất lớn. Vì vậy, số database round trip quan trọng đối với response time hơn lượng data được truyền.

>  Tip
>
> Execute join trong database.

Hầu hết ORM tool đều có cách tạo SQL join. Chế độ gọi là eager fetching có lẽ là cách quan trọng nhất. Nó thường được cấu hình ở property level trong entity mapping — ví dụ property employees trong class Sales. Khi đó ORM tool sẽ luôn join table EMPLOYEES khi access table SALES. Cấu hình eager fetching trong entity mapping chỉ hợp lý nếu lúc nào bạn cũng cần employee detail cùng với sales data.

Eager fetching phản tác dụng nếu bạn không cần child record mỗi lần access parent object. Với application dạng telephone directory, load record SALES khi hiển thị employee detail là không hợp lý. Bạn có thể cần sales data liên quan trong những trường hợp khác — nhưng không phải lúc nào cũng cần. Static configuration không giải quyết được vấn đề.

Để đạt performance tối ưu, bạn cần toàn quyền kiểm soát join. Các ví dụ sau cho thấy cách có được mức flexibility cao nhất bằng cách kiểm soát join behavior tại runtime.

Nested Loops

Java

Interface JPA CriteriaBuilder cung cấp method Root<>.fetch() để kiểm soát join. Method này cho phép bạn chỉ định thời điểm và cách join các object được tham chiếu vào main query. Trong ví dụ này, chúng ta dùng left join để lấy tất cả employee, kể cả những employee không có sale.

> Warning
>
> JPA và Hibernate trả về employee cho mỗi sale.
>
> Điều đó có nghĩa một employee có 30 sale sẽ xuất hiện 30 lần. Dù rất khó chịu, đây là behavior được quy định (EJB 3.0 persistency, paragraph 4.4.5.3 “Fetch Joins”). Bạn có thể tự de-duplicate parent relation hoặc dùng function distinct() như trong ví dụ.

```
    queryBuilder.upper(r.get(Employees_.lastName)),
    "WIN%")
);

r.fetch("sales", JoinType.LEFT);
// needed to avoid duplication of Employee records
query.distinct(true);

List<Employees> emp = em.createQuery(query).getResultList();
```

Hibernate 3.6.0 generates the following SQL statement:

```
select distinct
       employees0_.subsidiary_id as subsidiary1_0_0_
       employees0_.employee_id as employee2_0_0
```

Hibernate 3.6.0 generate SQL statement sau:

```
          on employees0_.subsidiary_id=sales1_.subsidiary_id
         and employees0_.employee_id=sales1_.employee_id
 where upper(employees0_.last_name) like ?
```

The query has the expected left join but also an unnecessary `distinct` keyword. Unfortunately, JPA does not provide separate API calls to filter duplicated parent entries without de-duplicating the child records as

Query có left join như kỳ vọng nhưng cũng có keyword distinct không cần thiết. Đáng tiếc là JPA không cung cấp API call riêng để filter các parent entry bị duplicate mà không de-duplicate cả child record

Chapter 4: The Join Operation

tương tự.

Keyword distinct trong SQL query đáng lo vì hầu hết database sẽ thực sự filter các duplicate record. Chỉ một số ít database nhận biết rằng primary key vốn đã bảo đảm tính unique trong trường hợp này.

ıe native Hibernate API solves the problem on the client side using a sult set transformer:

```
Criteria c = session.createCriteria(Employees.class);
c.add(Restrictions.ilike("lastName", 'Win%'));

c.setFetchMode("sales", FetchMode.JOIN);
c.setResultTransformer(Criteria.DISTINCT_ROOT_ENTITY);

List<Employees> result = c.list();
```

Nó generate query sau:

```
select this_.subsidiary_id as subsidiary1_0_1_
     , this_.employee_id as employee2_0_1_
       -- MORE this_ columns on employees
     , sales2_.sale_id as sale1_3_
       -- MORE sales2_ columns on sales
  from employees this_
  left outer join sales sales2_
          on this_.subsidiary_id=sales2_.subsidiary_id
         and this_.employee_id=sales2_.employee_id
 where lower(this_.last_name) like ?
```

Method này tạo ra SQL trực tiếp mà không có clause ngoài ý muốn. Lưu ý rằng Hibernate dùng lower() cho case-insensitive query — một chi tiết quan trọng đối với function-based indexing.

Nested Loops

Perl

Ví dụ sau dùng framework DBIx::Class của Perl:

```
my @employees =
   $schema->resultset('Employees')
          ->search({ 'UPPER(last_name)' => {-like => 'WIN%'}
                   , {prefetch => ['sales']}
                   });
```

DBIx::Class 0.08192 generate SQL statement sau:

```
SELECT me.employee_id, me.subsidiary_id, me.last_name
       -- MORE COLUMNS
  FROM employees me
  LEFT JOIN sales sales
         ON (sales.employee_id   = me.employee_id
        AND  sales.subsidiary_id = me.subsidiary_id)
 WHERE ( UPPER(last_name) LIKE ? )
 ORDER BY sales.employee_id, sales.subsidiary_id
```

Note the `order by` clause—it was not requested by the application. The database has to sort the result set accordingly, and that might take a while.

PHP

Ví dụ sau dùng framework Doctrine của PHP:

```
$qb = $em->createQueryBuilder();
$qb->select('e,s')
   ->from('Employees', 'e')
   ->leftJoin('e.sales', 's')
   ->where("upper(e.last_name) like :last_name")
   ->setParameter('last_name', 'WIN%');
$r = $qb->getQuery()->getResult();
```

Doctrine 2.0.5 generates the following SQL statement:

```
SELECT e0_.employee_id AS employee_id0
       -- MORE COLUMNS
```

Doctrine 2.0.5 generate SQL statement sau:

```
          ON e0_.subsidiary_id = s1_.subsidiary_id
         AND e0_.employee_id = s1_.employee_id
 WHERE UPPER(e0_.last_name) LIKE ?
```


Chapter 4: The Join Operation

```
-----------------------------------------------------------------
|Id |Operation                    | Name        | Rows | Cost |
-----------------------------------------------------------------
| 0 |SELECT STATEMENT             |             |  822 |   38 |
| 1 | NESTED LOOPS OUTER          |             |  822 |   38 |
| 2 |  TABLE ACCESS BY INDEX ROWID| EMPLOYEES   |    1 |    4 |
|*3 |   INDEX RANGE SCAN          | EMP_UP_NAME |    1 |      |
| 4 |  TABLE ACCESS BY INDEX ROWID| SALES       |  821 |   34 |
|*5 |   INDEX RANGE SCAN          | SALES_EMP   |   31 |      |
-----------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
  3 - access(UPPER("LAST_NAME") LIKE 'WIN%')
      filter(UPPER("LAST_NAME") LIKE 'WIN%')
  5 - access("E0_"."SUBSIDIARY_ID"="S1_"."SUBSIDIARY_ID"(+)
        AND  "E0_"."EMPLOYEE_ID"  ="S1_"."EMPLOYEE_ID"(+))
```

Database trước tiên lấy result từ table EMPLOYEES thông qua EMP_UP_NAME, sau đó fetch các record tương ứng từ table SALES cho từng employee.

> 💡 Tip
>
> Hiểu ORM của bạn và kiểm soát join.

Nested loops join cho performance tốt nếu driving query trả về một result set nhỏ. Nếu không, optimizer có thể chọn một join algorithm hoàn toàn khác — chẳng hạn hash join được mô tả trong section tiếp theo, nhưng chỉ khi application dùng join để nói cho database biết chính xác cần data nào.

Hash Join

Hash Join

Hash join algorithm nhắm vào điểm yếu của nested loops join: nhiều lần B-tree traversal khi execute inner query. Thay vào đó, nó load các candidate record từ một phía của join vào hash table, để mỗi row từ phía còn lại có thể probe rất nhanh. Tuning hash join đòi hỏi indexing approach hoàn toàn khác nested loops join. Ngoài ra, có thể cải thiện performance của hash join bằng cách select ít column hơn — một thách thức đối với hầu hết ORM tool.

Indexing strategy cho hash join rất khác vì không cần index các join column. Chỉ index cho các independent where predicate mới cải thiện performance của hash join.

> 💡 Tip
>
> Index các independent where predicate để cải thiện performance của hash join.

Xét ví dụ sau. Nó select toàn bộ sale trong sáu tháng vừa qua cùng employee detail tương ứng:

> **TIP**
> Index the *independent* `where` predicates to improve hash join performance.

sider the following example. It selects all sales for the past six months

Filter SALE_DATE là where clause độc lập duy nhất — nghĩa là nó chỉ tham chiếu đến một table và không thuộc join predicate.

Chapter 4: The Join Operation

```
-----------------------------------------------------------------
| Id | Operation          | Name      | Rows  | Bytes | Cost |
-----------------------------------------------------------------
|  0 | SELECT STATEMENT   |           | 49244 |    59M| 12049|
|* 1 |  HASH JOIN         |           | 49244 |    59M| 12049|
|  2 |   TABLE ACCESS FULL| EMPLOYEES | 10000 |     9M|   478|
|* 3 |   TABLE ACCESS FULL| SALES     | 49244 |    10M| 10521|
-----------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - access("S"."SUBSIDIARY_ID"="E"."SUBSIDIARY_ID"
         AND "S"."EMPLOYEE_ID"  ="E"."EMPLOYEE_ID")
   3 - filter("S"."SALE_DATE">TRUNC(SYSDATE@!)
                             -INTERVAL'+00-06' YEAR(2) TO MONTH)
```

The first execution step is a full table scan to load all employees into a

Bước thực thi đầu tiên là full table scan để load toàn bộ employee vào hash table (plan id 2). Hash table dùng join predicate làm key. Ở bước tiếp theo, database lại full table scan table SALES và loại bỏ các sale không thỏa điều kiện SALE_DATE (plan id 3). Với những SALES record còn lại, database access hash table để load employee detail tương ứng.

Mục đích duy nhất của hash table là làm temporary in-memory structure để tránh access table EMPLOYEE nhiều lần. Hash table được load một lần nên không cần index để fetch từng record hiệu

quả. Predicate information xác nhận không có filter nào được áp dụng trên table EMPLOYEES (plan id 2). Query không có independent predicate nào trên table này.

> Important
>
> Indexing join predicate không cải thiện performance của hash join.

Điều đó không có nghĩa là không thể index hash join. Independent predicate có thể được index. Đây là các condition được áp dụng trong một trong hai table access operation. Trong ví dụ trên, đó là filter trên SALE_DATE.

```
CREATE INDEX sales_date ON sales (sale_date);
```

Execution plan sau dùng index này. Tuy nhiên, nó vẫn dùng full table scan cho table EMPLOYEES vì query không có independent where predicate trên EMPLOYEES.

Hash Join

```
-------------------------------------------------------------------
| Id | Operation                     | Name      | Bytes| Cost|
-------------------------------------------------------------------
|  0 | SELECT STATEMENT              |           |   59M| 3252|
|* 1 |  HASH JOIN                    |           |   59M| 3252|
|  2 |   TABLE ACCESS FULL           | EMPLOYEES |    9M|  478|
|  3 |   TABLE ACCESS BY INDEX ROWID| SALES     |   10M| 1724|
|* 4 |    INDEX RANGE SCAN           | SALES_DATE|      |     |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - access("S"."SUBSIDIARY_ID"="E"."SUBSIDIARY_ID"
         AND "S"."EMPLOYEE_ID"  ="E"."EMPLOYEE_ID"  )
   4 - access("S"."SALE_DATE" > TRUNC(SYSDATE@!)
                           -INTERVAL'+00-06' YEAR(2) TO MONTH)
```

Indexing hash join là symmetric — trái ngược với nested loops join. Nghĩa là join order không ảnh hưởng đến indexing. Index SALES_DATE vẫn có thể được dùng để load hash table nếu đảo ngược join order.

> Note
>
> Indexing hash join độc lập với join order.

Một approach khá khác để tối ưu hash join performance là minimize kích thước hash table. Cách này hiệu quả vì hash join tối ưu chỉ có thể xảy ra khi toàn bộ hash table vừa trong memory. Vì vậy optimizer sẽ tự động dùng phía nhỏ hơn của join làm hash table. Oracle execution plan hiển thị memory requirement được estimate trong column “Bytes”. Trong execution plan ở trên, table EMPLOYEES cần chín megabyte nên là phía nhỏ hơn.

Cũng có thể giảm kích thước hash table bằng cách thay đổi SQL query, chẳng hạn thêm condition để database load ít candidate record hơn vào hash table. Tiếp tục ví dụ trên, điều đó có nghĩa thêm filter trên attribute DEPARTMENT để chỉ xét sales staff. Cách này cải thiện hash join performance ngay cả khi không có index trên attribute DEPARTMENT, vì database không cần lưu những employee không thể có sale vào hash table. Khi làm vậy, bạn phải chắc chắn không có SALES record nào thuộc employee không làm việc trong department tương ứng. Hãy dùng constraint để bảo vệ các assumption.

Chapter 4: The Join Operation

Khi minimize kích thước hash table, factor quan trọng không phải số row mà là memory footprint. Thực tế, cũng có thể giảm kích thước hash table bằng cách select ít column hơn — chỉ những attribute thực sự cần:

```
SELECT s.sale_date, s.eur_value
     , e.last_name, e.first_name
  FROM sales s
  JOIN employees e ON (s.subsidiary_id = e.subsidiary_id
                  AND  s.employee_id   = e.employee_id  )
 WHERE s.sale_date > trunc(sysdate) - INTERVAL '6' MONTH
```

Cách này hiếm khi tạo bug vì bỏ nhầm column có lẽ sẽ nhanh chóng dẫn đến error message. Tuy nhiên, vẫn có thể cắt giảm đáng kể kích thước hash table; trong trường hợp này từ 9 megabyte xuống 234 kilobytes — giảm 97%.

```
-------------------------------------------------------------------
| Id | Operation                     | Name      | Bytes| Cost|
-------------------------------------------------------------------
|  0 | SELECT STATEMENT              |           | 2067K| 2202|
|* 1 |  HASH JOIN                    |           | 2067K| 2202|
|  2 |   TABLE ACCESS FULL           | EMPLOYEES |  234K|  478|
|  3 |   TABLE ACCESS BY INDEX ROWID| SALES     |  913K| 1724|
|* 4 |    INDEX RANGE SCAN           | SALES_DATE|      |  133|
-------------------------------------------------------------------
```

> **Tip**
>
> Select ít column hơn để cải thiện hash join performance.

Thoạt nhìn, việc bỏ vài column khỏi SQL statement có vẻ đơn giản, nhưng đây là thách thức thực sự khi dùng object-relational mapping (ORM) tool. Support cho cái gọi là partial object rất hạn chế. Các ví dụ sau cho thấy một số khả năng.

Java

JPA định nghĩa FetchType.LAZY trong annotation @Basic. Nó có thể được áp dụng ở property level:

```
@Column(name="junk")
@Basic(fetch=FetchType.LAZY)
private String junk;
```

Hash Join

JPA provider được tự do ignore nó:

LAZY strategy là một hint cho persistence provider runtime rằng data nên được fetch lazily khi lần đầu được access. Implementation được phép eagerly fetch data mà LAZY strategy hint đã được chỉ định.

—EJB 3.0 JPA, paragraph 9.1.18

Hibernate 3.6 implement lazy property fetching thông qua compile-time bytecode instrumentation. Instrumentation thêm code vào các compiled class và không fetch LAZY property cho đến khi property được access. Approach này hoàn toàn transparent với application, nhưng mở ra một dimension mới của N+1 problem: một select cho mỗi record và property. Điều này đặc biệt nguy hiểm vì JPA không cung cấp runtime control để eager fetch khi cần.

Native query language HQL của Hibernate giải quyết vấn đề bằng clause FETCH ALL PROPERTIES:

```
elect s from Sales s FETCH ALL PROPERTIES
inner join fetch s.employee e FETCH ALL PROPERTIES
where s.saleDate >:dt
```

Clause FETCH ALL PROPERTIES buộc Hibernate eager fetch entity — ngay cả khi dùng instrumented code và LAZY annotation.

Một option khác để chỉ load các column được chọn là dùng data transport object (DTO) thay cho entity. Method này hoạt động giống nhau trong HQL và JPQL, tức là initialize một object ngay trong query:

```
select new SalesHeadDTO(s.saleDate , s.eurValue
                        ,e.firstName, e.lastName)
  from Sales s
  join s.employee e
 where s.saleDate > :dt
```

Query chỉ select data được yêu cầu và trả về object SalesHeadDTO — một Java object đơn giản (POJO), không phải entity.

Chapter 4: The Join Operation

Perl

Framework DBIx::Class không đóng vai trò entity manager nên inheritance không gây ra vấn đề aliasing.[1] Cookbook hỗ trợ approach này. Schema definition sau định nghĩa class Sales ở hai level:

```
package UseTheIndexLuke::Schema::Result::SalesHead;
use base qw/DBIx::Class::Core/;

__PACKAGE__->table('sales');
__PACKAGE__->add_columns(qw/sale_id employee_id subsidiary_id
                            sale_date eur_value/);
__PACKAGE__->set_primary_key(qw/sale_id/);
__PACKAGE__->belongs_to('employee', 'Employees',
          {'foreign.employee_id'   => 'self.employee_id'
          ,'foreign.subsidiary_id' => 'self.subsidiary_id'});

package UseTheIndexLuke::Schema::Result::Sales;
use base qw/UseTheIndexLuke::Schema::Result::SalesHead/;

__PACKAGE__->table('sales');
__PACKAGE__->add_columns(qw/junk/);
```

Class Sales được derived từ class SalesHead và bổ sung attribute còn thiếu. Bạn có thể dùng cả hai class tùy nhu cầu. Lưu ý rằng table setup cũng bắt buộc phải có trong derived class.

Bạn có thể fetch toàn bộ employee detail bằng prefetch hoặc chỉ các column được chọn như dưới đây:

```
my @sales =
   $schema->resultset('SalesHead')
          ->search($cond
                  ,{      join => 'employee'
                   ,'+columns' => ['employee.first_name'
                                   ,'employee.last_name']
                   }
                  );
```

Không thể chỉ load các column được chọn từ root table — trong trường hợp này là SalesHead.

1

http://en.wikipedia.org/wiki/Aliasing_%28computing%29

Hash Join

DBIx::Class 0.08192 generate SQL sau. Nó fetch toàn bộ column từ table SALES và các attribute được chọn từ EMPLOYEES:

```
SELECT me.sale_id,
       me.employee_id,
       me.subsidiary_id,
       me.sale_date,
       me.eur_value,
       employee.first_name,
       employee.last_name
  FROM sales me
  JOIN employees employee
        ON( employee.employee_id   = me.employee_id
       AND  employee.subsidiary_id = me.subsidiary_id)
 WHERE(sale_date > ?)
```

PHP

Version 2 của framework Doctrine hỗ trợ attribute selection tại runtime. Documentation nói rằng partially loaded object có thể behave oddly và yêu cầu dùng keyword partial để acknowledge risk. Ngoài ra, bạn phải select rõ các primary key column:

```
$qb = $em->createQueryBuilder();
$qb->select('partial s.{sale_id, sale_date, eur_value},'
          . 'partial e.{employee_id, subsidiary_id, '
                      . 'first_name , last_name}')
   ->from('Sales', 's')
   ->join('s.employee', 'e')
   ->where("s.sale_date > :dt")
   ->setParameter('dt', $dt, Type::DATETIME);
```

SQL được generate chứa các column được yêu cầu và một lần nữa có SUBSIDIARY_ID cùng EMPLOYEE_ID từ table SALES.

```
SELECT s0_.sale_id       AS sale_id0,
       s0_.sale_date     AS sale_date1,
       s0_.eur_value     AS eur_value2,
       e1_.employee_id   AS employee_id3,
       e1_.subsidiary_id AS subsidiary_id4,
       e1_.first_name    AS first_name5,
       e1_.last_name     AS last_name6,
       s0_.subsidiary_id AS subsidiary_id7,
       s0_.employee_id   AS employee_id8
  FROM sales s0_
 INNER JOIN employees e1_
         ON s0_.subsidiary_id = e1_.subsidiary_id
        AND s0_.employee_id   = e1_.employee_id
 WHERE s0_.sale_date > ?
```

Các object được trả về tương thích với object đã load đầy đủ, nhưng các column thiếu vẫn ở trạng thái chưa initialize. Access chúng không trigger exception.

> Warning
>
> MySQL hoàn toàn không support hash join (feature request #59025).

Chapter 4: The Join Operation

Sort Merge

Sort Merge

Sort-merge join kết hợp hai sorted list giống như một zipper. Cả hai phía của join đều phải được sort theo join predicate.

Sort-merge join cần cùng loại index như hash join, tức là index cho các independent condition để đọc toàn bộ candidate record trong một lần.

Indexing join predicate là vô ích. Cho đến đây, mọi thứ giống hash join. Tuy nhiên, sort-merge join có một đặc tính riêng: symmetry tuyệt đối. Join order không tạo khác biệt — kể cả về performance. Tính chất này đặc biệt hữu ích cho outer join. Với các algorithm khác, hướng của outer join (left hay right) quyết định join order — nhưng không phải với sort-merge join. Sort-merge join thậm chí có thể thực hiện left và right outer join cùng lúc — cái gọi là full outer join.

Dù sort-merge join hoạt động rất tốt khi input đã được sort, nó hiếm khi được dùng vì sort cả hai phía rất tốn kém. Ngược lại, hash join chỉ cần preprocess một phía.

Điểm mạnh của sort-merge join xuất hiện khi input đã được sort sẵn. Có thể đạt điều này bằng cách khai thác thứ tự của index để tránh hoàn toàn các sort operation. Chapter 6, “Sorting and Grouping”, giải thích chi tiết concept này.

Dù vậy, hash join algorithm vẫn vượt trội trong nhiều trường hợp.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Chapter 5

Clustering Data

The Second Power of Indexing

Thuật ngữ cluster được dùng trong nhiều lĩnh vực. Chẳng hạn, star cluster là một nhóm ngôi sao. Computer cluster lại là một nhóm computer làm việc chặt chẽ với nhau — để giải quyết một bài toán phức tạp (high-performance computing cluster) hoặc tăng availability (failover cluster). Nói chung, cluster là những thứ có liên quan và xuất hiện cùng nhau.

Trong computing còn có một loại cluster khác — loại thường bị hiểu sai: data cluster. Clustering data nghĩa là lưu những data được access liên tiếp ở gần nhau để việc access cần ít IO operation hơn. Data cluster rất quan trọng đối với database tuning. Computer cluster cũng rất phổ biến trong ngữ cảnh database — khiến thuật ngữ cluster trở nên mơ hồ. Câu “Let’s use a cluster to improve database performance” là một ví dụ; nó có thể nói đến computer cluster nhưng cũng có thể nghĩa là data cluster. Trong chapter này, cluster nói chung sẽ chỉ data cluster.

Data cluster đơn giản nhất trong SQL database là row. Database lưu tất cả column của một row trong cùng database block nếu có thể. Có ngoại lệ nếu row không vừa trong một block — ví dụ khi có LOB type.

Column Stores

Các database dạng column-oriented, hay column-store, tổ chức table theo column. Model này có lợi khi access nhiều row nhưng chỉ một vài column — một pattern rất phổ biến trong data warehouse (OLAP).

Chapter 5: Clustering Data

Index cho phép cluster data. Cơ sở của điều này đã được giải thích trong Chapter 1, “Anatomy of an Index”: index leaf node lưu indexed column theo thứ tự, nên các value tương tự được lưu cạnh nhau. Điều đó có nghĩa index xây dựng các cluster gồm những row có value tương tự.

Khả năng cluster data quan trọng đến mức tôi gọi nó là power thứ hai của indexing.

Các section sau giải thích cách dùng index để cluster data và cải thiện query performance.

Index Filter Predicates Used Intentionally

Rất thường xuyên, index filter predicate cho thấy index được dùng không đúng do column order sai trong concatenated index. Tuy nhiên, index filter predicate cũng có thể được dùng có chủ đích — không phải để cải thiện range scan performance mà để nhóm những data được access liên tiếp lại với nhau.

Các predicate trong where clause không thể làm access predicate là candidate tốt cho kỹ thuật này:

range" no matter if you index LAST_NAME or UPPER(last_name). This
ion is therefore no good candidate for indexing.

ver the condition on SUBSIDIARY_ID is well suited for indexing. We
even need to add a new index because the SUBSIDIARY_ID is already
ading column in the index for the primary key.

Hãy nhớ rằng LIKE expression có wildcard ở đầu không thể dùng index tree. Nghĩa là indexing LAST_NAME không thu hẹp index range được scan — bất kể bạn index LAST_NAME hay UPPER(last_name). Vì vậy condition này không phải candidate tốt để indexing.

Tuy nhiên, condition trên SUBSIDIARY_ID rất phù hợp để indexing. Chúng ta thậm chí không cần thêm index mới vì SUBSIDIARY_ID đã là leading column trong index của primary key.

Index Filter Predicates Used Intentionally

```
-------------------------------------------------------------------
|Id | Operation                    | Name       | Rows | Cost |
-------------------------------------------------------------------
| 0 | SELECT STATEMENT             |            |   17 |  230 |
|*1 |  TABLE ACCESS BY INDEX ROWID| EMPLOYEES  |   17 |  230 |
|*2 |   INDEX RANGE SCAN           | EMPLOYEE_PK|  333 |    2 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - filter(UPPER("LAST_NAME") LIKE '%INA%')
   2 - access("SUBSIDIARY_ID"=TO_NUMBER(:A))
```

Trong execution plan ở trên, cost value tăng một trăm lần từ INDEX RANGE SCAN đến TABLE ACCESS BY INDEX ROWID tiếp theo. Nói cách khác, table access thực hiện phần lớn công việc. Đây thực sự là pattern phổ biến và bản thân nó không phải vấn đề. Tuy nhiên, nó là contributor đáng kể nhất vào overall execution time của query này.

Table access không nhất thiết là bottleneck nếu các row được access nằm trong một table block, vì database có thể fetch toàn bộ row bằng một read operation. Ngược lại, nếu các row đó nằm rải rác

trong nhiều block, table access có thể trở thành performance problem nghiêm trọng vì database phải fetch nhiều block để lấy toàn bộ row. Nghĩa là performance phụ thuộc vào physical distribution của các row — nói cách khác, phụ thuộc vào clustering của row.

> Note
>
> Mối tương quan giữa index order và table order là một performance benchmark — cái gọi là index clustering factor.

Thực tế có thể cải thiện query performance bằng cách sắp xếp lại row trong table để chúng tương ứng với index order. Tuy nhiên, method này hiếm khi áp dụng được vì bạn chỉ có thể lưu table row theo một sequence. Nghĩa là bạn chỉ có thể optimize table cho một index. Ngay cả khi chọn được một index duy nhất để optimize table, đây vẫn là task khó vì hầu hết database chỉ cung cấp tool rất sơ sài cho việc này. Cái gọi là row sequencing rốt cuộc là một approach khá không thực tế.

Chapter 5: Clustering Data

The Index Clustering Factor

Index clustering factor là indirect measure của xác suất hai index entry liên tiếp tham chiếu đến cùng một table block. Optimizer tính đến xác suất này khi tính cost value của operation TABLE ACCESS BY INDEX ROWID.

Đây chính là nơi power thứ hai của indexing — clustering data — phát huy tác dụng. Bạn có thể thêm nhiều column vào index để chúng tự động được lưu theo một order được xác định rõ. Điều đó biến index thành một tool mạnh nhưng đơn giản để cluster data.

Để áp dụng concept này vào query ở trên, chúng ta phải mở rộng index để cover toàn bộ column trong where clause — kể cả những column không thu hẹp index range được scan:

```
CREATE INDEX empsubupnam ON employees
       (subsidiary_id, UPPER(last_name));
```

The column SUBSIDIARY_ID is the first index column so it can be used as an access predicate. The expression UPPER(last_name) covers the LIKE filter

Column SUBSIDIARY_ID là index column đầu tiên nên có thể được dùng làm access predicate. Expression UPPER(last_name) cover LIKE filter dưới dạng index filter predicate. Index biểu diễn uppercase giúp tiết kiệm một vài CPU cycle trong quá trình execute, nhưng index thông thường trên LAST_NAME cũng hoạt động. Bạn sẽ tìm hiểu thêm về điều này trong section tiếp theo.

```
-------------------------------------------------------------------
|Id | Operation                    | Name       | Rows | Cost |
-------------------------------------------------------------------
| 0 | SELECT STATEMENT             |            |   17 |   20 |
| 1 |  TABLE ACCESS BY INDEX ROWID| EMPLOYEES  |   17 |   20 |
|*2 |   INDEX RANGE SCAN           | EMPSUBUPNAM|   17 |    3 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   2 - access("SUBSIDIARY_ID"=TO_NUMBER(:A))
       filter(UPPER("LAST_NAME") LIKE '%INA%')
```

Execution plan mới hiển thị chính các operation như trước. Dù vậy, cost value đã giảm đáng kể. Trong predicate information, chúng ta thấy LIKE filter đã được áp dụng ngay trong INDEX RANGE SCAN. Các row không thỏa LIKE filter bị loại ngay lập tức. Table access không còn filter predicate nào nữa. Nghĩa là nó không load những row không thỏa where clause.

Dùng index filter predicate có chủ đích

Sự khác biệt giữa hai execution plan nhìn thấy rất rõ trong column “Rows”. Theo estimate của optimizer, query cuối cùng match 17 record. Tuy vậy, index scan trong execution plan thứ nhất trả về 333 row. Database sau đó phải load 333 row này từ table để áp dụng LIKE filter, làm kết quả giảm xuống còn 17 row. Trong execution plan thứ hai, index access ngay từ đầu đã không trả về những row đó, nên database chỉ phải thực hiện operation TABLE ACCESS BY INDEX ROWID 17 lần.

Bạn cũng nên lưu ý cost value của operation INDEX RANGE SCAN tăng từ 2 lên 3 vì column bổ sung làm index lớn hơn. Xét performance gain đạt được, đây là một compromise chấp nhận được.

> Warning
>
> Đừng tạo index mới chỉ vì mục đích duy nhất là dùng cho filter predicate. Thay vào đó, hãy mở rộng một index hiện có và giữ maintenance effort ở mức thấp. Với một số database, bạn thậm chí có thể thêm vào index của primary key những column không thuộc primary key.

Ví dụ đơn giản này dường như xác nhận common wisdom rằng nên index mọi column trong where clause. Tuy nhiên, “wisdom” này bỏ qua tầm quan trọng của column order, yếu tố quyết định condition nào có thể được dùng làm access predicate và vì vậy có ảnh hưởng rất lớn đến performance. Do đó, quyết định về column order không bao giờ nên để cho chance quyết định.

Index size cũng tăng theo số lượng column — đặc biệt khi thêm text column. Đương nhiên, index lớn hơn không làm performance tốt hơn, dù logarithmic scalability làm giảm đáng kể mức ảnh hưởng.

Tuyệt đối không nên thêm tất cả column được nhắc đến trong where clause vào index; thay vào đó, chỉ dùng index filter predicate có chủ đích để giảm data volume ở một execution step sớm hơn.

Chapter 5: Clustering Data

Index-Only Scan

Index-only scan là một trong những tuning method mạnh nhất. Nó không chỉ tránh access table để evaluate where clause, mà còn tránh hoàn toàn table access nếu database có thể tìm các column được select ngay trong index.

Để cover toàn bộ query, index phải chứa tất cả column từ SQL statement — đặc biệt là các column trong select clause như ví dụ sau:

```
CREATE INDEX sales_sub_eur
    ON sales
     ( subsidiary_id, eur_value );

SELECT SUM(eur_value)
  FROM sales
 WHERE subsidiary_id = ?;
```

Of course indexing the where clause takes precedence over the other

Indexing where clause đương nhiên được ưu tiên hơn các clause khác. Vì vậy, column SUBSIDIARY_ID nằm ở vị trí đầu tiên để đủ điều kiện làm access predicate.

Execution plan cho thấy index scan không có table access tiếp theo (TABLE ACCESS BY INDEX ROWID).

```
-----------------------------------------------------------
| Id  | Operation         | Name          |  Rows | Cost |
-----------------------------------------------------------
|   0 | SELECT STATEMENT  |               |     1 |  104 |
|   1 |  SORT AGGREGATE   |               |     1 |      |
|*  2 |   INDEX RANGE SCAN| SALES_SUB_EUR | 40388 |  104 |
-----------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   2 - access("SUBSIDIARY_ID"=TO_NUMBER(:A))
```

Index cover toàn bộ query nên còn được gọi là covering index.

Index-Only Scan

> Note
>
> Nếu một index ngăn table access thì nó cũng được gọi là covering index.
>
> Tuy nhiên, thuật ngữ này gây hiểu nhầm vì nghe như một property của index. Cụm từ index-only scan mô tả chính xác hơn vì nó gợi ý rằng đây là một operation trong execution plan.

Index có một bản copy của column EUR_VALUE nên database có thể sử dụng value được lưu trong index. Không cần access table vì index đã có toàn bộ thông tin để đáp ứng query.

Index-only scan có thể cải thiện performance rất lớn. Hãy nhìn row count estimate trong execution plan: optimizer dự kiến aggregate hơn 40,000 row. Điều đó có nghĩa index-only scan ngăn 40,000 table fetch — nếu mỗi row nằm trong một table block khác nhau. Nếu index có clustering factor tốt — tức là các row tương ứng được cluster trong một vài table block — lợi ích có thể thấp hơn đáng kể.

Ngoài clustering factor, số row được select cũng giới hạn performance gain tiềm năng của index-only scan. Chẳng hạn, nếu select một row duy nhất thì bạn chỉ tiết kiệm được một table access. Vì tree traversal cũng cần fetch một vài block, table access được tiết kiệm có thể trở nên không đáng kể.

>  Important
>
> Performance advantage của index-only scan phụ thuộc vào số row được access và index clustering factor.

Index-only scan là một indexing strategy mạnh tay. Đừng chỉ vì nghi ngờ mà thiết kế index cho index-only scan, vì cách đó sử dụng memory không cần thiết và làm tăng maintenance effort cần cho các update statement. Xem Chapter 8, “Modifying Data”. Trong thực tế, trước tiên bạn nên index mà không xét đến select clause, rồi chỉ mở rộng index nếu cần.

Chapter 5: Clustering Data

Index-only scan cũng có thể gây ra những bất ngờ khó chịu, chẳng hạn khi giới hạn query ở các sale gần đây:

```
SELECT SUM(eur_value)
  FROM sales
 WHERE subsidiary_id = ?
   AND sale_date > ?;
```

Nếu không xem execution plan, người ta có thể kỳ vọng query chạy nhanh hơn vì nó select ít row hơn. Tuy nhiên, where clause tham chiếu đến một column không nằm trong index, nên database phải access table để load column này.

```
-------------------------------------------------------------------
|Id | Operation                     | Name       | Rows  |Cost |
-------------------------------------------------------------------
| 0 | SELECT STATEMENT              |            |     1 | 371 |
| 1 |  SORT AGGREGATE               |            |     1 |     |
|*2 |   TABLE ACCESS BY INDEX ROWID| SALES      |  2019 | 371 |
|*3 |    INDEX RANGE SCAN           | SALES_DATE| 10541 |  30 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   2 - filter("SUBSIDIARY_ID"=TO_NUMBER(:A))
   3 - access("SALE_DATE">:B)
```

Table access làm tăng response time dù query select ít row hơn. Factor liên quan không phải số row query trả về, mà là số row database phải inspect để tìm chúng.

> 💡 Warning
>
> Mở rộng where clause có thể gây ra behavior performance "phi logic". Hãy kiểm tra execution plan trước khi mở rộng query.

Nếu một index không còn được dùng cho index-only scan, optimizer sẽ chọn execution plan tốt tiếp theo. Điều đó có nghĩa optimizer có thể chọn một execution plan hoàn toàn khác hoặc, như ở trên, một execution plan tương tự nhưng dùng index khác. Trong trường hợp này, nó dùng index trên SALE_DATE, là phần còn lại từ chapter trước.

Index-Only Scan

Từ góc nhìn của optimizer, index này có 2 advantage so với SALES_SUB_EUR. Optimizer cho rằng filter trên SALE_DATE có selectivity cao hơn filter trên SUBSIDIARY_ID. Bạn có thể thấy điều đó trong column "Rows" tương ứng của 2 execution plan gần nhất (khoảng 10,000 so với 40,000). Tuy nhiên, các estimation này hoàn

toàn arbitrary vì query dùng bind parameter. Chẳng hạn, condition SALE_DATE có thể select toàn bộ table nếu truyền vào ngày của sale đầu tiên.

Advantage thứ hai của index SALES_DATE là nó có clustering factor tốt hơn. Đây là một lý do hợp lệ vì table SALES chỉ tăng theo chronological order. Row mới luôn được append vào cuối table nếu không có row nào bị delete. Vì vậy, table order tương ứng với index order vì cả hai được sort gần đúng theo chronological order — index có clustering factor tốt.

Khi dùng index có clustering factor tốt, các row được select được lưu gần nhau, nên database chỉ cần đọc một vài table block để lấy toàn bộ row. Dùng index này, query có thể đủ nhanh mà không cần index-only scan. Trong trường hợp đó, chúng ta nên gỡ các column không cần thiết khỏi index kia.

> Note
>
> Một số index tự động có clustering factor tốt, nên performance advantage của index-only scan là tối thiểu.

Trong ví dụ cụ thể này, có một happy coincidence. Filter mới trên SALE_DATE không chỉ ngăn index-only scan mà đồng thời còn mở ra một access path mới. Vì vậy optimizer có thể giới hạn performance impact của thay đổi này. Tuy nhiên, cũng có thể ngăn index-only scan bằng cách thêm column vào các clause khác. Việc thêm column vào select clause lại không bao giờ mở được một access path mới để hạn chế impact của việc mất index-only scan.

> Tip
>
> Maintain các index-only scan của bạn.
>
> Thêm comment nhắc về index-only scan và tham chiếu đến trang đó để bất kỳ ai cũng có thể đọc phần giải thích.

Chapter 5: Clustering Data

Function-based index cũng có thể gây bất ngờ khó chịu khi kết hợp với index-only scan. Index trên UPPER(last_name) không thể dùng cho index-only scan khi select column LAST_NAME. Ở section trước, đáng lẽ chúng ta phải index chính column LAST_NAME để hỗ trợ LIKE filter và cho phép dùng nó cho index-only scan khi select column LAST_NAME.

>  Tip
>
> Luôn ưu tiên index data gốc vì đó thường là thông tin hữu ích nhất bạn có thể đưa vào index.
>
> Tránh function-based indexing cho những expression không thể dùng làm access predicate.

Các query aggregation như ví dụ trên là candidate tốt cho index-only scan. Chúng query nhiều row nhưng chỉ một vài column, nên một index mảnh là đủ để hỗ trợ index-only scan. Càng query nhiều column, bạn càng phải thêm nhiều column vào index để hỗ trợ index-only scan. Vì vậy, developer chỉ nên select những column thực sự cần.

>  Tip
>
> Tránh select * và chỉ fetch những column bạn cần.

Ngoài việc indexing nhiều row cần nhiều space, bạn cũng có thể chạm đến giới hạn của database. Hầu hết database đặt ra giới hạn khá cứng về số column mỗi index và tổng size của một index entry. Điều đó có nghĩa bạn không thể index số column tùy ý hoặc column có độ dài tùy ý. Overview sau đây liệt kê những giới hạn quan trọng nhất. Tuy vậy, vẫn có những index cover toàn bộ table như chúng ta sẽ thấy trong section tiếp theo.

> Think about it
>
> Query không select bất kỳ table column nào thường được execute bằng index-only scan.
>
> Bạn có nghĩ ra một ví dụ có ý nghĩa không?

Index-Only Scan

MySQL

MySQL 5.6 với InnoDB giới hạn mỗi column ở 767 byte và tổng tất cả column ở 3072 byte. Index MyISAM bị giới hạn ở 16 column và key length tối đa 1000 byte.

MySQL có một feature độc đáo gọi là “prefix indexing” (đôi khi còn gọi là “partial indexing”). Nghĩa là chỉ index một vài character đầu tiên của column — nên nó không liên quan gì đến partial index được mô tả trong Chapter 2. Nếu bạn index một column vượt quá column length cho phép (767 byte với InnoDB), MySQL tự động truncate column tương ứng. Đó là lý do create index statement vẫn thành công với warning “Specified key was too long; max key length is 767 bytes” khi bạn vượt giới hạn. Nghĩa là index không còn chứa full copy của column và vì vậy chỉ hữu ích hạn chế cho index-only scan (tương tự function-based index).

Bạn có thể dùng prefix indexing của MySQL một cách tường minh để tránh vượt quá total key length limit nếu gặp error message “Specified key was too long; max key length is [1000/3072] bytes.” Ví dụ sau chỉ index 10 character đầu tiên của column LAST_NAME.

Tip

Oracle Database

Maximum index key length phụ thuộc vào block size và index storage parameter (75% database block size trừ một phần overhead). B-tree index bị giới hạn ở 32 column.

Khi dùng Oracle 11g với mọi default được giữ nguyên (8k block), maximum index key length là 6398 byte. Vượt quá giới hạn này gây ra error message “ORA-01450: maximum key length (6398) exceeded.”

PostgreSQL

Database PostgreSQL hỗ trợ index-only scan từ release 9.2.

Key length của B-tree index bị giới hạn ở 2713 byte (hardcoded, xấp xỉ BLCKSZ/3). Error message tương ứng “index row size ... exceeds btree maximum, 2713” chỉ xuất hiện khi execute một insert hoặc update vượt quá giới hạn. B-tree index có thể chứa tối đa 32 column.

Chapter 5: Clustering Data

SQL Server

SQL Server giới hạn key length ở 900 byte và 16 key column. Tuy nhiên, SQL Server có feature cho phép thêm column dài tùy ý vào index chỉ nhằm hỗ trợ index-only scan. Vì mục đích đó, SQL Server phân biệt key column và nonkey column.

Key column là các index column như đã thảo luận từ đầu. Nonkey column là các column bổ sung chỉ được lưu trong index leaf node. Nonkey column có thể dài tùy ý nhưng không thể dùng làm access predicate (seek predicate).

Nonkey column được định nghĩa bằng keyword include của create index command:

```
 CREATE INDEX empsubupnam
     ON employees
       (subsidiary_id, last_name)
INCLUDE(phone_number, first_name);
```

Index-Organized Tables

Index-only scan execute một SQL statement chỉ bằng redundant data được lưu trong index. Original data trong heap table không cần thiết. Nếu đẩy concept đó lên mức tiếp theo và đưa tất cả column vào index, bạn có thể tự hỏi tại sao chúng ta còn cần heap table.

Một số database thực sự có thể dùng index làm primary table store. Database Oracle gọi concept này là index-organized tables (IOT), còn database khác dùng thuật ngữ clustered index. Trong section này, cả hai thuật ngữ được dùng tùy theo việc muốn nhấn mạnh đặc điểm của table hay của index.

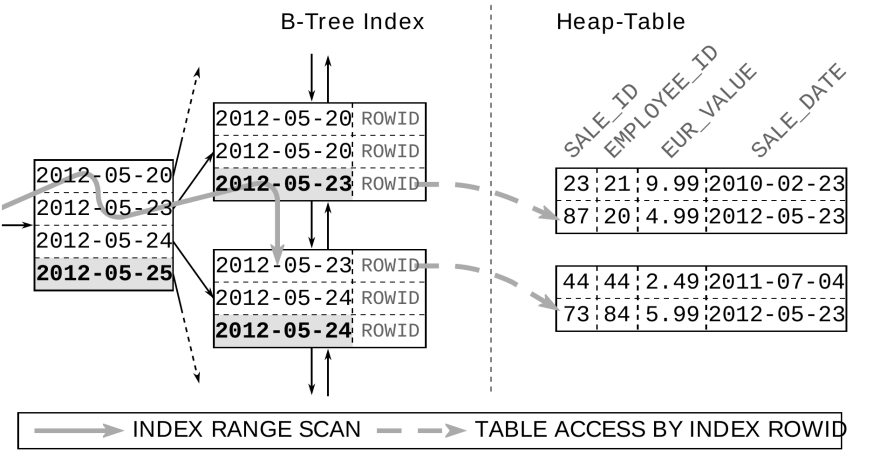
Như vậy, index-organized table là một B-tree index không có heap table. Điều này tạo ra 2 benefit: (1) tiết kiệm space cho heap structure; (2) mọi access trên clustered index tự động là một index-only scan. Cả hai benefit nghe đều hứa hẹn nhưng thực tế khó đạt được.

Index-Organized Tables

Nhược điểm của index-organized table trở nên rõ ràng khi tạo một index khác trên cùng table. Tương tự regular index, secondary index tham chiếu đến original table data — vốn được lưu trong clustered index. Ở đó, data không được lưu tĩnh như trong heap table mà có thể di chuyển bất kỳ lúc nào để duy trì index order. Vì vậy, không thể lưu physical location của row trong index-organized table vào secondary index. Thay vào đó database phải dùng logical key.

Các figure sau cho thấy một index lookup để tìm tất cả sale vào May 23rd 2012. Để so sánh, trước tiên chúng ta xem Figure 5.1, mô tả process khi dùng heap table. Execution gồm 2 bước: (1) INDEX RANGE SCAN; (2) TABLE ACCESS BY INDEX ROWID.

Figure 5.1. Index-Based Access trên Heap Table



Dù table access có thể trở thành bottleneck, nó vẫn chỉ giới hạn ở một read operation mỗi row vì index có ROWID làm direct pointer đến table row. Database có thể load row ngay lập tức từ heap table vì index có exact position của row. Tuy nhiên, picture thay đổi khi dùng secondary index trên index-organized table. Secondary index không lưu physical pointer (ROWID), mà chỉ lưu key value của clustered index — cái gọi là clustering key. Thường đó là primary key của index-organized table.

Chapter 5: Clustering Data

Access secondary index không trả về ROWID mà trả về logical key để tìm kiếm clustered index. Tuy nhiên, một access là chưa đủ để tìm trong clustered index — cần full tree traversal. Nghĩa là access table

thông qua secondary index sẽ search 2 index: secondary index một lần (INDEX RANGE SCAN), sau đó search clustered index cho mỗi row tìm thấy trong secondary index (INDEX UNIQUE SCAN).

Figure 5.2. Secondary Index trên một IOT

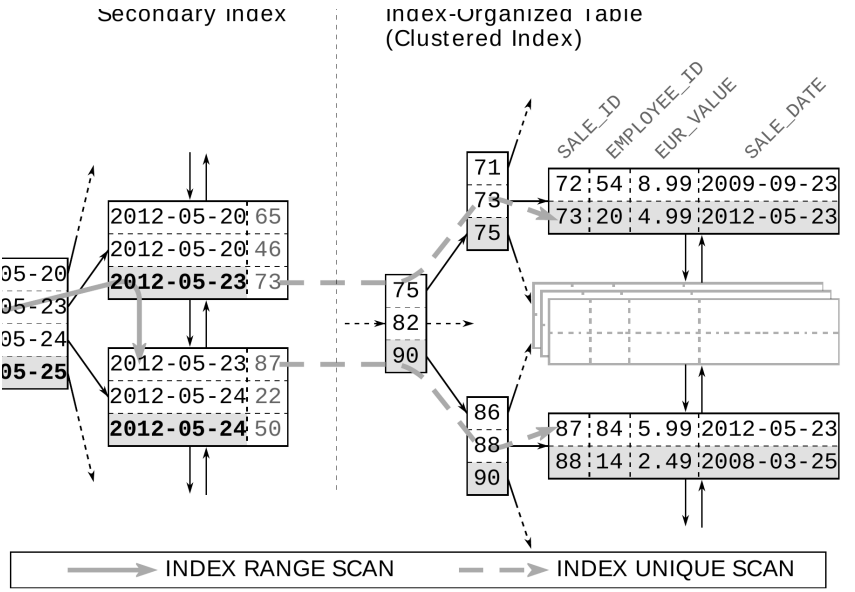


Figure 5.2 cho thấy rõ B-tree của clustered index nằm giữa secondary index và table data.

Access index-organized table thông qua secondary index rất inefficient, và có thể ngăn điều đó giống như cách ngăn table access trên heap table: dùng index-only scan — trong trường hợp này mô tả chính xác hơn là “secondary-index-only scan”. Performance advantage của index-only scan còn lớn hơn vì nó không chỉ ngăn một access mà ngăn toàn bộ INDEX UNIQUE SCAN.

> **Important**
>
> Access index-organized table thông qua secondary index rất inefficient.

Index-Organized Tables

Dùng ví dụ này, chúng ta cũng thấy database khai thác mọi redundancy mà nó có. Hãy nhớ rằng secondary index lưu clustering key cho mỗi index entry. Vì vậy, chúng ta có thể query clustering key từ secondary index mà không cần access index-organized table:

```
SELECT sale_id
  FROM sales_iot
 WHERE sale_date = ?;

------------------------------------------------
| Id | Operation         | Name           | Cost |
------------------------------------------------
|  0 | SELECT STATEMENT  |                |    4 |
|* 1 |  INDEX RANGE SCAN| SALES_IOT_DATE |    4 |
------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - access("SALE_DATE"=:DT)
```

Table SALES_IOT là một index-organized table dùng SALE_ID làm clustering key. Dù index SALE_IOT_DATE chỉ nằm trên column SALE_DATE, nó vẫn có một bản copy của clustering key SALE_ID, nên có thể đáp ứng query chỉ bằng secondary index.

Khi select các column khác, database phải chạy một INDEX UNIQUE SCAN trên clustered index cho mỗi row:

```
SELECT eur_value
  FROM sales_iot
 WHERE sale_date = ?;

------------------------------------------------
| Id  | Operation         | Name           | Cost |
------------------------------------------------
|   0 | SELECT STATEMENT  |                |   13 |
|*  1 |  INDEX UNIQUE SCAN| SALES_IOT_PK   |   13 |
|*  2 |   INDEX RANGE SCAN| SALES_IOT_DATE |    4 |
------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - access("SALE_DATE"=:DT)
   2 - access("SALE_DATE"=:DT)
```

Chapter 5: Clustering Data

Index-organized table và clustered index, sau cùng, không hữu ích như ấn tượng ban đầu. Performance improvement trên clustered index dễ dàng mất đi khi dùng secondary index. Clustering key thường dài hơn ROWID, nên secondary index lớn hơn so với trên heap table, thường xóa bỏ phần space tiết kiệm được do không có heap table. Điểm mạnh của index-organized table và clustered index chủ yếu giới hạn ở những table không cần index thứ hai. Heap table có lợi thế cung cấp một master copy cố định, dễ tham chiếu.

> Important
>
> Table chỉ có một index được implement tốt nhất dưới dạng clustered index hoặc index-organized table.
>
> Table có nhiều index thường có thể hưởng lợi từ heap table. Bạn vẫn có thể dùng index-only scan để tránh table access. Cách này đem lại select performance của clustered index mà không làm chậm các index khác.

Database support cho index-organized table và clustered index rất inconsistent. Overview ở trang tiếp theo giải thích những điểm đặc thù quan trọng nhất.



Why Secondary Indexes have no ROWID

Một direct pointer đến table row cũng sẽ hữu ích cho secondary index. Nhưng điều đó chỉ khả thi nếu table row nằm ở các storage position cố định. Đáng tiếc là điều này không thể xảy ra nếu row là một phần của index structure được duy trì theo thứ tự. Duy trì index order đôi khi cần di chuyển row. Điều này cũng đúng với operation không tác động đến bản thân row. Chẳng hạn, một insert statement có thể split leaf node để tạo space cho entry mới. Điều đó có nghĩa một số entry được chuyển sang data block mới ở một vị trí khác.

Ngược lại, heap table không giữ row theo thứ tự nào. Database lưu entry mới ở bất cứ nơi nào tìm thấy đủ space. Sau khi được ghi, data không di chuyển trong heap table.

Index-Organized Tables

MySQL

Engine MyISAM chỉ dùng heap table, trong khi engine InnoDB luôn dùng clustered index. Nghĩa là bạn không trực tiếp có quyền lựa chọn.

Oracle Database

Database Oracle mặc định dùng heap table. Có thể tạo index-organized table bằng clause ORGANIZATION INDEX:

PostgreSQL only uses heap tables.

You can, however, use the CLUSTER clause to align the contents of th
heap table with an index.

SQL Server

Database Oracle luôn dùng primary key làm clustering key.

PostgreSQL

PostgreSQL chỉ dùng heap table.

Tuy nhiên, bạn có thể dùng clause CLUSTER để căn chỉnh nội dung của heap table theo một index.

SQL Server

Mặc định SQL Server dùng clustered index (index-organized table), sử dụng primary key làm clustering key. Tuy nhiên, bạn có thể dùng column bất kỳ làm clustering key — kể cả column không unique.

Để tạo heap table, bạn phải dùng clause NONCLUSTERED trong primary key definition:

```
    CONSTRAINT pk PRIMARY KEY NONCLUSTERED (id)
);
```

Dropping a clustered index transforms the table into a heap table.

SQL Server's default behavior often causes performance problem
when using secondary indexes.

Drop một clustered index sẽ transform table thành heap table.

Behavior mặc định của SQL Server thường gây performance problem khi dùng secondary index.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Chapter 6

Sorting and Grouping

Sorting là một operation tiêu tốn rất nhiều resource. Nó cần khá nhiều CPU time, nhưng vấn đề chính là database phải tạm thời buffer result. Sau cùng, sort operation phải đọc toàn bộ input trước khi có thể tạo output đầu tiên. Sort operation không thể execute theo pipelined manner — điều này có thể trở thành vấn đề với data set lớn.

Index cung cấp một ordered representation của indexed data: principle này đã được mô tả trong Chapter 1. Ta cũng có thể nói index lưu data theo dạng presorted. Thực tế index được sort giống như khi dùng index definition trong order by clause. Vì vậy, không có gì bất ngờ khi ta có thể dùng index để tránh sort operation nhằm đáp ứng order by clause.

Trớ trêu là INDEX RANGE SCAN cũng trở nên inefficient với data set lớn — đặc biệt khi theo sau là table access. Điều này có thể triệt tiêu phần tiết kiệm do tránh sort operation. Trong trường hợp này, FULL TABLE SCAN với explicit sort operation thậm chí có thể nhanh hơn. Một lần nữa, công việc của optimizer là đánh giá các execution plan khác nhau và chọn plan tốt nhất.

Indexed order by execution không chỉ tiết kiệm sorting effort; nó còn có thể trả về các result đầu tiên mà không cần xử lý toàn bộ input data. Vì vậy order by được execute theo pipelined manner. Chapter 7, “Partial Results”, giải thích cách khai thác pipelined execution để implement pagination query hiệu quả. Điều này khiến pipelined order by quan trọng đến mức tôi gọi nó là power thứ ba của indexing.

Chapter này giải thích cách dùng index cho pipelined order by execution. Để làm vậy, chúng ta phải đặc biệt chú ý đến tương tác với where clause cũng như modifier ASC và DESC. Chapter kết thúc bằng việc áp dụng các kỹ thuật này cho group by clause.

Chapter 6: Sorting and Grouping

Indexing Order By

SQL query có order by clause không cần explicit sort result nếu relevant index đã trả về row theo đúng order cần thiết. Nghĩa là cùng index được dùng cho where clause cũng phải cover order by clause.

```
SELECT sale_date, product_id, quantity
  FROM sales
 WHERE sale_date = TRUNC(sysdate) - INTERVAL '1' DAY
 ORDER BY sale_date, product_id;
```

Đã có index trên SALE_DATE có thể dùng cho where clause. Tuy nhiên, database phải thực hiện explicit sort operation để đáp ứng order by clause:

```
-------------------------------------------------------------------
|Id | Operation                    | Name       | Rows | Cost |
-------------------------------------------------------------------
| 0 | SELECT STATEMENT             |            |  320 |   18 |
| 1 |  SORT ORDER BY               |            |  320 |   18 |
| 2 |   TABLE ACCESS BY INDEX ROWID| SALES      |  320 |   17 |
|*3 |    INDEX RANGE SCAN          | SALES_DATE |  320 |    3 |
-------------------------------------------------------------------
```

INDEX RANGE SCAN vốn đã trả result theo index order. Để tận dụng điều này, chúng ta chỉ cần mở rộng index definition để nó tương ứng với order by clause:

An INDEX RANGE SCAN delivers the result in index order anyway. To take advantage of this fact, we just have to extend the index definition so it corresponds to the **order by** clause:

```
CREATE INDEX sales_dt_pr ON sales (sale_date, product_id);
```

```
-------------------------------------------------------------------
|Id | Operation                    | Name        | Rows | Cost |
-------------------------------------------------------------------
| 0 | SELECT STATEMENT             |             |  320 |  300 |
| 1 |  TABLE ACCESS BY INDEX ROWID| SALES       |  320 |  300 |
|*2 |   INDEX RANGE SCAN           | SALES_DT_PR |  320 |    4 |
-------------------------------------------------------------------
```

Indexing Order By

Sort operation SORT ORDER BY biến mất khỏi execution plan dù query vẫn có order by clause. Database khai thác index order và bỏ qua explicit sort operation.

> Important
>
> Nếu index order tương ứng với order by clause, database có thể bỏ qua explicit sort operation.

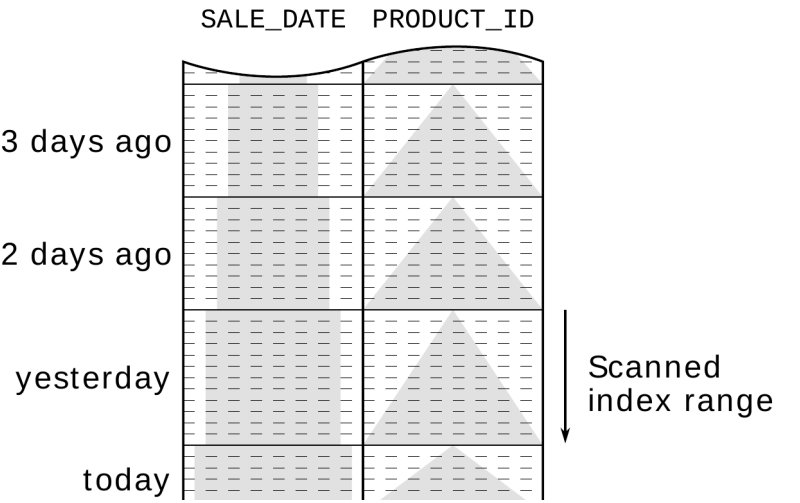
Dù execution plan mới có ít operation hơn, cost value lại tăng đáng kể vì clustering factor của index mới tệ hơn (xem “Automatically Optimized Clustering Factor” ở page 133). Ở thời điểm này chỉ cần ghi nhận rằng cost value không phải lúc nào cũng là indicator tốt cho execution effort.

Với optimization này, chỉ cần scanned index range được sort theo order by clause. Vì vậy, optimization cũng hoạt động trong ví dụ cụ thể này khi chỉ sort theo PRODUCT_ID:

has increased considerably because the clustering factor of the new index is worse (see “Automatically Optimized Clustering Factor” on page 133). At this point, it should just be noted that the cost value is not always a good indicator of the execution effort.

Trong Figure 6.1, ta thấy PRODUCT_ID là sort criterion duy nhất liên quan trong scanned index range. Do đó index order tương ứng với order by clause trong index range này, nên database có thể bỏ qua sort operation.

Figure 6.1. Sort Order trong Relevant Index Range



Optimization này có thể gây behavior bất ngờ khi mở rộng scanned index range:

```
SELECT sale_date, product_id, quantity
  FROM sales
 WHERE sale_date >= TRUNC(sysdate) - INTERVAL '1' DAY
 ORDER BY product_id;
```

Query này không lấy sale của yesterday mà lấy tất cả sale kể từ yesterday. Nghĩa là nó cover nhiều ngày và scan một index range không chỉ được sort theo PRODUCT_ID. Nếu nhìn lại Figure 6.1 và

mở rộng scanned index range xuống dưới, ta sẽ thấy lại có các PRODUCT_ID value nhỏ hơn. Vì vậy database phải dùng explicit sort operation để đáp ứng order by clause.

```
-------------------------------------------------------------------
|Id |Operation                      | Name         | Rows | Cost |
-------------------------------------------------------------------
| 0 |SELECT STATEMENT               |              |  320 |  301 |
| 1 | SORT ORDER BY                 |              |  320 |  301 |
| 2 |  TABLE ACCESS BY INDEX ROWID| SALES        |  320 |  300 |
|*3 |   INDEX RANGE SCAN            | SALES_DT_PR |  320 |    4 |
-------------------------------------------------------------------
```

f the database uses a sort operation even though you expected a pipelined
xecution, it can have two reasons: (1) the execution plan with the explicit

Nếu database dùng sort operation dù bạn kỳ vọng pipelined execution, có thể có 2 lý do: (1) execution plan với explicit sort operation có cost value tốt hơn; (2) index order trong scanned index range không tương ứng với order by clause.

Một cách đơn giản để phân biệt 2 trường hợp là dùng full index definition trong order by clause — tức là điều chỉnh query theo index để loại bỏ nguyên nhân thứ hai. Nếu database vẫn dùng explicit sort operation, optimizer ưu tiên plan này do cost value; nếu không, database không thể dùng index cho order by clause ban đầu.

> Tip
>
> Dùng full index definition trong order by clause để tìm lý do của explicit sort operation.

Indexing Order By

Trong cả 2 trường hợp, bạn có thể tự hỏi liệu có thể đạt pipelined order by execution hay không và đạt bằng cách nào. Để làm vậy, bạn có thể execute query với full index definition trong order by clause rồi inspect result. Bạn thường sẽ nhận ra perception của mình về

index là sai, và index order thực tế không đúng như order by clause ban đầu yêu cầu, nên database không thể dùng index để tránh sort operation.

Nếu optimizer ưu tiên explicit sort operation vì cost value, thường là do optimizer chọn execution plan tốt nhất cho full execution của query. Nói cách khác, optimizer chọn execution plan nhanh nhất để lấy record cuối cùng. Nếu database nhận ra application chỉ fetch một vài row đầu, nó có thể chuyển sang ưu tiên indexed order by. Chapter 7, “Partial Results”, giải thích các optimization method tương ứng.

Automatically Optimized Clustering Factor

Database Oracle giữ clustering factor ở mức tối thiểu bằng cách xét ROWID trong index order. Khi 2 index entry có cùng key value, ROWID quyết định order cuối cùng của chúng. Vì vậy index cũng được order theo table order và có clustering factor nhỏ nhất có thể, vì ROWID biểu diễn physical address của table row.

Khi thêm một column khác vào index, bạn chèn một sort criterion mới trước ROWID. Database có ít tự do hơn trong việc align index entry theo table order, nên index clustering factor chỉ có thể xấu đi.

Dù vậy, index order vẫn có thể tương ứng gần đúng với table order. Sale của một ngày có lẽ vẫn được cluster cùng nhau trong table cũng như trong index — dù sequence của chúng không còn hoàn toàn giống nhau. Database phải đọc table block nhiều lần khi dùng index SALE_DT_PR — nhưng đó vẫn là các table block giống như trước. Do data thường xuyên được access được cache, performance impact có thể thấp hơn đáng kể so với mức cost value thể hiện.

Chapter 6: Sorting and Grouping

Indexing ASC, DESC and NULLS FIRST/LAST

Database có thể đọc index theo cả 2 direction. Điều đó có nghĩa pipelined order by cũng khả thi nếu scanned index range ở đúng order ngược với order được chỉ định trong order by clause. Dù modifier ASC và DESC trong order by clause có thể ngăn pipelined execution, hầu hết database cung cấp cách đơn giản để đổi index order, khiến index có thể dùng cho pipelined order by.

Ví dụ sau dùng index theo reverse order. Nó trả về các sale kể từ yesterday, được order theo date giảm dần và PRODUCT_ID giảm dần.

of a pipelined order by.

The following example uses an index in reverse order. It delivers the sales since yesterday ordered by descending date and descending `PRODUCT_ID`.

```
SELECT sale_date, product_id, quantity
  FROM sales
```

Execution plan cho thấy database đọc index theo descending direction.

irection.

```
-------------------------------------------------------------------
|Id |Operation                    | Name        | Rows | Cost |
-------------------------------------------------------------------
| 0 |SELECT STATEMENT             |             |  320 |  300 |
| 1 | TABLE ACCESS BY INDEX ROWID | SALES       |  320 |  300 |
|*2 |  INDEX RANGE SCAN DESCENDING| SALES_DT_PR |  320 |    4 |
-------------------------------------------------------------------
```

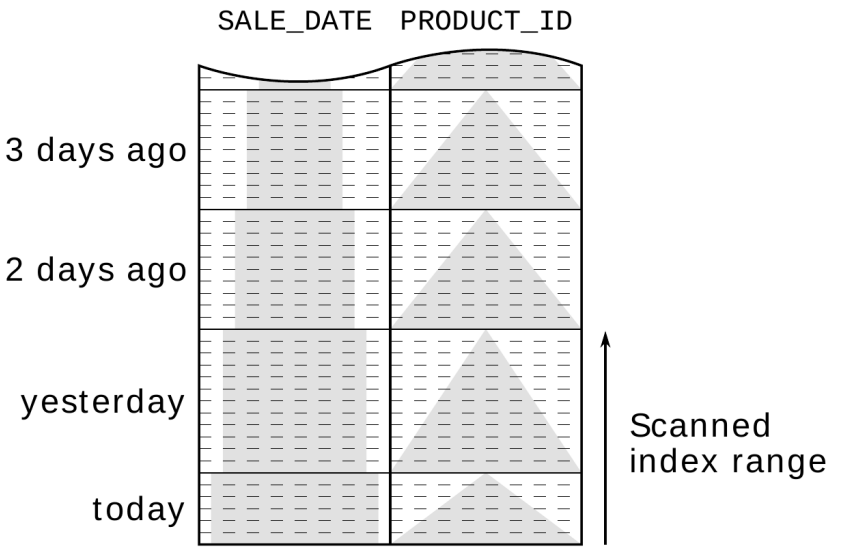
Trong trường hợp này, database dùng index tree để tìm matching entry cuối cùng. Từ đó, nó đi theo leaf node chain "lên trên" như trong Figure 6.2. Sau cùng, đó là lý do database dùng doubly linked list để xây dựng leaf node chain.

Đương nhiên, điều quan trọng là scanned index range phải ở đúng order ngược với order mà order by clause cần.

>  Important
>
> Database có thể đọc index theo cả 2 direction.

Indexing ASC, DESC và NULLS FIRST/LAST

Figure 6.2. Reverse Index Scan

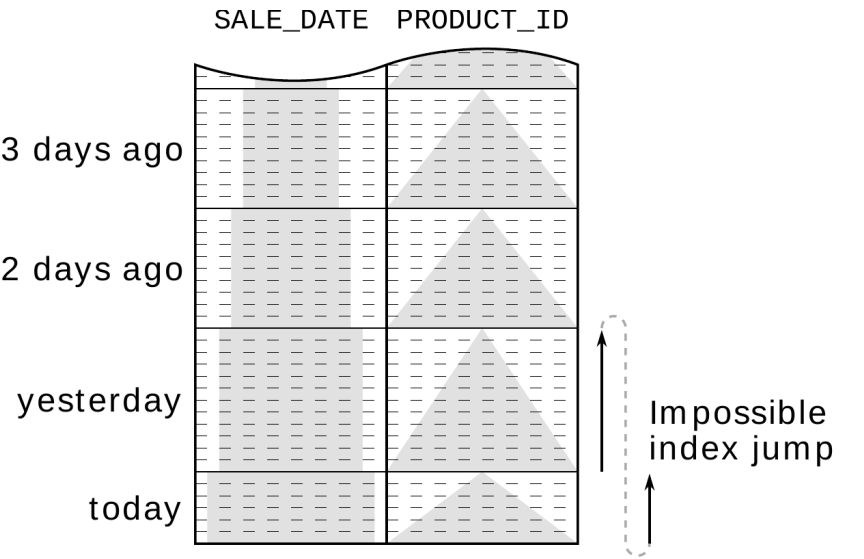


Ví dụ sau không đáp ứng prerequisite này vì nó trộn modifier ASC và DESC trong order by clause:

```
SELECT sale_date, product_id, quantity
  FROM sales
 WHERE sale_date >= TRUNC(sysdate) - INTERVAL '1' DAY
 ORDER BY sale_date ASC, product_id DESC;
```

Query trước hết phải trả về các sale của yesterday theo thứ tự PRODUCT_ID giảm dần, sau đó là các sale của today, cũng theo PRODUCT_ID giảm dần. Figure 6.3 minh họa process này. Để lấy các sale theo đúng order yêu cầu, database sẽ phải “jump” trong lúc index scan.

Figure 6.3. Impossible Pipelined order by



Tuy nhiên, index không có link từ sale của yesterday có PRODUCT_ID nhỏ nhất đến sale của today có PRODUCT_ID lớn nhất. Vì vậy database không thể dùng index này để tránh explicit sort operation.

Với những trường hợp như vậy, hầu hết database đều cung cấp một cách đơn giản để điều chỉnh index order theo order by clause. Cụ thể, bạn có thể dùng modifier ASC và DESC trong index declaration:

```
  DROP INDEX sales_dt_pr;

CREATE INDEX sales_dt_pr
    ON sales (sale_date ASC, product_id DESC);
```

> **Warning**
>
> Database MySQL bỏ qua modifier ASC và DESC trong index definition.

Bây giờ index order tương ứng với order by clause nên database có thể bỏ qua sort operation:

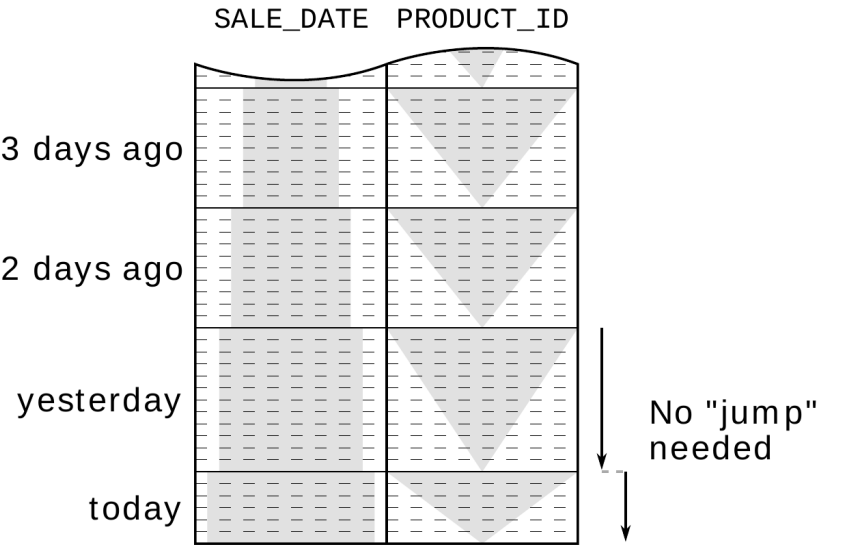
```
-------------------------------------------------------------------
|Id | Operation                    | Name        | Rows | Cost |
-------------------------------------------------------------------
| 0 | SELECT STATEMENT             |             |  320 |  301 |
| 1 |  TABLE ACCESS BY INDEX ROWID| SALES       |  320 |  301 |
|*2 |   INDEX RANGE SCAN           | SALES_DT_PR |  320 |    4 |
-------------------------------------------------------------------
```

Figure 6.4 cho thấy index order mới. Việc thay đổi sort direction của column thứ hai thực chất đảo direction của các mũi tên trong figure trước. Nhờ vậy, mũi tên thứ nhất kết thúc tại nơi mũi tên thứ hai bắt đầu, khiến index có các row theo đúng order mong muốn.

>  Important
>
> Khi dùng các modifier ASC và DESC kết hợp trong order by clause, bạn cũng phải define index theo cách tương tự để có thể dùng index cho pipelined order by.
>
> Điều này không ảnh hưởng đến khả năng dùng index cho where clause.

Indexing ASC, DESC và NULLS FIRST/LAST

Figure 6.4. Mixed-Order Index



ASC/DESC indexing chỉ cần thiết khi sort từng column theo các direction đối nghịch nhau. Nó không cần thiết để đảo order của tất cả column vì database vẫn có thể đọc index theo descending order nếu cần — ngoại lệ duy nhất là secondary index trên index-organized table. Secondary index ngầm thêm clustering key vào index nhưng không cung cấp khả năng chỉ định sort order. Nếu cần sort clustering key theo descending order, bạn không còn lựa chọn nào khác ngoài việc sort tất cả column còn lại theo descending order. Khi đó database có thể đọc index theo reverse direction để lấy order mong muốn.

Ngoài ASC và DESC, SQL standard còn định nghĩa hai modifier ít được biết đến cho order by clause: NULLS FIRST và NULLS LAST. Khả năng kiểm soát rõ ràng việc sort NULL được đưa vào như một optional extension “gần đây” trong SQL:2003. Vì vậy, mức độ hỗ trợ giữa các database khá hạn chế. Điều này đặc biệt đáng lo vì standard không định nghĩa chính xác sort order của NULL. Standard chỉ quy

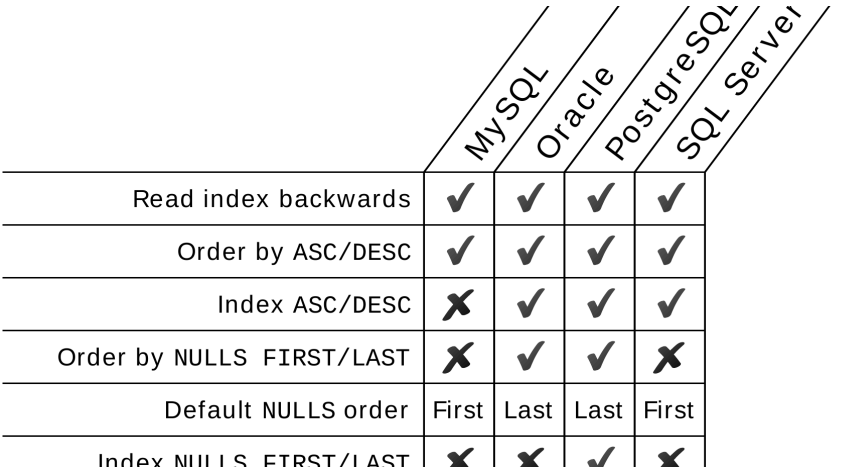
định tất cả NULL phải nằm cạnh nhau sau khi sort, nhưng không nói chúng phải đứng trước hay sau các entry khác. Nói chính xác hơn, bạn cần chỉ định NULL sorting cho mọi column có thể nhận NULL trong order by clause để có behavior được định nghĩa rõ ràng.

Tuy nhiên, thực tế là optional extension này chưa được SQL Server 2012 hoặc MySQL 5.6 implement. Ngược lại, Oracle database đã hỗ trợ NULLS sorting từ trước khi nó được đưa vào standard, nhưng tính đến release 11g, Oracle không chấp nhận nó trong index definition. Vì vậy Oracle database không thể thực hiện pipelined order by khi sort với

NULLS FIRST. Chỉ PostgreSQL database (từ release 8.3) hỗ trợ modifier NULLS trong cả order by clause và index definition.

Overview sau đây tóm tắt các feature được những database khác nhau cung cấp.

Figure 6.5. Database/Feature Matrix



Indexing Group By

Indexing Group By

SQL database dùng hai group by algorithm hoàn toàn khác nhau. Algorithm thứ nhất là hash algorithm, aggregate các input record trong một temporary hash table. Sau khi xử lý toàn bộ input record, hash table được trả về làm result. Algorithm thứ hai là sort/group algorithm, trước hết sort input data theo grouping key để các row của mỗi group nằm liền kề nhau. Sau đó database chỉ cần aggregate chúng. Nói chung, cả hai algorithm đều cần materialize một intermediate state, nên chúng không được execute theo pipelined manner. Tuy vậy, sort/group algorithm có thể dùng index để tránh sort operation, nhờ đó enable pipelined group by.

>  Note
>
> MySQL 5.6 không dùng hash algorithm. Tuy nhiên, optimization cho sort/group algorithm vẫn hoạt động như mô tả bên dưới.

Xét query sau. Query này trả về revenue của yesterday được group theo PRODUCT_ID:

```
SELECT product_id, sum(eur_value)
  FROM sales
 WHERE sale_date = TRUNC(sysdate) - INTERVAL '1' DAY
 GROUP BY product_id;
```

Vì đã biết index trên SALE_DATE và PRODUCT_ID từ section trước, sort/group algorithm phù hợp hơn vì INDEX RANGE SCAN tự động trả về các row theo order cần thiết. Nghĩa là database tránh được materialization vì không cần explicit sort operation — group by được execute theo pipelined manner.

```
-------------------------------------------------------------------
|Id |Operation                      | Name        | Rows | Cost |
-------------------------------------------------------------------
| 0 |SELECT STATEMENT               |             |   17 |  192 |
| 1 | SORT GROUP BY NOSORT          |             |   17 |  192 |
| 2 |  TABLE ACCESS BY INDEX ROWID| SALES       |  321 |  192 |
|*3 |   INDEX RANGE SCAN            | SALES_DT_PR |  321 |    3 |
```

Chapter 6: Sorting and Grouping

Execution plan của Oracle database đánh dấu một SORT GROUP BY được execute theo pipelined manner bằng addendum NOSORT. Execution plan của các database khác không nhắc đến sort operation nào.

Pipelined group by có cùng prerequisite với pipelined order by, ngoại trừ việc không có modifier ASC và DESC. Nghĩa là define index với modifier ASC/DESC không nên ảnh hưởng đến pipelined group by execution. Điều tương tự cũng đúng với NULLS FIRST/LAST. Tuy nhiên, có những database không thể sử dụng đúng một ASC/DESC index cho pipelined group by.

> Warning
>
> Với PostgreSQL, bạn phải thêm order by clause để index có NULLS LAST sorting có thể được dùng cho pipelined group by.
>
> Oracle database không thể đọc index theo backward direction để execute pipelined group by theo sau bởi order by.

Nếu mở rộng query để xét tất cả sale kể từ yesterday, như trong ví dụ về pipelined order by, điều đó sẽ ngăn pipelined group by vì cùng lý do trước đó: INDEX RANGE SCAN không trả về các row theo order của grouping key (so sánh Figure 6.1 ở page 131).

```
SELECT product_id, sum(eur_value)
  FROM sales
 WHERE sale_date >= TRUNC(sysdate) - INTERVAL '1' DAY
 GROUP BY product_id;
```

```
-------------------------------------------------------------------
|Id |Operation                     | Name        | Rows | Cost |
-------------------------------------------------------------------
| 0 |SELECT STATEMENT              |             |   24 |  356 |
| 1 |  HASH GROUP BY               |             |   24 |  356 |
| 2 |   TABLE ACCESS BY INDEX ROWID| SALES       |  596 |  355 |
|*3 |    INDEX RANGE SCAN          | SALES_DT_PR |  596 |    4 |
-------------------------------------------------------------------
```

Thay vào đó, Oracle database dùng hash algorithm. Ưu điểm của hash algorithm là nó chỉ cần buffer aggregated result, trong khi sort/group algorithm materialize toàn bộ input set. Nói cách khác, hash algorithm cần ít memory hơn.

Indexing Group By

Cũng như pipelined order by, execution nhanh không phải là khía cạnh quan trọng nhất của pipelined group by execution. Quan trọng hơn là database execute nó theo pipelined manner và trả về result đầu tiên trước khi đọc toàn bộ input. Đây là prerequisite cho các advanced optimization method được giải thích trong chapter tiếp theo.

> Think about it
>
> Ngoài sorting và grouping, bạn có nghĩ ra database operation nào khác có thể dùng index để tránh sorting không?

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Chapter 7

Partial Results

Đôi khi bạn không cần toàn bộ result của một SQL query mà chỉ cần vài row đầu tiên — chẳng hạn để chỉ hiển thị mười message mới nhất. Trong trường hợp này, người dùng cũng thường được phép browse các message cũ hơn — bằng paging navigation truyền thống hoặc biến thể hiện đại hơn là "infinite scrolling". Tuy nhiên, các SQL query liên quan đến chức năng này có thể gây ra performance problem nghiêm trọng nếu phải sort toàn bộ message để tìm các message mới nhất. Vì vậy, pipelined order by là một phương pháp optimization rất mạnh cho các query như vậy.

Chapter này trình bày cách dùng pipelined order by để retrieve partial result một cách hiệu quả. Dù syntax của các query này khác nhau giữa các database, chúng vẫn execute query theo cách rất giống nhau. Một lần nữa, điều này cho thấy tất cả chúng đều put their pants on one leg at a time.

Querying Top-N Rows

Top-N query là query giới hạn result ở một số row cụ thể. Đây thường là query lấy các entry mới nhất hoặc "tốt nhất" trong một result set. Để execute hiệu quả, việc ranking phải được thực hiện bằng pipelined order by.

Cách đơn giản nhất để chỉ fetch các row đầu tiên của query là fetch các row cần thiết rồi đóng statement. Đáng tiếc là optimizer không thể dự đoán điều đó khi prepare execution plan. Để chọn execution plan tốt nhất, optimizer phải biết application cuối cùng sẽ fetch toàn bộ row hay không. Trong trường hợp đó, full table scan với explicit sort operation có thể perform tốt nhất, dù pipelined order by có thể

tốt hơn khi chỉ fetch mười row — ngay cả khi database phải fetch từng row một. Nghĩa là optimizer phải biết bạn sẽ abort statement trước khi fetch toàn bộ row để có thể chọn execution plan tốt nhất.

Chapter 7: Partial Results

> 💡 Tip
>
> Hãy thông báo cho database khi bạn không cần tất cả row.

SQL standard đã không yêu cầu điều này trong một thời gian dài. Extension tương ứng (fetch first) chỉ mới được giới thiệu trong SQL:2008 và hiện chỉ có trong IBM DB2, PostgreSQL và SQL Server 2012. Một mặt, đó là vì feature này là non-core extension; mặt khác, mỗi database đã cung cấp giải pháp proprietary riêng trong nhiều năm.

Các ví dụ sau cho thấy cách dùng những extension quen thuộc này bằng cách query mười sale mới nhất. Cơ sở luôn giống nhau: fetch toàn bộ sale, bắt đầu từ sale mới nhất. Top-N syntax tương ứng chỉ abort execution sau khi fetch mười row.

MySQL

MySQL và PostgreSQL dùng limit clause để giới hạn số row được fetch.

```
SELECT *
  FROM sales
 ORDER BY sale_date DESC
 LIMIT 10;
```

Oracle Database

Oracle database cung cấp pseudo column ROWNUM, tự động đánh số các row trong result set. Để dùng column này trong một filter, chúng ta phải wrap query:

```
SELECT *
  FROM (
       SELECT *
         FROM sales
        ORDER BY sale_date DESC
       )
 WHERE rownum <= 10;
```

Querying Top-N Rows

PostgreSQL

PostgreSQL hỗ trợ fetch first extension từ version 8.4. Limit clause được dùng trước đây vẫn hoạt động như trong ví dụ MySQL.

```
SELECT *
  FROM sales
 ORDER BY sale_date DESC
 FETCH FIRST 10 ROWS ONLY;
```

SQL Server

SQL Server cung cấp top clause để giới hạn số row được fetch.

```
SELECT TOP 10 *
  FROM sales
 ORDER BY sale_date DESC;
```

Starting with release 2012, SQL Server supports the `fetch f`

Bắt đầu từ release 2012, SQL Server cũng hỗ trợ fetch first extension.

Tất cả SQL query nêu trên đều đặc biệt vì database nhận diện chúng là top-N query.

> Important
>
> Database chỉ có thể optimize query cho partial result nếu ngay từ đầu nó biết điều này.

Nếu optimizer biết rằng chúng ta chỉ cần mười row, nó sẽ ưu tiên dùng pipelined order by nếu phù hợp:

```
-------------------------------------------------------------
| Operation                     | Name        | Rows | Cost |
-------------------------------------------------------------
| SELECT STATEMENT              |             |   10 |    9 |
|  COUNT STOPKEY                |             |      |      |
|   VIEW                        |             |   10 |    9 |
|    TABLE ACCESS BY INDEX ROWID| SALES       | 1004K|    9 |
|     INDEX FULL SCAN DESCENDING| SALES_DT_PR |   10 |    3 |
-------------------------------------------------------------
```

Execution plan của Oracle cho biết termination đã được plan bằng operation COUNT STOPKEY. Điều đó nghĩa là database đã nhận diện top-N syntax.

Chapter 7: Partial Results

> Tip
>
> Appendix A, “Execution Plans”, tóm tắt các operation tương ứng cho MySQL, Oracle, PostgreSQL và SQL Server.

Dùng đúng syntax mới chỉ là một nửa câu chuyện vì để termination hiệu quả, các operation bên dưới cũng phải được execute theo pipelined manner. Nghĩa là order by clause phải được cover bởi một index — trong ví dụ này là index SALE_DT_PR trên SALE_DATE và PRODUCT_ID. Khi dùng index này, database có thể tránh explicit sort operation và ngay lập tức gửi các row cho application khi đọc chúng từ index. Execution bị abort sau khi fetch mười row nên database không đọc nhiều row hơn số row được select.

> Important
>
> Pipelined top-N query không cần đọc và sort toàn bộ result set.

Nếu không có index phù hợp trên SALE_DATE cho pipelined order by, database phải đọc và sort toàn bộ table. Row đầu tiên chỉ được trả về sau khi đọc row cuối cùng từ table.

```
-------------------------------------------------
| Operation                | Name  | Rows |  Cost |
-------------------------------------------------
| SELECT STATEMENT         |       |   10 | 59558 |
|  COUNT STOPKEY           |       |      |       |
|   VIEW                   |       | 1004K| 59558 |
|    SORT ORDER BY STOPKEY|       | 1004K| 59558 |
|     TABLE ACCESS FULL    | SALES | 1004K|  9246 |
-------------------------------------------------
```

Execution plan này không có pipelined order by và gần như chậm tương đương với việc abort execution từ phía client. Dùng top-N syntax vẫn tốt hơn vì database không cần materialize toàn bộ result mà chỉ cần mười row mới nhất. Cách này cần ít memory hơn đáng kể. Execution plan của Oracle cho biết optimization này bằng modifier STOPKEY trên operation SORT ORDER BY.

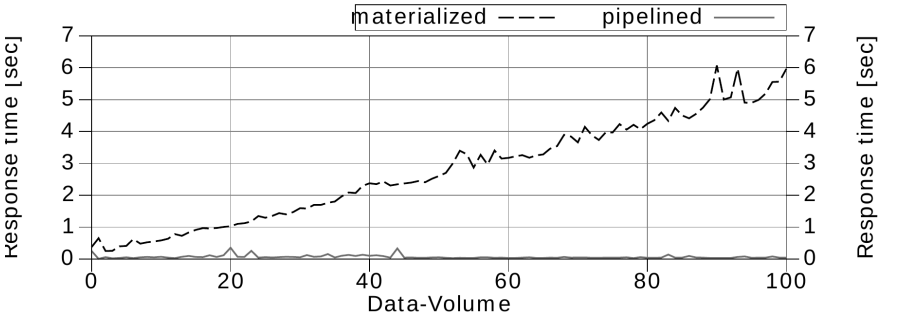
Các advantage của pipelined top-N query không chỉ là performance gain ngay lập tức mà còn là scalability tốt hơn. Nếu không dùng pipelined execution, response time của top-N query này tăng theo table size. Ngược lại, response time khi dùng pipelined execution chỉ tăng

Paging Through Results

theo số row được select. Nói cách khác, response time của pipelined top-N query luôn như nhau; nó gần như độc lập với table size. Chỉ khi B-tree depth tăng thì query mới chậm hơn một chút.

Figure 7.1 cho thấy scalability của cả hai variant khi data volume tăng. Response time tăng tuyến tính đối với execution không có pipelined order by được thể hiện rất rõ. Response time của pipelined execution vẫn giữ nguyên.

Figure 7.1. Scalability của Top-N Query



Dù response time của pipelined top-N query không phụ thuộc vào table size, nó vẫn tăng theo số row được select. Vì vậy response time sẽ tăng gấp đôi khi select số row nhiều gấp đôi. Điều này đặc biệt đáng chú ý với các query "paging" load thêm result vì các query này thường bắt đầu lại từ entry đầu tiên; chúng sẽ đọc các row đã hiển thị ở page trước rồi loại bỏ chúng trước khi cuối cùng tới được result của page thứ hai. Tuy nhiên, vấn đề này cũng có lời giải như chúng ta sẽ thấy trong section tiếp theo.

Paging Through Results

Sau khi implement pipelined top-N query để retrieve page đầu tiên một cách hiệu quả, bạn thường cũng cần một query khác để fetch các page tiếp theo. Challenge phát sinh là query đó phải skip các row từ những page trước. Có hai method khác nhau để xử lý challenge này: thứ nhất là offset method, đánh số row từ đầu và dùng filter trên row number để loại bỏ các row trước page được yêu cầu. Method thứ hai, tôi gọi là seek method, tìm entry cuối cùng của page trước và chỉ fetch các row tiếp theo.

Chapter 7: Partial Results

Các ví dụ sau trình bày offset method được dùng phổ biến hơn. Advantage chính của nó là rất dễ xử lý — đặc biệt với các database có keyword riêng cho nó (offset). Keyword này thậm chí đã được đưa vào SQL standard như một phần của fetch first extension.

MySQL

MySQL và PostgreSQL cung cấp offset clause để loại bỏ số row được chỉ định từ đầu của một top-N query. Limit clause được áp dụng sau đó.

```
SELECT *
  FROM sales
 ORDER BY sale_date DESC
 LIMIT 10 OFFSET 10;
```

Oracle Database

Oracle database cung cấp pseudo column ROWNUM, tự động đánh số row trong result set. Tuy nhiên, không thể áp dụng filter lớn hơn hoặc bằng (>=) trên pseudo-column này. Để làm được điều đó, trước hết bạn phải "materialize" row number bằng cách đổi tên column với một alias.

```
SELECT *
  FROM ( SELECT tmp.*, rownum rn
           FROM ( SELECT *
                    FROM sales
                   ORDER BY sale_date DESC
                ) tmp
          WHERE rownum <= 20
       )
 WHERE rn > 10;
```

Lưu ý việc dùng alias RN cho lower bound và pseudo column ROWNUM cho upper bound.

PostgreSQL

Fetch first extension cũng định nghĩa một offset ... rows clause. Tuy nhiên, PostgreSQL chỉ chấp nhận offset không có keyword rows. Syntax limit/offset được dùng trước đây vẫn hoạt động như trong ví dụ MySQL.

```
SELECT *
  FROM sales
 ORDER BY sale_date DESC
OFFSET 10
 FETCH NEXT 10 ROWS ONLY;
```

Paging Through Results

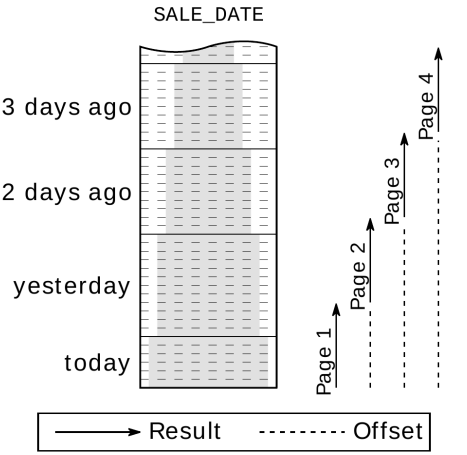
SQL Server

SQL Server không có extension "offset" cho proprietary top clause của nó nhưng đã giới thiệu fetch first extension cùng SQL Server 2012 ("Denali"). Offset clause là bắt buộc dù standard định nghĩa nó là một optional addendum.

```
SELECT *
  FROM sales
 ORDER BY sale_date DESC
OFFSET 10 ROWS
 FETCH NEXT 10 ROWS ONLY;
```

Ngoài sự đơn giản, một advantage khác của method này là bạn chỉ cần row offset để fetch một page bất kỳ. Tuy nhiên, database phải count tất cả row từ đầu cho đến khi tới page được yêu cầu. Figure 7.2 cho thấy scanned index range lớn hơn khi fetch nhiều page hơn.

Figure 7.2. Access Using the Offset Method



Điều này có hai disadvantage: (1) các page bị drift khi insert sale mới vì việc đánh số luôn được thực hiện lại từ đầu; (2) response time tăng khi browse lùi về xa hơn.

Seek method tránh được cả hai vấn đề vì nó dùng các value của page trước làm delimiter. Nghĩa là nó tìm các value phải nằm sau entry cuối cùng của page trước. Điều này có thể biểu diễn bằng một where clause đơn giản. Nói theo cách khác: seek method đơn giản là không select những value đã được hiển thị.

Chapter 7: Partial Results

Ví dụ tiếp theo trình bày seek method. Để minh họa, chúng ta giả sử mỗi ngày chỉ có một sale. Khi đó SALE_DATE là một unique key. Để select những sale phải nằm sau một date cụ thể, bạn phải dùng điều kiện less than (<) vì sort order là descending. Với ascending order, bạn phải dùng điều kiện greater than (>). Fetch first clause chỉ được dùng để giới hạn result ở mười row.

ecify the lower bound. This has a huge benefit in terms of performance
cause the database can use the SALE_DATE < ? condition for index access.
at means that the database can truly skip the rows from the previous
ges. On top of that, you will also get stable results if new rows are
erted.

vertheless, this method does not work if there is more than one sale per

Thay vì row number, bạn dùng value cuối cùng của page trước để chỉ định lower bound. Điều này mang lại benefit rất lớn về performance vì database có thể dùng condition SALE_DATE < ? để access index. Nghĩa là database thực sự có thể skip các row từ những page trước. Ngoài ra, bạn cũng nhận được result ổn định nếu có row mới được insert.

Tuy nhiên, method này không hoạt động nếu mỗi ngày có nhiều hơn một sale — như trong Figure 7.2 — vì dùng date cuối cùng của page đầu tiên ("yesterday") sẽ skip tất cả result của yesterday, không chỉ những result đã hiển thị trên page đầu. Vấn đề là order by clause không thiết lập một row sequence deterministic. Thế nhưng đó lại là prerequisite để dùng một range condition đơn giản cho page break.

Nếu không có deterministic order by clause, theo định nghĩa database không trả về một row sequence deterministic. Lý do duy nhất khiến bạn thường nhận được một row sequence nhất quán là database thường execute query theo cùng một cách. Tuy nhiên, database thực tế có thể shuffle các row có cùng SALE_DATE mà vẫn đáp ứng order by clause. Ở các release gần đây, bạn thực sự có thể nhận result theo order khác nhau mỗi lần chạy query, không phải vì database cố ý shuffle result mà vì database có thể sử dụng parallel query execution. Nghĩa là cùng một execution plan vẫn có thể tạo ra row sequence khác nhau vì các thread thực thi kết thúc theo một order không deterministic.

>  Important
>
> Paging yêu cầu một sort order deterministic.

Paging Through Results

Ngay cả khi functional specification chỉ yêu cầu sort "theo date, mới nhất trước", developer vẫn phải bảo đảm order by clause tạo ra một row sequence deterministic. Vì mục đích này, có thể chúng ta cần mở rộng order by clause bằng các column bất kỳ chỉ để bảo đảm row sequence deterministic. Nếu index được dùng cho pipelined order by có thêm column, trước tiên nên thêm chúng vào order by clause để tiếp tục dùng index này cho pipelined order by. Nếu như vậy vẫn chưa tạo ra deterministic sort order, chỉ cần thêm một hoặc nhiều unique column và mở rộng index tương ứng.

Trong ví dụ sau, chúng ta mở rộng order by clause và index bằng primary key SALE_ID để có một row sequence deterministic. Hơn nữa, chúng ta phải áp dụng logic "comes after" cho cả hai column cùng nhau để có result mong muốn:

```
CREATE INDEX sl_dtid ON sales (sale_date, sale_id);

SELECT *
  FROM sales
 WHERE (sale_date, sale_id) < (?, ?)
 ORDER BY sale_date DESC, sale_id DESC
 FETCH FIRST 10 ROWS ONLY;
```

Where clause dùng syntax "row values" ít được biết đến (xem box có tiêu đề "SQL Row Values"). Syntax này kết hợp nhiều value thành một logical unit có thể áp dụng các comparison operator thông thường. Cũng như scalar value, điều kiện less-than tương ứng với "comes after" khi sort theo descending order. Nghĩa là query chỉ xét những sale nằm sau cặp SALE_DATE, SALE_ID được cung cấp.

Dù row values syntax là một phần của SQL standard, chỉ có một vài database hỗ trợ nó. SQL Server 2012 ("Denali") hoàn toàn không hỗ trợ row values. Oracle database về nguyên tắc có hỗ trợ row values, nhưng không thể áp dụng range operator trên chúng (ORA-01796). MySQL evaluate row value expression chính xác nhưng không thể

dùng chúng làm access predicate trong index access. Tuy nhiên, PostgreSQL hỗ trợ row value syntax và dùng chúng để access index nếu có index tương ứng.

Chapter 7: Partial Results

Tuy nhiên, vẫn có thể dùng một biến thể xấp xỉ của seek method với những database không hỗ trợ row values đúng cách — dù cách xấp xỉ này không elegant và hiệu quả bằng row values trong PostgreSQL. Với cách xấp xỉ này, chúng ta phải dùng các comparison "thông thường" để biểu diễn logic cần thiết như trong Oracle example sau:

```
SELECT *
  FROM ( SELECT *
           FROM sales
          WHERE sale_date <= ?
            AND NOT (sale_date = ? AND sale_id >= ?)
          ORDER BY sale_date DESC, sale_id DESC
       )
 WHERE rownum <= 10;
```

Where clause gồm hai phần. Phần thứ nhất chỉ xét SALE_DATE và dùng điều kiện less than or equal to (<=) — nó select nhiều row hơn mức cần thiết. Phần này đủ đơn giản để mọi database có thể dùng nó nhằm access index. Phần thứ hai loại bỏ các row dư đã được hiển thị trên page trước. Box có tiêu đề "Indexing Equivalent Logic" giải thích vì sao where clause được viết theo cách này.

Execution plan cho thấy database dùng phần thứ nhất của where clause làm access predicate.

```
-------------------------------------------------------------------
Id | Operation                          | Name     |  Rows | Cost |
-------------------------------------------------------------------
 0 | SELECT STATEMENT                   |          |    10 |    4 |
*1 |  COUNT STOPKEY                     |          |       |      |
 2 |   VIEW                             |          |    10 |    4 |
 3 |    TABLE ACCESS BY INDEX ROWID     | SALES    | 50218 |    4 |
*4 |     INDEX RANGE SCAN DESCENDING    | SL_DTIT  |     2 |    3 |
-------------------------------------------------------------------

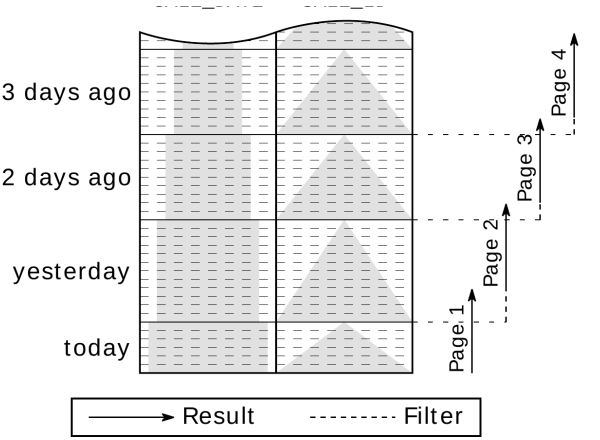
Predicate Information (identified by operation id):
---------------------------------------------------
   1 - filter(ROWNUM<=10)
   4 - access("SALE_DATE"<=:SALE_DATE)
       filter("SALE_DATE"<>:SALE_DATE
           OR "SALE_ID"<TO_NUMBER(:SALE_ID))
```

Access predicate trên SALE_DATE cho phép database skip những ngày đã được hiển thị đầy đủ ở các page trước. Phần thứ hai của where clause chỉ là filter predicate. Nghĩa là database inspect

Paging Through Results

một vài entry từ page trước lần nữa, nhưng loại bỏ chúng ngay lập tức. Figure 7.3 cho thấy access path tương ứng.

Figure 7.3. Access Using the Seek Method



SQL Row Values

Ngoài scalar value thông thường, SQL standard còn định nghĩa row value constructor, tức là chúng “Specify an ordered set of values to be constructed into a row or partial row” [SQL:92, §7.1: <row value constructor>]. Về syntax, row value là các list trong dấu ngoặc. Syntax này được biết đến nhiều nhất khi dùng trong insert statement.

Tuy nhiên, dùng row value constructor trong where clause ít phổ biến hơn nhưng hoàn toàn hợp lệ. SQL standard thực tế định nghĩa tất cả comparison operator cho row value constructor. Chẳng hạn, định nghĩa cho phép less than là như sau:

"Rx < Ry" is true if and only if RXi = RYi for all i < n and

RXn < RYn for some n.

—SQL:92, §8.2.7.2

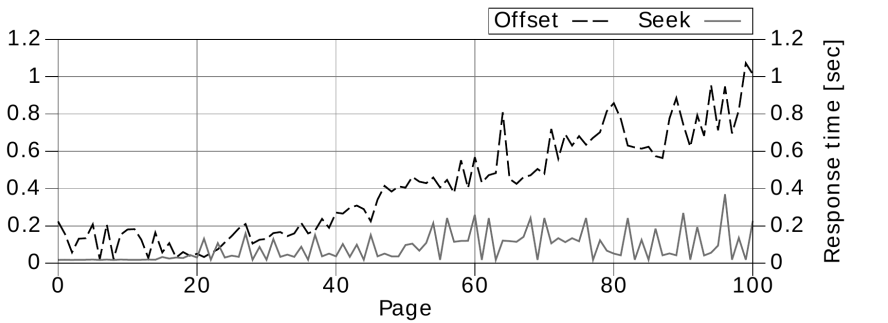
Trong đó i và n là các positional index trong list. Nghĩa là row value RX nhỏ hơn RY nếu bất kỳ value RXn nào nhỏ hơn RYn tương ứng và tất cả cặp value đứng trước đều bằng nhau (RXi = RYi; for i<n).

Định nghĩa này khiến expression RX < RY đồng nghĩa với “RX sorts before RY”, chính là logic chúng ta cần cho seek method.

Chapter 7: Partial Results

Figure 7.4 so sánh các đặc điểm performance của offset method và seek method. Độ chính xác của phép đo không đủ để nhìn thấy khác biệt ở phía bên trái của chart, nhưng khác biệt hiện rõ từ khoảng page 20 trở đi.

Figure 7.4. Scalability khi Fetch Page Tiếp Theo



Tất nhiên seek method cũng có drawback, trong đó khó xử lý là drawback quan trọng nhất. Bạn không chỉ phải viết where clause rất cẩn thận — bạn còn không thể fetch các page bất kỳ. Hơn nữa, để đổi direction browse, bạn phải đảo ngược tất cả comparison và sort operation. Chính hai chức năng này — skip page và browse ngược — không cần thiết khi dùng cơ chế infinite scrolling cho user interface.

Paging Through Results

> Indexing Equivalent Logic
>
> Một logical condition luôn có thể được biểu diễn theo nhiều cách khác nhau. Chẳng hạn, bạn cũng có thể implement logic skip ở trên như sau:

```
WHERE (
         (sale_date < ?)
       OR
         (sale_date = ? AND sale_id < ?)
      )
```



Biến thể này chỉ dùng các điều kiện bao hàm và có lẽ dễ hiểu hơn — ít nhất là với con người. Database có cách nhìn khác. Chúng không nhận ra rằng where clause select tất cả row bắt đầu từ cặp SALE_DATE/SALE_ID tương ứng — với điều kiện SALE_DATE giống nhau ở cả hai nhánh. Ít nhất chúng ta có thể kỳ vọng optimizer "factor condition SALE_DATE <= ? out" khỏi hai nhánh OR, nhưng không database nào cung cấp service này.

Tuy vậy, chúng ta có thể tự thêm condition dư thừa này — dù nó không làm tăng readability:

```
WHERE sale_date <= ?
  AND (
         (sale_date < ?)
       OR
         (sale_date = ? AND sale_id < ?)
      )
```



May mắn là tất cả database đều có thể dùng phần này của where clause làm access predicate. Tuy nhiên, clause đó còn khó nắm bắt hơn cả logic xấp xỉ được trình bày ở trên. Hơn nữa, logic ban đầu tránh được rủi ro phần "không cần thiết" (dư thừa) vô tình bị xóa khỏi where clause về sau.

Chapter 7: Partial Results

Dùng Window Function cho Pagination

Window function cung cấp thêm một cách để implement pagination trong SQL.

Đây là một method linh hoạt và trên hết là tuân thủ SQL standard. Tuy nhiên, chỉ SQL Server và Oracle database mới có thể dùng chúng cho pipelined top-N query. PostgreSQL không abort index scan sau khi fetch đủ row nên execute các query này rất kém hiệu quả. MySQL hoàn toàn không hỗ trợ window function.

Ví dụ sau dùng window function ROW_NUMBER cho một pagination query:

```
SELECT *
  FROM ( SELECT sales.*
              , ROW_NUMBER() OVER (ORDER BY sale_date DESC
                                             , sale_id   DESC) rn
           FROM sales
       ) tmp
 WHERE rn between 11 and 20
 ORDER BY sale_date DESC, sale_id DESC;
```

Function ROW_NUMBER đánh số các row theo sort order được định nghĩa trong over clause. Outer where clause dùng cách đánh số này để giới hạn result ở page thứ hai (row 11 đến 20).

Oracle database nhận diện abort condition và dùng index trên SALE_DATE và SALE_ID để tạo behavior pipelined top-N:

```
-------------------------------------------------------------------
|Id | Operation                       | Name    | Rows |  Cost |
-------------------------------------------------------------------
| 0 | SELECT STATEMENT                |         | 1004K| 36877 |
|*1 |  VIEW                           |         | 1004K| 36877 |
|*2 |   WINDOW NOSORT STOPKEY         |         | 1004K| 36877 |
| 3 |    TABLE ACCESS BY INDEX ROWID  | SALES   | 1004K| 36877 |
| 4 |     INDEX FULL SCAN DESCENDING  | SL_DTID | 1004K|  2955 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
1 - filter("RN">=11 AND "RN"<=20)
2 - filter(ROW_NUMBER() OVER (
           ORDER BY "SALE_DATE" DESC, "SALE_ID" DESC )<=20)
```

Dùng Window Function cho Pagination

Operation WINDOW NOSORT STOPKEY cho biết không có sort operation (NOSORT) và database abort execution khi đạt upper threshold (STOPKEY). Vì các operation bị abort được execute theo pipelined manner, query này có hiệu quả tương đương offset method đã giải thích ở section trước.

Tuy nhiên, strength của window function không nằm ở pagination mà ở analytical calculation. Nếu chưa từng dùng window function, bạn chắc chắn nên dành vài giờ để đọc documentation tương ứng.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Chapter 8

Modifying Data

Cho đến nay chúng ta mới chỉ bàn về query performance, nhưng SQL không chỉ có query. SQL cũng hỗ trợ data manipulation. Các command tương ứng — insert, delete và update — tạo thành phần gọi là “data manipulation language” (DML), một phần của SQL standard. Performance của các command này phần lớn bị index ảnh hưởng theo hướng tiêu cực.

Index là redundancy thuần túy. Nó chỉ chứa data cũng được lưu trong table. Trong write operation, database phải giữ các redundancy này nhất quán. Cụ thể, insert, delete và update không chỉ tác động đến table mà còn tác động đến các index giữ bản sao của data bị ảnh hưởng.

Insert

Số lượng index trên một table là factor chi phối mạnh nhất đối với insert performance. Table càng có nhiều index thì execution càng chậm. Insert statement là operation duy nhất không thể trực tiếp hưởng lợi từ indexing vì nó không có where clause.

Thêm một row mới vào table gồm nhiều bước. Trước tiên database phải tìm nơi lưu row. Với một regular heap table — không có row order cụ thể — database có thể chọn bất kỳ table block nào còn đủ free space. Đây là process rất đơn giản và nhanh, phần lớn được execute trong main memory. Sau đó database chỉ cần thêm entry mới vào data block tương ứng.

Nếu table có index, database phải bảo đảm entry mới cũng được tìm thấy thông qua các index đó. Vì vậy nó phải thêm entry mới vào từng index của table. Do đó, số lượng index là một multiplier của

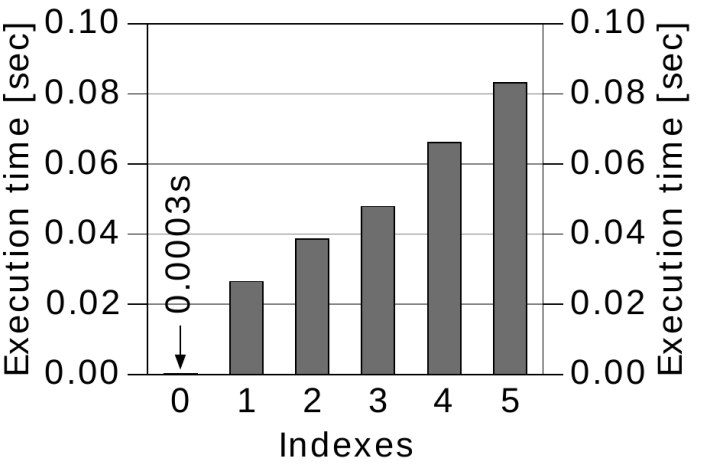
cost của insert statement.

Hơn nữa, thêm một entry vào index đắt hơn nhiều so với chèn entry vào heap structure vì database phải duy trì index order và tree balance. Điều đó nghĩa là entry mới không thể ghi vào bất kỳ block nào — nó thuộc về một leaf node cụ thể. Dù database dùng chính index tree để tìm leaf node đúng, nó vẫn phải đọc một vài index block để traverse tree.

Sau khi xác định đúng leaf node, database kiểm tra node còn đủ free space hay không. Nếu không, database split leaf node và phân phối các entry giữa node cũ và node mới. Process này cũng tác động đến reference trong branch node tương ứng vì reference đó cũng phải được duplicate. Branch node cũng có thể hết space nên có thể phải split tiếp. Trong trường hợp xấu nhất, database phải split tất cả node cho đến root node. Đây là trường hợp duy nhất khiến tree có thêm một layer và tăng depth.

Sau cùng, index maintenance là phần tốn kém nhất của insert operation. Điều này cũng thấy rõ trong Figure 8.1, “Insert Performance by Number of Indexes”: execution time gần như không đáng kể nếu table không có index. Tuy vậy, chỉ cần thêm một index là đủ làm execute time tăng gấp một trăm lần. Mỗi index thêm vào lại làm execution chậm hơn.

Figure 8.1. Insert Performance by Number of Indexes



Insert

> Note
>
> Index đầu tiên tạo ra khác biệt lớn nhất.

Để optimize insert performance, việc giữ số lượng index ở mức nhỏ là rất quan trọng.

> Tip
>
> Hãy dùng index có chủ đích và tiết chế, đồng thời tránh các index redundant bất cứ khi nào có thể. Điều này cũng có lợi cho delete và update statement.

Nếu chỉ xét insert statement, tốt nhất là tránh index hoàn toàn — cách này cho insert performance tốt hơn rất nhiều. Tuy nhiên, table không có index khá thiếu thực tế trong các application thực tế.

Thông thường bạn muốn retrieve data đã lưu nên cần index để cải thiện query speed. Ngay cả write-only log table cũng thường có primary key và index tương ứng.

Dù vậy, performance khi không có index tốt đến mức việc tạm thời drop toàn bộ index trong lúc load một lượng data lớn có thể hợp lý — với điều kiện các index đó không cần cho bất kỳ SQL statement nào khác trong thời gian này. Cách này có thể tạo ra speed-up rất lớn, thể hiện trên chart và thực tế là một practice phổ biến trong data warehouse.

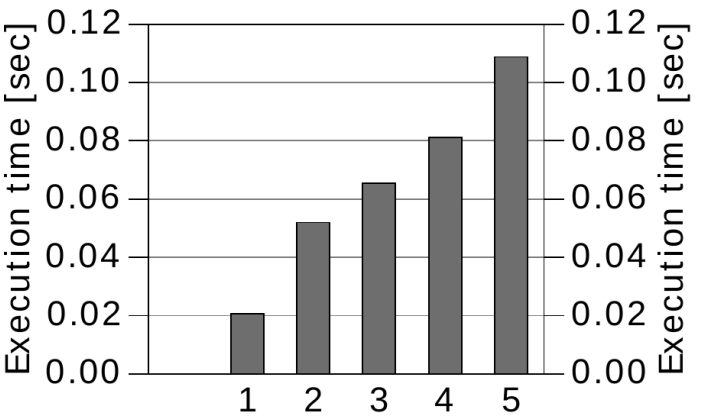
> 💡 Think about it
>
> Figure 8.1 sẽ thay đổi thế nào nếu dùng index organized table hoặc clustered index?
>
> Có cách gián tiếp nào để insert statement có thể hưởng lợi từ indexing không? Nghĩa là một index bổ sung có thể làm insert statement nhanh hơn không?

Delete

Không giống insert statement, delete statement có where clause và có thể dùng mọi method được mô tả trong Chapter 2, “The Where Clause”, để hưởng lợi trực tiếp từ index. Thực tế, delete statement hoạt động giống một select rồi thêm một bước để delete các row đã xác định.

Việc xóa thực tế một row tương tự process insert một row mới — đặc biệt là việc loại bỏ các reference khỏi index và các hoạt động để giữ index tree cân bằng. Vì vậy performance chart trong Figure 8.2 rất giống chart đã trình bày cho insert.

Figure 8.2. Delete Performance by Number of Indexes



Về lý thuyết, chúng ta kỳ vọng delete performance tốt nhất với table không có index — giống như insert. Tuy nhiên, nếu không có index, database phải đọc toàn bộ table để tìm các row cần delete. Nghĩa là việc delete row nhanh nhưng việc tìm row rất chậm. Vì vậy trường hợp này không được thể hiện trong Figure 8.2.

Dù vậy, execute delete statement không có index vẫn có thể hợp lý, cũng giống như execute select statement không có index khi nó trả về phần lớn table.

> Tip
>
> Ngay cả delete và update statement cũng có execution plan.

Update

Delete statement không có where clause là một ví dụ rõ ràng trong đó database không thể dùng index, dù đây là trường hợp đặc biệt có SQL command riêng: truncate table. Command này có effect giống delete không có where, ngoại trừ việc nó delete toàn bộ row trong

một lần. Nó rất nhanh nhưng có hai side effect quan trọng: (1) nó thực hiện implicit commit (ngoại lệ: PostgreSQL); (2) nó không execute trigger nào.

> 💡 Side Effects of MVCC
>
> Multiversion concurrency control (MVCC) là một database mechanism cho phép concurrent data access không blocking và cung cấp transaction view nhất quán. Tuy nhiên, các implementation khác nhau giữa các database và thậm chí có thể ảnh hưởng đáng kể đến performance.
>
> Ví dụ, PostgreSQL database chỉ giữ version information (=visibility information) ở table level: xóa một row chỉ set "deleted" flag trong table block. Vì vậy delete performance của PostgreSQL không phụ thuộc vào số lượng index trên table. Việc xóa vật lý table row và index maintenance liên quan chỉ được thực hiện trong process VACCUM.

Update

Update statement phải relocate các index entry đã thay đổi để duy trì index order. Vì vậy database phải remove entry cũ và add entry mới ở vị trí mới. Response time về cơ bản giống tổng response time của delete và insert statement tương ứng.

Update performance, cũng như insert và delete, phụ thuộc vào số lượng index trên table. Điểm khác biệt duy nhất là update statement không nhất thiết tác động đến tất cả column vì chúng thường chỉ modify một vài column được chọn. Do đó update statement không nhất thiết tác động đến tất cả index trên table mà chỉ tác động đến những index chứa các column được update.

Figure 8.3. Update Performance by Indexes and Column Count

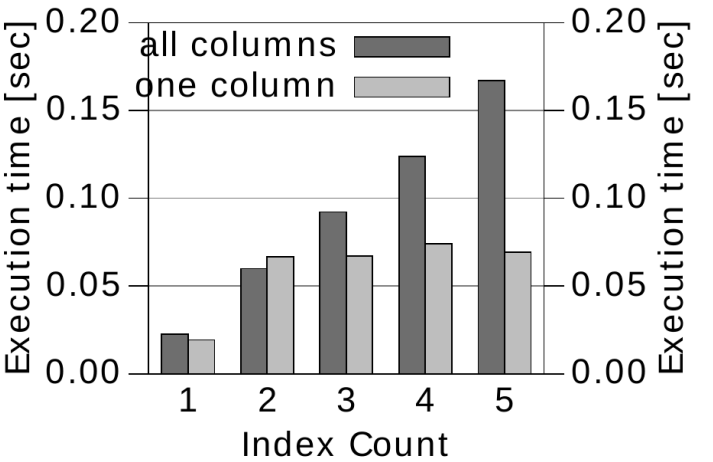


Figure 8.3 cho thấy response time của hai update statement: một statement set tất cả column và tác động đến tất cả index, sau đó là một statement thứ hai chỉ update một column nên chỉ tác động đến một index.

Update trên tất cả column cho thấy cùng pattern đã quan sát ở các section trước: response time tăng theo mỗi index bổ sung. Response time của update statement chỉ tác động đến một index không tăng nhiều vì nó giữ nguyên phần lớn index.

Để optimize update performance, bạn phải cẩn thận chỉ update những column đã thay đổi. Điều này hiển nhiên nếu bạn tự viết update statement. Tuy nhiên, ORM tool có thể generate update statement set tất cả column mỗi lần. Chẳng hạn, Hibernate làm vậy khi disable dynamic-update mode. Từ version 4.0, mode này được enable mặc định.

Khi dùng ORM tool, một practice tốt là thỉnh thoảng enable query logging trong development environment để kiểm tra các SQL statement được generate. Tip có tiêu đề “Enabling SQL Logging” ở page 95 có phần overview ngắn về cách enable SQL logging trong một số ORM tool được dùng phổ biến.

>  Think about it
>
> Bạn có nghĩ ra trường hợp insert hoặc delete statement không tác động đến tất cả index của một table không?

Appendix A

Execution Plans

Trước khi database có thể execute một SQL statement, optimizer phải tạo execution plan cho statement đó. Sau đó database execute plan này theo từng bước. Ở khía cạnh này, optimizer rất giống compiler vì nó translate source code (SQL statement) thành một executable program (execution plan).

Execution plan là nơi đầu tiên cần xem khi tìm nguyên nhân của statement chậm. Các section sau giải thích cách lấy và đọc execution plan để optimize performance trong nhiều database khác nhau.

Contents

Oracle Database 166
- Getting an Execution Plan 166
- Operations 167
- Distinguishing Access and Filter-Predicates 170

PostgreSQL 172
- Getting an Execution Plan 172
- Operations 174
- Distinguishing Access and Filter-Predicates 177

SQL Server 180
- Getting an Execution Plan 180
- Operations 182
- Distinguishing Access and Filter-Predicates 185

MySQL 188
- Getting an Execution Plan 188
- Operations 188
- Distinguishing Access and Filter-Predicates 190

Oracle Database

Hầu hết development environment (IDE) đều có thể hiển thị execution plan rất dễ dàng nhưng dùng những cách format rất khác nhau trên màn hình. Method được mô tả trong section này trả về execution plan như trong toàn bộ cuốn sách và chỉ yêu cầu Oracle database release 9iR2 hoặc mới hơn.

Getting an Execution Plan

Xem execution plan trong Oracle database gồm hai bước:

1. explain plan for — lưu execution plan vào PLAN_TABLE.

2. Format và hiển thị execution plan.

Creating and Saving an Execution Plan

Để tạo execution plan, bạn chỉ cần thêm explain plan for vào trước SQL statement tương ứng:

```
EXPLAIN PLAN FOR select * from dual;
```

Bạn có thể execute explain plan for command trong bất kỳ development environment hoặc SQL*Plus nào. Tuy nhiên, command này không hiển thị plan mà lưu plan vào table tên PLAN_TABLE. Từ release 10g, table này tự động có sẵn dưới dạng global temporary table. Với các release trước, bạn phải tạo table trong từng schema khi cần. Hãy yêu cầu database administrator tạo giúp hoặc cung cấp create table statement từ Oracle database installation:

```
$ORACLE_HOME/rdbms/admin/utlxplan.sql
```

Bạn có thể execute statement này trong bất kỳ schema nào để tạo PLAN_TABLE trong schema đó.

> Warning
>
> Explain plan for command không nhất thiết tạo cùng execution plan với execution plan được tạo khi execute statement.

Showing Execution Plans

Package DBMS_XPLAN được giới thiệu từ release 9iR2 và có thể format, hiển thị execution plan từ PLAN_TABLE. Ví dụ sau cho biết cách hiển thị execution plan gần nhất đã được explain trong database session hiện tại:

```
select * from table(dbms_xplan.display);
```

Một lần nữa, nếu statement đó không hoạt động ngay, bạn nên nhờ DBA hỗ trợ.

Query sẽ hiển thị execution plan như trong cuốn sách:

```
-----------------------------------------------------------------
| Id | Operation         | Name | Rows | Bytes | Cost (%CPU)|.
-----------------------------------------------------------------
|  0 | SELECT STATEMENT  |      |    1 |     2 |     2   (0)|.
|  1 |  TABLE ACCESS FULL| DUAL |    1 |     2 |     2   (0)|.
-----------------------------------------------------------------
```

Một số column trong execution plan này đã được bỏ đi để vừa trang hơn trong sách.

Operations

Index and Table Access

INDEX UNIQUE SCAN

INDEX UNIQUE SCAN chỉ thực hiện B-tree traversal. Database dùng operation này khi một unique constraint bảo đảm search criteria match không quá một entry. Xem thêm Chapter 1, “Anatomy of an Index”.

INDEX RANGE SCAN

INDEX RANGE SCAN thực hiện B-tree traversal và đi theo leaf node chain để tìm tất cả matching entry. Xem thêm Chapter 1, “Anatomy of an Index”.

Các index filter predicate thường gây ra performance problem cho INDEX RANGE SCAN. Section tiếp theo giải thích cách nhận diện chúng.

INDEX FULL SCAN

Đọc toàn bộ index — tất cả row — theo index order. Tùy system statistics, database có thể thực hiện operation này nếu cần tất cả row theo index order — chẳng hạn vì có order by clause tương ứng. Thay vào đó, optimizer cũng có thể dùng INDEX FAST FULL SCAN và thực hiện thêm một sort operation. Xem Chapter 6, “Sorting and Grouping”.

INDEX FAST FULL SCAN

Đọc toàn bộ index — tất cả row — theo cách chúng được lưu trên disk. Operation này thường được thực hiện thay cho full table scan nếu mọi column cần thiết đều có trong index. Tương tự TABLE ACCESS FULL, INDEX FAST FULL SCAN có thể hưởng lợi từ multi-block read operation. Xem Chapter 5, “Clustering Data”.

TABLE ACCESS BY INDEX ROWID

Retrieve một row từ table bằng ROWID lấy được từ index lookup ngay trước đó. Xem thêm Chapter 1, “Anatomy of an Index”.

TABLE ACCESS FULL

Còn được gọi là full table scan. Đọc toàn bộ table — tất cả row và column — theo cách chúng được lưu trên disk. Dù multi-block read operation cải thiện đáng kể tốc độ full table scan, đây vẫn là một trong những operation tốn kém nhất. Ngoài IO rate cao, full table scan phải kiểm tra mọi table row nên cũng có thể tiêu tốn nhiều CPU time. Xem thêm “Full Table Scan” ở page 13.

Joins

Nhìn chung, join operation chỉ xử lý hai table mỗi lần. Nếu query có nhiều join hơn, chúng được execute tuần tự: trước tiên là hai table, sau đó là intermediate result với table tiếp theo. Trong ngữ cảnh join, từ “table” vì vậy cũng có thể có nghĩa là “intermediate result”.

NESTED LOOPS JOIN

Join hai table bằng cách lấy result từ một table và query table còn lại cho từng row của table thứ nhất. Xem thêm “Nested Loops” ở page 92.

HASH JOIN

Hash join load candidate record từ một phía của join vào hash table, sau đó probe hash table cho từng row từ phía còn lại của join. Xem thêm “Hash Join” ở page 101.

MERGE JOIN

Merge join kết hợp hai sorted list như một zipper. Cả hai phía của join phải được presort. Xem thêm “Sort Merge” ở page 109.

Sorting and Grouping

SORT ORDER BY

Sort result theo order by clause. Operation này cần lượng memory lớn để materialize intermediate result (không pipelined). Xem thêm “Indexing Order By” ở page 130.

SORT ORDER BY STOPKEY

Sort một subset của result theo order by clause. Dùng cho top-N query nếu pipelined execution không thể thực hiện. Xem thêm “Querying Top-N Rows” ở page 143.

SORT GROUP BY

Sort result set theo các group by column và aggregate result đã sort ở bước thứ hai. Operation này cần lượng memory lớn để materialize intermediate result set (không pipelined). Xem thêm “Indexing Group By” ở page 139.

SORT GROUP BY NOSORT

Aggregate một presorted set theo group by clause. Operation này không buffer intermediate result: nó được execute theo pipelined manner. Xem thêm “Indexing Group By” ở page 139.

HASH GROUP BY

Group result bằng hash table. Operation này cần lượng memory lớn để materialize intermediate result set (không pipelined). Output không được order theo cách có ý nghĩa nào. Xem thêm “Indexing Group By” ở page 139.

Top-N Queries

Hiệu quả của top-N query phụ thuộc vào execution mode của các operation bên dưới. Chúng rất kém hiệu quả khi abort các nonpipelined operation như SORT ORDER BY.

COUNT STOPKEY

Abort các operation bên dưới khi đã fetch đủ số row mong muốn. Xem thêm section có tên “Querying Top-N Rows”.

WINDOW NOSORT STOPKEY

Dùng window function (over clause) để abort execution khi đã fetch đủ số row mong muốn. Xem thêm “Using Window Functions for Pagination” ở page 156.

Distinguishing Access and Filter-Predicates

Oracle database dùng ba method khác nhau để apply where clause (predicate):

Access predicate (“access”)

Access predicate thể hiện start và stop condition của leaf node traversal.

Index filter predicate (“filter” cho index operation)

Index filter predicate chỉ được apply trong lúc leaf node traversal. Chúng không đóng góp vào start và stop condition và không thu hẹp scanned range.

Table level filter predicate (“filter” cho table operation)

Predicate trên các column không thuộc index được evaluate ở table level. Để làm vậy, database trước hết phải load row từ table.

Execution plan được tạo bằng DBMS_XPLAN utility (xem “Getting an Execution Plan” ở page 166) cho biết index usage trong section “Predicate Information” bên dưới tabular execution plan:

```
-----------------------------------------------------
| Id | Operation         | Name       | Rows  | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT  |            |     1 | 1445 |
|  1 |  SORT AGGREGATE   |            |     1 |      |
|* 2 |   INDEX RANGE SCAN| SCALE_SLOW |  4485 | 1445 |
-----------------------------------------------------

Predicate Information (identified by operation id):
   2 - access("SECTION"=:A AND "ID2"=:B)
       filter("ID2"=:B)
```

Numbering của predicate information tham chiếu đến column “Id” của execution plan. Ở đó database cũng hiển thị dấu hoa thị để đánh dấu các operation có predicate information.

Ví dụ này, lấy từ chapter “Performance and Scalability”, cho thấy một INDEX RANGE SCAN có cả access và filter predicate. Oracle database có đặc điểm là cũng hiển thị một số filter predicate như access predicate — chẳng hạn ID2=:B trong execution plan ở trên.

> 💡 Important
>
> Nếu một condition xuất hiện dưới dạng filter predicate thì nó là filter predicate — không quan trọng việc nó cũng được hiển thị dưới dạng access predicate.

Điều này nghĩa là INDEX RANGE SCAN scan toàn bộ range cho condition "SECTION"=:A và apply filter "ID2"=:B trên từng row.

Filter predicate ở table level được hiển thị trên table access tương ứng, chẳng hạn TABLE ACCESS BY INDEX ROWID hoặc TABLE ACCESS FULL.

PostgreSQL

Các method được mô tả trong section này áp dụng cho PostgreSQL 8.0 trở lên.

Getting an Execution Plan

PostgreSQL execution plan được lấy bằng cách đặt explain trước một SQL statement. Tuy nhiên có một limitation quan trọng: SQL statement có bind parameter (ví dụ $1, $2, v.v.) không thể được explain theo cách này — trước hết chúng phải được prepare:

```
PREPARE stmt(int) AS SELECT $1;
```

Lưu ý PostgreSQL dùng "$n" cho bind parameter. Database abstraction layer của bạn có thể ẩn chi tiết này để bạn dùng question mark như SQL standard định nghĩa.

Prepared statement có thể được explain như sau:

```
EXPLAIN EXECUTE stmt(1);
```

Cho đến PostgreSQL 9.1, execution plan đã được tạo tại prepare call nên không thể xét actual value được cung cấp cùng execute. Từ PostgreSQL 9.2, việc tạo execution plan được trì hoãn đến lúc execution nên có thể xét actual value của bind parameter.

>  Note
>
> Statement không có bind parameter có thể được explain trực tiếp:

```
EXPLAIN SELECT 1;
```

Trong trường hợp này, optimizer luôn xét actual value trong lúc query planning. Nếu bạn dùng PostgreSQL 9.1 hoặc cũ hơn và bind parameter trong program, bạn cũng nên dùng explain với bind parameter để lấy cùng execution plan.

Output của explain plan như sau:

```
                QUERY PLAN
-----------------------------------------
 Result  (cost=0.00..0.01 rows=1 width=0)
```

Output chứa thông tin tương tự execution plan của Oracle được trình bày trong sách: operation name (“Result”), cost tương ứng, estimated row count và expected row width.

Lưu ý PostgreSQL hiển thị hai cost value. Giá trị thứ nhất là startup cost, giá trị thứ hai là total cost cho execution nếu retrieve toàn bộ row. Execution plan của Oracle chỉ hiển thị giá trị thứ hai.

PostgreSQL explain command có hai option. Option VERBOSE cung cấp thêm thông tin như fully qualified table name — VERBOSE thường không có nhiều giá trị.

Option explain thứ hai là ANALYZE. Dù được dùng rộng rãi, tôi khuyên bạn không nên hình thành thói quen tự động dùng nó vì nó thực sự execute statement. Điều đó phần lớn vô hại với select statement nhưng sẽ modify data khi dùng cho insert, update hoặc delete. Để tránh rủi ro vô tình modify data, bạn có thể đặt nó trong transaction rồi thực hiện rollback sau đó.

ANALYZE option execute statement và ghi lại actual timing, row count. Thông tin này có giá trị khi tìm nguyên nhân của cardinality estimate không chính xác (row count estimate):

```
BEGIN;
EXPLAIN ANALYZE EXECUTE stmt(1);

                    QUERY PLAN
--------------------------------------------------
 Result  (cost=0.00..0.01 rows=1 width=0)
         (actual time=0.002..0.002 rows=1 loops=1)
 Total runtime: 0.020 ms

ROLLBACK;
```

Lưu ý plan được format để vừa trang hơn. PostgreSQL in các value "actual" trên cùng dòng với value estimate.

> 💡 Warning
>
> explain analyze execute statement được explain, kể cả khi statement là insert, update hoặc delete.

Row count là value duy nhất được hiển thị ở cả hai phần — estimated figure và actual figure. Điều đó cho phép bạn nhanh chóng tìm các cardinality estimate sai.

Cuối cùng, prepared statement phải được close lại:

```
DEALLOCATE stmt;
```

Operations

Index and Table Access

Seq Scan

Operation Seq Scan scan toàn bộ relation (table) theo cách được lưu trên disk (giống TABLE ACCESS FULL).

Index Scan

Index Scan thực hiện B-tree traversal, đi qua các leaf node để tìm mọi matching entry và fetch table data tương ứng. Nó giống INDEX RANGE SCAN theo sau bởi TABLE ACCESS BY INDEX ROWID operation. Xem thêm Chapter 1, “Anatomy of an Index”.

Các index filter predicate thường gây performance problem cho Index Scan. Section tiếp theo giải thích cách nhận diện chúng.

Index Only Scan (since PostgreSQL 9.2)

Index Only Scan thực hiện B-tree traversal và đi qua leaf node để tìm mọi matching entry. Không cần table access vì index có đủ column để đáp ứng query (ngoại lệ: MVCC visibility information). Xem thêm “Index-Only Scan” ở page 116.

Bitmap Index Scan / Bitmap Heap Scan / Recheck Cond

Bài post của Tom Lane trên PostgreSQL performance mailing list rất rõ ràng và cô đọng.

Một Index Scan thông thường fetch từng tuple-pointer một từ index và ngay lập tức truy cập tuple đó trong table. Bitmap scan fetch tất cả tuple-pointer từ index trong một lần, sort chúng bằng cấu trúc data “bitmap” trong memory, rồi truy cập table tuple theo physical tuple-location order.

—Tom Lane[1]

Join Operations

Nhìn chung, join operation chỉ xử lý hai table mỗi lần. Nếu query có nhiều join hơn, chúng được execute tuần tự: trước tiên là hai table, sau đó là intermediate result với table tiếp theo. Trong ngữ cảnh join, từ “table” vì vậy cũng có thể có nghĩa là “intermediate result”.

Nested Loops

Join hai table bằng cách lấy result từ một table và query table còn lại cho từng row của table thứ nhất. Xem thêm “Nested Loops” ở page 92.

Hash Join / Hash

Hash join load candidate record từ một phía của join vào hash table (được đánh dấu bằng Hash trong plan), sau đó probe hash table cho từng record từ phía còn lại của join. Xem thêm “Hash Join” ở page 101.

Merge Join

(Sort) merge join kết hợp hai sorted list như một zipper. Cả hai phía của join phải được presort. Xem thêm “Sort Merge” ở page 109.

1

http://archives.postgresql.org/pgsql-performance/2005-12/msg00623.php

Sorting and Grouping

Sort / Sort Key

Sort set theo các column được nêu trong Sort Key. Operation Sort cần lượng memory lớn để materialize intermediate result (không pipelined). Xem thêm “Indexing Order By” ở page 130.

GroupAggregate

Aggregate một presorted set theo group by clause. Operation này không buffer lượng data lớn (pipelined). Xem thêm “Indexing Group By” ở page 139.

HashAggregate

Dùng temporary hash table để group record. Operation HashAggregate không cần presorted data set; thay vào đó nó dùng lượng memory lớn để materialize intermediate result (không

pipelined). Output không được order theo cách có ý nghĩa nào. Xem thêm “Indexing Group By” ở page 139.

Top-N Queries

Limit

Abort các operation bên dưới khi đã fetch đủ số row mong muốn. Xem thêm “Querying Top-N Rows” ở page 143.

Hiệu quả của top-N query phụ thuộc vào execution mode của các operation bên dưới. Nó rất kém hiệu quả khi abort các nonpipelined operation như Sort.

WindowAgg

Cho biết đang dùng window function. Xem thêm “Using Window Functions for Pagination” ở page 156.

>  Caution
>
> PostgreSQL không thể execute pipelined top-N query khi dùng window function.

PostgreSQL: Phân biệt Access Predicate và Filter Predicate

Phân biệt Access Predicate và Filter Predicate

Database PostgreSQL dùng ba method khác nhau để apply where clause (predicate):

Access Predicate (“Index Cond”)

Access predicate thể hiện start và stop condition của leaf node traversal.

Index Filter Predicate (“Index Cond”)

Index filter predicate chỉ được apply trong lúc leaf node traversal. Chúng không đóng góp vào start và stop condition, đồng thời không thu hẹp scanned range.

Table level filter predicate (“Filter”)

Predicate trên các column không thuộc index được evaluate ở table level. Để làm vậy, database trước hết phải load row từ heap table.

PostgreSQL execution plan không hiển thị riêng index access predicate và index filter predicate — cả hai đều xuất hiện dưới dạng “Index Cond”. Vì vậy, phải đối chiếu execution plan với index definition để phân biệt access predicate và index filter predicate.

> Note
>
> PostgreSQL explain plan không cung cấp đủ thông tin để tìm index filter predicate.

Các predicate được hiển thị dưới dạng “Filter” luôn là table level filter predicate — kể cả khi chúng được hiển thị cho một Index Scan operation.

Hãy xem ví dụ sau, vốn xuất hiện lần đầu trong chapter “Performance and Scalability”:

```
CREATE TABLE scale_data (
   section NUMERIC NOT NULL,
   id1     NUMERIC NOT NULL,
   id2     NUMERIC NOT NULL
);
CREATE INDEX scale_data_key ON scale_data(section, id1);
```

Select sau filter trên column ID2, column không được đưa vào index:

```
PREPARE stmt(int) AS SELECT count(*)
                       FROM scale_data
                      WHERE section = 1
                        AND id2 = $1;
EXPLAIN EXECUTE stmt(1);

                       QUERY PLAN
-------------------------------------------------------
Aggregate  (cost=529346.31..529346.32 rows=1 width=0)
  Output: count(*)
  -> Index Scan using scale_data_key on scale_data
     (cost=0.00..529338.83 rows=2989 width=0)
     Index Cond: (scale_data.section = 1::numeric)
     Filter: (scale_data.id2 = ($1)::numeric)
```

Predicate ID2 xuất hiện dưới dạng "Filter" bên dưới Index Scan operation. Điều này là vì PostgreSQL thực hiện table access như một phần của Index Scan operation. Nói cách khác, operation TABLE ACCESS BY INDEX ROWID của Oracle database bị ẩn bên trong PostgreSQL’s Index Scan operation. Vì vậy, một Index Scan có thể filter trên các column không nằm trong index.

> Important
>
> PostgreSQL Filter predicate là table level filter predicate — kể cả khi được hiển thị cho một Index Scan.

Khi thêm index từ chapter “Performance and Scalability”, ta thấy tất cả column đều xuất hiện dưới dạng “Index Cond” — bất kể chúng là access predicate hay filter predicate.

```
CREATE INDEX scale_slow
          ON scale_data (section, id1, id2);
```

Execution plan với index mới không hiển thị filter condition nào:

```
                        QUERY PLAN
-----------------------------------------------------
Aggregate  (cost=14215.98..14215.99 rows=1 width=0)
  Output: count(*)
  -> Index Scan using scale_slow on scale_data
     (cost=0.00..14208.51 rows=2989 width=0)
     Index Cond: (section = 1::numeric AND id2 = ($1)::numeric)
```

Lưu ý rằng condition trên ID2 không thể thu hẹp leaf node traversal vì index có column ID1 đứng trước ID2. Điều đó nghĩa là Index Scan sẽ scan toàn bộ range của condition SECTION=1::numeric và apply filter ID2=($1)::numeric trên từng row thỏa clause trên SECTION.

SQL Server

Method được mô tả trong section này áp dụng cho SQL Server Management Studio 2005 trở lên.

Getting an Execution Plan

Với SQL Server, có vài cách để lấy execution plan. Hai method quan trọng nhất là:

Graphically

Biểu diễn dạng graphical của SQL Server execution plan dễ truy cập trong Management Studio nhưng khó chia sẻ, vì thông tin predicate chỉ hiển thị khi di chuột qua operation tương ứng (“hover”).

Tabular

Execution plan dạng tabular khó đọc nhưng dễ copy vì hiển thị toàn bộ thông tin liên quan cùng lúc.

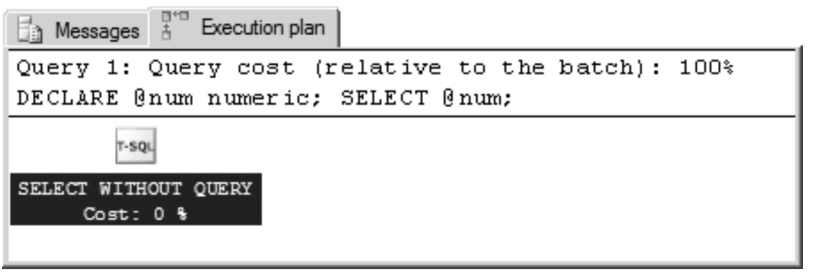
Graphically

Graphical explain plan được tạo bằng một trong hai button được highlight bên dưới.



Button bên trái explain statement đang được highlight trực tiếp. Button bên phải sẽ capture plan vào lần tiếp theo một SQL statement được execute.

Trong cả hai trường hợp, graphical representation của execution plan xuất hiện trong tab “Execution plan” của pane “Results”.



Graphical representation khá dễ đọc sau một chút thực hành. Tuy nhiên, nó chỉ hiển thị thông tin cơ bản nhất: các operation và table hoặc index mà chúng tác động lên.

Management Studio hiển thị thêm thông tin khi di chuột qua một operation (mouseover/hover). Điều này khiến việc chia sẻ execution plan với đầy đủ chi tiết trở nên khó khăn.

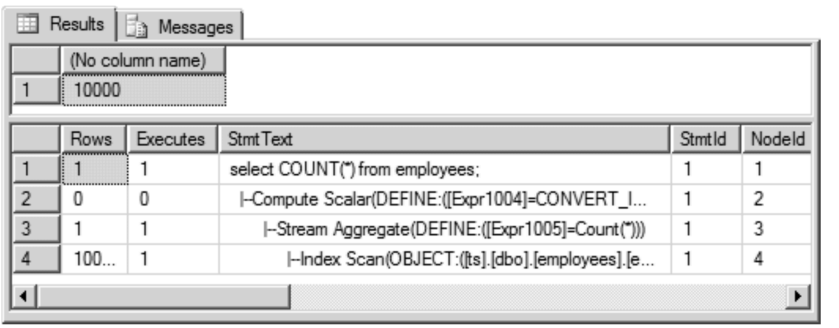
Tabular

Biểu diễn dạng tabular của SQL Server execution plan được lấy bằng cách profile việc execute một statement. Command sau bật chế độ đó:

```
SET STATISTICS PROFILE ON
```

Sau khi bật, mỗi statement được execute sẽ tạo thêm một result set. Chẳng hạn, select statement tạo ra hai result set — trước tiên là result của statement, sau đó là execution plan.

Execution plan dạng tabular hầu như không dùng được trong SQL Server Management Studio vì StmtText quá rộng để vừa màn hình.



Ưu điểm của cách biểu diễn này là có thể copy mà không làm mất thông tin quan trọng. Điều này rất tiện nếu bạn muốn đăng SQL Server execution plan lên forum hoặc nền tảng tương tự. Trong

trường hợp đó, thường chỉ cần copy column StmtText rồi format lại một chút:

```
select COUNT(*) from employees;
  |--Compute Scalar(DEFINE:([Expr1004]=CONVERT_IMPLICIT(...))
       |--Stream Aggregate(DEFINE:([Expr1005]=Count(*)))
            |--Index Scan(OBJECT:([employees].[employees_pk]))
```

Cuối cùng, bạn có thể tắt profiling lại:

```
SET STATISTICS PROFILE OFF
```

Operations

Index and Table Access

SQL Server có terminology đơn giản: operation “Scan” đọc toàn bộ index hoặc table, còn operation “Seek” dùng B-tree hoặc physical address (RID, giống Oracle ROWID) để truy cập một phần cụ thể của index hoặc table.

Index Seek

Index Seek thực hiện B-tree traversal và đi qua các leaf node để tìm mọi matching entry. Xem thêm “Anatomy of an Index” ở page 1.

Index Scan

Đọc toàn bộ index — tất cả row — theo index order. Tùy system statistics, database có thể thực hiện operation này nếu cần mọi row theo index order — chẳng hạn vì có order by clause tương ứng.

Key Lookup (Clustered)

Retrieve một row từ clustered index. Operation này tương tự Oracle INDEX UNIQUE SCAN cho một Index-Organized-Table (IOT). Xem thêm “Clustering Data” ở page 111.

RID Lookup (Heap)

Retrieve một row từ table — giống Oracle TABLE ACCESS BY INDEX ROWID. Xem thêm “Anatomy of an Index” ở page 1.

Table Scan

Còn được gọi là full table scan. Đọc toàn bộ table — tất cả row và column — theo cách chúng được lưu trên disk. Dù multi-block read operation có thể cải thiện đáng kể tốc độ Table Scan, đây vẫn là một trong những operation tốn kém nhất. Ngoài IO rate cao, Table Scan còn phải kiểm tra mọi table row nên có thể tiêu tốn nhiều CPU time. Xem thêm “Full Table Scan” ở page 13.

Join Operations

Nhìn chung, join operation chỉ xử lý hai table mỗi lần. Nếu query có nhiều join hơn, chúng được execute tuần tự: trước tiên là hai table, sau đó là intermediate result với table tiếp theo. Trong ngữ cảnh join, từ “table” vì vậy cũng có thể có nghĩa là “intermediate result”.

Nested Loops

Join hai table bằng cách lấy result từ một table rồi query table còn lại cho từng row của table thứ nhất. SQL Server cũng dùng nested loops operation để retrieve table data sau một index access. Xem thêm “Nested Loops” ở page 92.

Hash Match

Hash match join load các candidate record từ một phía của join vào hash table, sau đó probe hash table cho từng row từ phía còn lại của join. Xem thêm “Hash Join” ở page 101.

Merge Join

Merge join kết hợp hai sorted list như một zipper. Cả hai phía của join phải được presort. Xem thêm “Sort Merge” ở page 109.

Sorting and Grouping

Sort

Sort result theo order by clause. Operation này cần lượng memory lớn để materialize intermediate result (không pipelined). Xem thêm “Indexing Order By” ở page 130.

Sort (Top N Sort)

Sort một subset của result theo order by clause. Dùng cho top-N query nếu pipelined execution không thể thực hiện. Xem thêm “Querying Top-N Rows” ở page 143.

Stream Aggregate

Aggregate một presorted set theo group by clause. Operation này không buffer intermediate result — nó được execute theo pipelined manner. Xem thêm “Indexing Group By” ở page 139.

Hash Match (Aggregate)

Group result bằng hash table. Operation này cần lượng memory lớn để materialize intermediate result (không pipelined). Output không được order theo cách có ý nghĩa nào. Xem thêm “Indexing Group By” ở page 139.

Top-N Queries

Top

Abort các operation bên dưới khi đã fetch đủ số row mong muốn. Xem thêm “Querying Top-N Rows” ở page 143.

Hiệu quả của top-N query phụ thuộc vào execution mode của các operation bên dưới. Nó rất kém hiệu quả khi abort các non-pipelined operation như Sort.

Distinguishing Access and Filter-Predicates

Database SQL Server dùng ba method khác nhau để apply where clause (predicate):

Access Predicate (“Seek Predicates”)

Access predicate thể hiện start và stop condition của leaf node traversal.

Index Filter Predicate (“Predicates” hoặc “where” cho index operation)

Index filter predicate chỉ được apply trong lúc leaf node traversal. Chúng không đóng góp vào start và stop condition, đồng thời không thu hẹp scanned range.

Table level filter predicate (“where” cho table operation)

Predicate trên các column không thuộc index được evaluate ở table level. Để làm vậy, database trước hết phải load row từ heap table.

Section sau giải thích cách nhận diện filter predicate trong SQL Server execution plan. Nội dung dựa trên sample dùng để minh họa tác động của index filter predicate trong Chapter 3.

```
CREATE TABLE scale_data (
   section NUMERIC NOT NULL,
   id1     NUMERIC NOT NULL,
   id2     NUMERIC NOT NULL
);

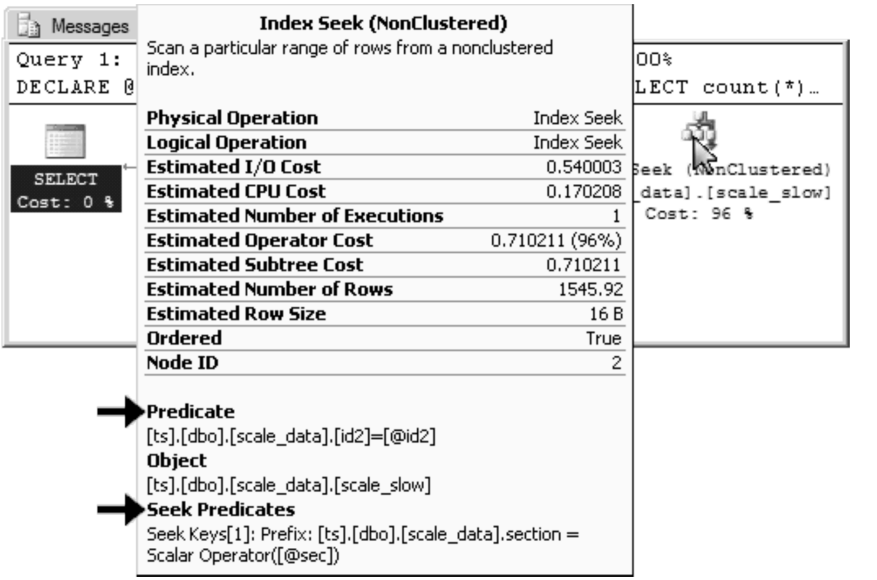
CREATE INDEX scale_slow ON scale_data(section, id1, id2);
```

Sample statement select theo SECTION và ID2:

```
SELECT count(*)
  FROM scale_data
 WHERE section = @sec
   AND id2 = @id2
```

In Graphical Execution Plans

Graphical execution plan ẩn thông tin predicate trong một tooltip chỉ xuất hiện khi di chuột qua Index Seek operation.



Seek Predicates của SQL Server tương ứng với access predicate của Oracle — chúng thu hẹp leaf node traversal. Filter predicate chỉ được gắn nhãn là Predicates trong graphical execution plan của SQL Server.

In Tabular Execution Plans

Tabular execution plan có thông tin predicate trong cùng column với nơi các operation xuất hiện. Vì vậy, rất dễ copy và paste toàn bộ thông tin liên quan trong một lần.

```
DECLARE @sec numeric;
DECLARE @id2 numeric;

SET STATISTICS PROFILE ON

SELECT count(*)
  FROM scale_data
 WHERE section = @sec
   AND id2 = @id2

SET STATISTICS PROFILE OFF
```

Execution plan xuất hiện dưới dạng result set thứ hai trong results pane. Sau đây là column StmtText — đã format lại một chút để dễ đọc:

```
|--Compute Scalar(DEFINE:([Expr1004]=CONVERT_IMPLICIT(...))
     |--Stream Aggregate(DEFINE:([Expr1008]=Count(*)))
          |--Index Seek(OBJECT:([scale_data].[scale_slow]),
             SEEK: ([scale_data].[section]=[@sec])
                   ORDERED FORWARD
             WHERE:([scale_data].[id2]=[@id2]))
```

Nhãn SEEK mở đầu cho access predicate, còn nhãn WHERE đánh dấu filter predicate.

MySQL

Method được mô tả trong section này áp dụng cho mọi version của MySQL.

Getting an Execution Plan

Đặt explain trước một SQL statement để lấy execution plan.

```
EXPLAIN SELECT 1;
```

Plan được hiển thị dưới dạng tabular (đã bỏ một số column ít quan trọng):

```
~+-------+------+---------------+------+~+------+------------~
~| table | type | possible_keys | key  |~| rows | Extra
~+-------+------+---------------+------+~+------+------------~
~| NULL  | NULL | NULL          | NULL |~| NULL | No tables...
~+-------+------+---------------+------+~+------+------------~
```

Thông tin quan trọng nhất nằm trong column TYPE. Dù tài liệu MySQL gọi đây là “join type”, tôi thích gọi nó là “access type” hơn vì thực tế nó chỉ cách data được truy cập. Ý nghĩa của từng type value được mô tả trong section tiếp theo.

Operations

Index and Table Access

Explain plan của MySQL dễ tạo cảm giác an toàn giả vì nói rất nhiều về việc index được sử dụng. Dù đúng về mặt kỹ thuật, điều đó không có nghĩa là index đang được dùng hiệu quả. Thông tin quan trọng nhất nằm trong column TYPE của MySQL explain output — nhưng ngay cả keyword INDEX ở đó cũng không cho biết việc indexing có đúng hay không.

eq_ref

Chỉ thực hiện B-tree traversal. Database dùng operation này khi primary key hoặc unique constraint bảo đảm search criteria match không quá một entry. Xem thêm “Anatomy of an Index” ở page 1.

ref, range

Thực hiện B-tree traversal và đi qua leaf node để tìm mọi matching entry (tương tự INDEX RANGE SCAN). Xem thêm “Anatomy of an Index” ở page 1.

index

Đọc toàn bộ index — tất cả row — theo index order (tương tự INDEX FULL SCAN).

ALL

Đọc toàn bộ table — tất cả row và column — theo cách chúng được lưu trên disk. Ngoài IO rate cao, table scan còn phải kiểm tra mọi row trong table nên có thể tạo tải CPU đáng kể. Xem thêm “Full Table Scan” ở page 13.

Using Index (trong column “Extra”)

Khi column “Extra” hiển thị “Using Index”, nghĩa là table không được access vì index có toàn bộ data cần thiết. Hãy nghĩ đến “using index ONLY”. Xem thêm “Clustering Data” ở page 111.

PRIMARY (trong column “key” hoặc “possible_keys”)

PRIMARY là tên của index được tự động tạo cho primary key.

Sorting and Grouping

using filesort (trong column “Extra”)

“using filesort” trong column Extra cho biết có explicit sort operation — bất kể sort diễn ra ở đâu (main memory hay disk). “Using filesort” cần lượng memory lớn để materialize intermediate result (không pipelined). Xem thêm “Indexing Order By” ở page 130.

Top-N Queries

implicit: không có “using filesort” trong column “Extra”

MySQL execution plan không hiển thị top-N query một cách rõ ràng. Nếu dùng limit syntax mà không thấy “using filesort” trong extra column, query được execute theo pipelined manner. Xem thêm “Querying Top-N Rows” ở page 143.

Distinguishing Access and Filter-Predicates

Database MySQL dùng ba cách khác nhau để evaluate where clause (predicate):

Access predicate (thông qua column “key_len”)

Access predicate thể hiện start và stop condition của leaf node traversal.

Index filter predicate (“Using index condition”, từ MySQL 5.6)

Index filter predicate chỉ được apply trong lúc leaf node traversal. Chúng không đóng góp vào start và stop condition, đồng thời không thu hẹp scanned range.

Table level filter predicate (“Using where” trong column “Extra”)

Predicate trên các column không thuộc index được evaluate ở table level. Để làm vậy, database trước hết phải load row từ table.

MySQL execution plan không cho biết condition nào dùng loại predicate nào — chúng chỉ liệt kê các loại predicate đang được sử dụng.

MySQL: Phân biệt Access và Filter Predicate

Trong ví dụ sau, toàn bộ where clause được dùng làm access predicate:

```
CREATE TABLE demo (
   id1 NUMERIC
 , id2 NUMERIC
 , id3 NUMERIC
 , val NUMERIC);

INSERT INTO demo VALUES (1,1,1,1);
INSERT INTO demo VALUES (2,2,2,2);

CREATE INDEX demo_idx
          ON demo
             (id1, id2, id3);

EXPLAIN
 SELECT *
   FROM demo
  WHERE id1=1
    AND id2=1;

+------+----------+---------+------+-------+
| type | key      | key_len | rows | Extra |
+------+----------+---------+------+-------+
| ref  | demo_idx | 12      |    1 |       |
+------+----------+---------+------+-------+
```

Trong column “Extra” không hiển thị “Using where” hoặc “Using index condition”. Tuy nhiên, index đã được sử dụng (type=ref, key=demo_idx), vì vậy có thể giả định toàn bộ where clause đủ điều kiện làm access predicate.

Có thể dùng key_len value để kiểm tra điều này. Nó cho biết query sử dụng 12 byte đầu tiên của index definition. Để map giá trị này với tên column, bạn “chỉ” cần biết mỗi column cần bao nhiêu storage space (xem “Data Type Storage Requirements” trong tài liệu MySQL). Khi không có NOT NULL constraint, MySQL cần thêm một byte cho

mỗi column. Sau cùng, mỗi NUMERIC column trong ví dụ cần 6 byte. Vì vậy, key length bằng 12 xác nhận hai index column đầu tiên được dùng làm access predicate.

Khi filter bằng column ID3 (thay vì ID2), MySQL 5.6 trở lên dùng index filter predicate (“Using index condition”):

```
EXPLAIN
 SELECT *
   FROM demo
  WHERE id1=1
    AND id3=1;

+------+----------+---------+------+-----------------------+
| type | key      | key_len | rows | Extra                 |
+------+----------+---------+------+-----------------------+
| ref  | demo_idx | 6       |    1 | Using index condition |
+------+----------+---------+------+-----------------------+
```

Trong trường hợp này, key length bằng six nghĩa là chỉ một column được dùng làm access predicate.

Các version MySQL trước đây dùng table level filter predicate cho query này — được nhận diện bởi “Using where” trong column “Extra”:

```
+------+----------+---------+------+-------------+
| type | key      | key_len | rows | Extra       |
+------+----------+---------+------+-------------+
| ref  | demo_idx | 6       |    1 | Using where |
+------+----------+---------+------+-------------+
```

Index

Symbols

2PC, *89*
?, :var, @var (see bind parameter)

A

Access Predicate, *44*
access predicates
  recognizing in execution plans
    Oracle, *170*
    PostgreSQL, *177*
    SQL Server, *185*
adaptive cursor sharing (Oracle), *75*
auto parameterization (SQL Server), *39*

B

B-tree (balanced search tree), *4*
between, *44*
bind parameter, *32*
  contraindications
    histograms, *34*
    `LIKE` filters, *47*
    partitions, *35*
  for execution plan caching, *32*
  type safety, *66*
bind peeking (Oracle), *75*
bitmap index, *50*
`Bitmap Index Scan` (PostgreSQL), *175*
Brewer's CAP theorem, *89*

C

CAP theorem, *89*
cardinality estimate, *27*
CBO (see optimizer, cost based)
clustered index, *122*
  transform to SQL Server heap table, *127*
clustering factor, *21*, *114*
  automatically optimized, *133*
clustering key, *123*
collation, *24*
commit
  deferrable constraints, *11*
  implicit for **`truncate table`**, *163*
  two phase, *89*
compiling, *18*
computed columns (SQL Server), *27*
constraint
  deferrable, *11*
  `NOT NULL`, *56*
cost value, *18*
`count(*)`
  often as index-only scan, *120*
  Oracle requires `NOT NULL` constraint, *57*
`COUNT STOPKEY`, *145*
cursor sharing (Oracle), *39*

D

data transport object (DTO), *105*
`DATE`
  efficiently working with, *62*
`DBMS_XPLAN`, *167*
`DEALLOCATE`, *174*
`DEFERRABLE` constraint, *11*
`DETERMINISTIC` (Oracle), *30*
`distinct`, *97*
`distinct()`
  in JPA and Hibernate, *97*
DML, *159*
doubly linked list, *2*
dynamic-update (Hibernate), *164*

E

eager fetching, *96*
eventual consistency, *89*
execution plan, *10*, *165*
  cache, *32*, *75*
  creating
    MySQL, *188*
    Oracle, *166*
    PostgreSQL, *172*
    SQL Server, *180*
  operations
    MySQL, *188*
    Oracle, *167*
    PostgreSQL, *174*
    SQL Server, *182*
`explain`
  MySQL, *188*
  Oracle, *166*
  PostgreSQL, *172*

F

FBI (see index, function-based)
`FETCH ALL PROPERTIES` (HQL), *105*
`fetch first`, *144*
filter predicates
  effects (chart), *81*
  recognizing in execution plans
    Oracle, *170*
    PostgreSQL, *177*
    SQL Server, *185*

full table scan, *13*
`All` (MySQL), *189*
`Seq Scan` (PostgreSQL), *174*
`TABLE ACCESS FULL` (Oracle), *168*
`Table Scan` (SQL Server), *183*
functions, *24*
deterministic, *29*
in partial indexes, *52*
window, *156*

G

`group by`, *139*
with PostrgesSQL and the Oracle database and an `ASC`/`DESC` index not pipelined, *140*

H

hash join, *101*
`HASH GROUP BY`, *169*
`HASH JOIN` (Oracle), *169*
`HASH Join` (PostgreSQL), *175*
`Hash Match`, *183*
`Hash Match (Aggregate)`, *184*
heap table, *3*, *122*
creating in SQL Server, *127*
Hibernate
eager fetching, *96*
`ILIKE` uses `LOWER`, *98*
updates all columns, *164*
hint, *19*

I

`IMMUTABLE` (PostgreSQL), *30*
index
covering, *117*
fulltext, *48*
function-based, *24*
case insensitive, *24*
to index mathematical calculations, *77*
join, *50*
limits
MySQL, Oracle, PostgreSQL, *121*
SQL Server, *122*
merge, *49*
multi-column, *12*
wrong order (effects), *81*
partial, *51*
prefix (MySQL), *121*
secondary, *123*
`index` in MySQL execution plans, *189*
index-only scan, *116*
index-organized table, *122*
database support, *127*
`Index Cond` (PostgreSQL), *177*
`INDEX FAST FULL SCAN`, *168*
`INDEX FULL SCAN`, *168*
`Index Only Scan` (PostgreSQL), *174*
`INDEX RANGE SCAN`, *167*
`Index Scan` (PostgreSQL), *174*
`Index Seek`, *182*
`INDEX UNIQUE SCAN`, *167*
when accessing an IOT, *124*
`INTERNAL_FUNCTION`, *67*
IOT (index-organized table), *122*

J

join, *91*
full outer, *109*

K

`Key Lookup (Clustered)`, *182*

L

lazy fetching
for scalar attributes (columns), *104*
leaf node, *2*
split, *160*
LIKE, *45*
alternatives, *48*
as index filter predicate, *112*
on DATE column, *67*
on `DATE` columns, *67*
`limit` (MySQL, PostgreSQL), *144*
logarithmic scalability, *7*
`LOWER`, *24*

M

`Merge Join`, *109*
PostgreSQL, *175*
SQL Server, *183*
`MERGE JOIN` (Oracle), *169*
multi-block read
for a full table scan, *13*
for a `INDEX FAST FULL SCAN`, *168*
MVCC, *163*
affects PostgreSQL index-only scan, *174*
myths
dynamic SQL is slow, *72*, *74*
most selective column first
disproof, *43*
origin, *49*
Oracle cannot index `NULL`, *56*

N

N+1 problem, *92*
`Nested Loops`, *92*
PostgreSQL, *175*
SQL Server, *183*
`NESTED LOOPS` (Oracle), *168*

NOSORT
SORT GROUP BY, *140*
WINDOW, *157*
NULL
indexing in Oracle, *54*

O

offset (MySQL, PostgreSQL), *148*
optimizer, *18*
cost based, *18*
hint, *19*
rule based, *18*
statistics, *21*
OPTIMIZE FOR (SQL Server), *76*
OPTION (SQL Server), *76*
OR
to disable filters, *72*
order by, *130*
ASC, DESC, *134*
NULLS FIRST/LAST, *137*
support matrix, *138*
OVER(), *156*

P

paging, *147*
offset method, *148*
seek method, *149*
approximated, *152*
parameter sniffing (SQL Server), *76*
parsing, *18*
partial index, *51*
partial objects (ORM), *104*
partitions and bind parameters, *35*
pipelining, *92*
PLAN_TABLE, *166*
predicate information, *20*
access vs. filter predicates, *44*
in execution plans
MySQL, *190*
Oracle, *170*
SQL Server, *185*
prepare (PostgreSQL), *172*
primary key w/o unique index, *11*

Q

query planner (see optimizer)

R

RBO (see optimizer, rule based)
RECOMPILE (SQL Server hint), *76*
result set transformer, *98*
RID, *3*
RID Lookup (Heap), *183*
root node, *5*
split, *160*
row sequencing, *113*
row values, *153*
ROWID, *3*
ROWNUM (Oracle pseudo column), *144*, *148*
ROW_NUMBER, *156*

S

scalability, *81*
horizontal, *87*
logarithmic, *7*
Scalability, *79*
Seek Predicates (SQL Server), *185*
select *, avoid to
enable index-only scans, *120*
improve hash join performance, *104*
Seq Scan, *174*
Sort (SQL Server), *184*
SORT GROUP BY, *169*
NOSORT, *140*
SORT ORDER BY, *130*
STOPKEY, *145*
SQL area, *75*
SQL injection, *32*
SSD (Solid State Disk), *90*
statistics, *21*
for Oracle function-based indexes, *28*
STATISTICS PROFILE, *181*
STOPKEY
COUNT, *145*
SORT ORDER BY, *146*
WINDOW, *157*
Stream Aggregate, *184*

T

top (SQL Server), *145*
Top-N Query, *143*
TO_CHAR(DATE), *66*
TRUNC(DATE), *62*
truncate table, *163*
triggers not executed, *163*

U

UPPER, *24*

V

Vaccum (PostgreSQL), *163*
virtual columns for NOT NULL constraints on FBI, *58*

W

where, *9*
conditional, *72*
in SQL Server execution plan, *187*
window functions, *156*

SQL Performance Explained! — Giờ thì sao?

Có thể bạn vẫn còn câu hỏi hoặc gặp một vấn đề rất cụ thể mà “SQL Performance Explained” chưa trả lời thỏa đáng? Instant Coaching là giải pháp cho bạn.

Instant Coaching

Instant Coaching là cách nhanh, dễ và không phiền phức để developer giải quyết các vấn đề khó về SQL database performance thông qua desktop sharing.

Instant Coaching không dùng các ví dụ dựng sẵn; các case thực tế được trình bày vì đó là cách tối ưu để hỗ trợ bạn giải quyết vấn đề hiện tại với SQL database chậm. Instant Coaching độc lập với vendor, hiệu quả… và có mức giá rất hợp lý!

Hãy thử ngay!

Đây là lúc tìm hiểu thêm về Instant Coaching! Hãy truy cập website của chúng tôi để biết thêm thông tin:

http://winand.at

SQL Performance Explained

SQL Performance Explained giúp developer cải thiện database performance. Trọng tâm là SQL — bao quát mọi SQL database chính mà không sa đà vào chi tiết của một product cụ thể nào.

Bắt đầu từ nền tảng về indexing và where clause, SQL Performance Explained dẫn developer đi qua mọi thành phần của một SQL statement và giải thích các vấn đề thường gặp của object-relational mapping (ORM) tool như Hibernate.

Các chủ đề được đề cập gồm:

» Sử dụng multi-column index

» Áp dụng SQL function đúng cách

» Sử dụng LIKE query hiệu quả

» Tối ưu join operation

» Clustering data để cải thiện performance

» Pipelined execution của order by và group by

» Đạt performance tốt nhất cho pagination query

» Hiểu khả năng scalability của database

Cấu trúc có hệ thống khiến SQL Performance Explained vừa là textbook vừa là reference manual mà mọi developer nên có trên bookshelf.

Bao quát

Oracle® Database · SQL Server® · MySQL · PostgreSQL

Về Markus Winand

Markus Winand đã phát triển SQL application từ năm 1998. Mối quan tâm chính của ông gồm performance, scalability, reliability và nói chung là mọi khía cạnh kỹ thuật khác của software quality. Hiện Markus làm trainer và coach độc lập tại Vienna, Austria.

http://winand.at/

ISBN 978-3-9503078-2-5



EUR 29.95
GBP 26.99

9 783950 307825 >